



上海交通大学学士学位论文

基于动作扩散策略的
足式机器人视觉目标导航与探索

姓 名：陈 样

学 号：522031910056

导 师：雷华明

院 系：自动化与感知学院

学 科 / 专 业：智能感知工程

2026 年 5 月 2 日

**A Dissertation Submitted to
Shanghai Jiao Tong University for the Degree of Bachelor**

**VISUAL GOAL NAVIGATION AND EXPLORATION
FOR LEGGED ROBOTS BASED ON ACTION
DIFFUSION POLICY**

Author: Chen Yang

Supervisor: Lei Huaming

School of Automation and Intelligent Sensing

Shanghai Jiao Tong University

Shanghai, P.R. China

May 2nd, 2026

上海交通大学

学位论文原创性声明

本人郑重声明：所呈交的学位论文，是本人在导师的指导下，独立进行研究工作所取得的成果。除文中已经注明引用的内容外，本论文不包含任何其他个人或集体已经发表或撰写过的作品成果。对本文的研究做出重要贡献的个人和集体，已在文中以适当方式予以致谢。若在论文撰写过程中使用了人工智能工具，本人已遵循《上海交通大学关于在教育教学中使用 AI 的规范》，确保人工智能生成内容的应用场景、引用范围及标注方式均符合规定，并杜绝学术不端行为。本人完全知晓本声明的法律后果由本人承担。

学位论文作者签名：

日期： 年 月 日

上海交通大学

学位论文使用授权书

本人同意学校保留并向国家有关部门或机构送交论文的复印件和电子版，允许论文被查阅和借阅。

本学位论文属于：

☐ 公开论文

☐ 内部论文，保密 ☐ 1 年 / ☐ 2 年 / ☐ 3 年，过保密期后适用本授权书。

☐ 秘密论文，保密 ____ 年（不超过 10 年），过保密期后适用本授权书。

☐ 机密论文，保密 ____ 年（不超过 20 年），过保密期后适用本授权书。

（请在以上方框内选择打“√”）

学位论文作者签名：

指导教师签名：

日期： 年 月 日

日期： 年 月 日

摘 要

具身智能的发展正在推动机器人从实验室走向复杂开放的真实场景。视觉目标导航作为连接感知与决策的核心环节，要求机器人在弱先验条件下根据图像目标完成自主移动。近年来，NoMaD 等通用导航模型将扩散策略引入视觉导航，通过条件扩散过程生成多模态局部轨迹，展现了强大的表达能力。但现有验证大多围绕轮式平台展开，面向足式机器人的部署研究仍相对有限。

本文以 NoMaD 为高层视觉导航核心，以云深处 Lite3 四足机器人为目标平台，围绕“基于动作扩散策略的足式机器人视觉目标导航与探索”展开研究。主要工作包含三方面：第一，在离线层面对 NoMaD 进行部署导向优化，研究了 DDIM 推理加速、Classifier-Free Guidance 引导增强、Test-Time Scaling 推理时搜索与视觉编码器升级，建立了统一的离线统计实验链路；第二，构建了由统一推理模块、状态机、导航主机、中层控制映射与平台接口组成的分层闭环系统，并在 MuJoCo 仿真中完成了 Lite3 四足导航、探索与多目标任务的系统级验证；第三，形成了面向 Jetson Orin 与 Lite3 的真机部署方案，包括分阶段联调流程与多级安全机制。实验结果表明，DDIM-2 配置在保持轨迹质量的同时实现约 4.84 倍加速，MuJoCo 闭环验证中所有导航任务均成功到达目标，Orin 阶段一独立调试已通过验证。本文将原本面向轮式平台的 NoMaD 扩展为面向四足平台的可分析、可仿真、可部署的完整研究链路。

关键词：视觉导航，扩散策略，足式机器人，NoMaD，DDIM，四足平台

ABSTRACT

The development of embodied intelligence is driving robots from laboratory settings into complex open-world scenarios. Visual goal navigation, as a core component connecting perception and decision-making, requires robots to achieve autonomous movement toward image-specified goals under weak prior conditions. Recently, general navigation models such as NoMaD have introduced diffusion policies into visual navigation, generating multi-modal local trajectories through conditional diffusion processes with strong expressiveness. However, existing validations primarily focus on wheeled platforms, and deployment research for legged robots remains limited.

This thesis takes NoMaD as the high-level visual navigation core and the DeepRobotics Lite3 quadruped robot as the target platform, conducting research on “visual goal navigation and exploration for legged robots based on action diffusion policy.” The main contributions span three aspects. First, deployment-oriented optimizations are performed on NoMaD at the offline level, including DDIM inference acceleration, Classifier-Free Guidance enhancement, Test-Time Scaling search, and visual encoder upgrades, establishing a unified offline statistical experiment pipeline. Second, a hierarchical closed-loop system comprising a unified inference module, state machine, navigation host, mid-level control mapping, and platform interface is constructed, with system-level verification completed in MuJoCo simulation for Lite3 quadruped navigation, exploration, and multi-goal tasks. Third, a real-robot deployment scheme targeting Jetson Orin and Lite3 is developed, including staged commissioning procedures and multi-level safety mechanisms. Experimental results show that the DDIM-2 configuration achieves approximately $4.84\times$ speedup while maintaining trajectory quality, all navigation tasks in MuJoCo closed-loop verification successfully reach their goals, and Orin Stage-1 standalone validation has passed. This work extends NoMaD from a wheeled-platform framework to an analyzable, simulatable, and deployable research pipeline for quadruped platforms.

Key words: visual navigation, diffusion policy, legged robot, NoMaD, DDIM, quadruped platform

目 录

第 1 章 绪论	1
1.1 研究背景.....	1
1.2 研究意义.....	2
1.3 国内外研究现状.....	3
1.3.1 传统导航与学习型导航	3
1.3.2 视觉目标导航与拓扑导航	3
1.3.3 扩散模型、扩散策略与通用导航模型	3
1.3.4 足式平台上的视觉导航研究.....	4
1.4 本文结构安排.....	4
第 2 章 预备知识	5
2.1 导航与视觉导航基础.....	5
2.1.1 导航任务的基本定义.....	5
2.1.2 视觉导航与目标导航.....	5
2.1.3 拓扑导航与 topomap	6
2.2 扩散模型与扩散策略.....	6
2.2.1 扩散模型基本原理	6
2.2.2 DDPM 与 DDIM.....	8
2.2.3 Classifier-Free Guidance	10
2.2.4 扩散策略与 Test-Time Scaling	10
2.2.5 扩散模型、ViT 与导航建模的关系	12
2.3 NoMaD 机制.....	12
2.3.1 NoMaD 的基本结构.....	12
2.3.2 距离预测头与视觉编码器的直接耦合	14
2.3.3 goal mask 与导航/探索统一建模.....	14
2.4 轮式平台、足式平台与 Lite3	15
2.4.1 轮式与足式平台的部署差异.....	15
2.4.2 云深处 Lite3 平台与关键参数	15
第 3 章 算法设计与实验.....	18
3.1 任务定义与研究思路.....	18
3.1.1 问题建模.....	18
3.1.2 Waypoint 表示的优势分析	19
3.2 NoMaD 基础模型复现.....	19
3.2.1 输入输出形式.....	19
3.2.2 视觉编码器架构.....	20
3.2.3 Token 构建与条件拼接.....	21

3.2.4 模型结构与训练目标.....	22
3.2.5 推理链路的统一封装.....	22
3.3 数据集与实验设备.....	23
3.3.1 GoStanford 数据集.....	23
3.3.2 数据预处理流程.....	23
3.3.3 感知输入与设备说明.....	24
3.4 基线实验.....	24
3.4.1 基线推理可视化.....	25
3.5 实验指标与统计原则.....	26
3.5.1 指标的形式化定义.....	26
3.5.2 统计显著性与多指标分析原则.....	27
3.6 DDIM 推理加速实验.....	28
3.6.1 理论动机.....	28
3.6.2 实验设计.....	28
3.6.3 实验结果.....	29
3.6.4 详细分析.....	29
3.6.5 结果分析.....	29
3.7 CFG 引导增强实验.....	30
3.7.1 理论分析.....	30
3.7.2 实验设计.....	30
3.7.3 实验结果与分析.....	31
3.8 TTS 推理时搜索实验.....	32
3.8.1 实验设计.....	32
3.8.2 评分函数设计.....	32
3.8.3 计算预算分配策略.....	33
3.8.4 实验组织方式.....	33
3.8.5 结果与分析.....	33
3.9 视觉编码器升级实验.....	35
3.9.1 实验目标.....	35
3.9.2 候选编码器结构与对比.....	35
3.9.3 特征维度对齐.....	36
3.9.4 训练配置.....	36
3.9.5 实验结果与分析.....	36
3.10 联合结论.....	36
3.10.1 优化方法之间的关系.....	38
3.10.2 消融验证与编码器联合补充.....	40
3.10.3 最终算法框架确定.....	42
3.11 本章小结.....	43

第 4 章 算法框架设计与仿真环境验证	44
4.1 本章目标	44
4.2 统一闭环系统架构	44
4.2.1 分层设计	44
4.2.2 统一推理模块	45
4.3 状态机与导航主机设计	45
4.3.1 状态机职责	45
4.3.2 导航主机职责	46
4.3.3 中层控制映射	47
4.4 MuJoCo 仿真平台设计	47
4.4.1 选择 MuJoCo 的原因	47
4.4.2 仿真中的系统组成	48
4.4.3 场景设计与物理参数	48
4.4.4 仿真任务类型	49
4.5 MuJoCo 闭环结果分析	49
4.5.1 仿真完整性复核	49
4.5.2 仿真实验	50
4.5.3 结果分析	52
4.6 本章小结	52
第 5 章 真机实验	53
5.1 本章目标	53
5.2 真机系统的双链路设计	53
5.2.1 横向实时推理链路：从图像到 twist 指令	54
5.2.2 纵向分层控制链路：从 NoMaD 到运控	55
5.3 Orin 真机部署架构	56
5.3.1 边缘计算平台的角色	56
5.3.2 真机执行链路 with 资源约束	56
5.3.3 真机桥接层	57
5.3.4 真机部署中的时序等待机制	58
5.4 真机部署流程	60
5.4.1 阶段一：Orin 端独立调试	60
5.4.2 阶段一实测结果与分析	61
5.4.3 阶段二：Orin 与 Lite3 联合调试	62
5.5 真实 topomap 采集与滑动窗口导航流程	62
5.5.1 导航输入与实时子目标	63
5.5.2 真实 topomap 的采集与组织	63
5.5.3 滑动窗口导航机制	64
5.6 导航模式、探索模式与图像记录机制	65
5.6.1 导航模式与探索模式	65

5.6.2 可视化模式与无可可视化模式.....	65
5.7 安全机制.....	66
5.8 真机闭环实验结果.....	66
5.8.1 真实场景 topomap 采集.....	66
5.8.2 多场景目标导航实验.....	67
5.8.3 多场景探索实验.....	69
5.9 本章小结.....	70
第 6 章 全文总结与未来展望.....	71
6.1 全文工作总结.....	71
6.2 主要贡献.....	72
6.3 研究不足.....	72
6.4 未来展望.....	73
参考文献.....	75
致 谢.....	78

符号对照表

NoMaD	Goal Masked Diffusion Policies for Navigation and Exploration
DDPM	Denoising Diffusion Probabilistic Model
DDIM	Denoising Diffusion Implicit Model
CFG	Classifier-Free Guidance
TTS	Test-Time Scaling
GNM	General Navigation Model
ViNT	Visual Navigation Transformer
ViT	Vision Transformer
DiT	Diffusion Transformer
DINOv2	Self-Distillation with No Labels v2
MuJoCo	Multi-Joint dynamics with Contact
SLAM	Simultaneous Localization and Mapping
PD	Proportional-Derivative (Controller)
topomap	Topological Map (拓扑地图)
waypoint	局部路径点
O_t	时刻 t 的历史视觉观察序列
I_g	目标图像或拓扑节点图像
$\mathbf{A}_t$	未来局部 waypoint 轨迹
$\mathbf{a}_i$	第 i 个 waypoint, $(\Delta x_i, \Delta y_i)$
H	轨迹预测 horizon (waypoint 数量)
β_t	扩散前向过程第 t 步噪声调度系数
$\bar{\alpha}_t$	累计噪声调度系数
ϵ	真实高斯噪声
ϵ_θ	模型预测噪声
w	CFG guidance scale
N	TTS 候选轨迹数量
$\hat{d}_j$	距离预测头对第 j 个候选节点的距离估计
cmd_vel	速度控制指令

UDP	User Datagram Protocol
Orin	NVIDIA Jetson Orin 边缘计算平台
Lite3	云深处 Lite3 四足机器人平台

第1章 绪论

1.1 研究背景

具身智能的发展正在推动机器人从规则明确的实验室环境走向开放、动态且弱先验的真实场景。无论是园区巡检还是室内导览，机器人都需要在地图不完备、定位弱和任务目标表达灵活的条件下完成自主移动。导航不再只是“从起点到终点”的几何规划问题，而是连接感知、决策与执行的核心环节^[1]。

传统导航方法通常遵循“感知建图、定位估计、路径规划、运动控制”的经典链路，在结构化环境中具有较高精度和可解释性^[2-3]。但当任务目标以图像、语义对象或语言指令给出时，传统方法对高质量地图、精确标定和稳定环境的依赖会迅速上升^[4]。这使得视觉导航逐渐成为机器人自主移动研究中的重要方向：机器人直接利用图像观察环境，并通过学习方法把实时图像和目标图像转化为可执行的局部行为。

从技术发展历程来看，视觉导航经历了多个阶段。早期方法以光流、特征点匹配为核心。之后端到端策略学习开始被引入导航系统，研究者尝试直接从图像像素到控制量的端到端映射^[5]。然而，这类方法在泛化能力和可解释性方面仍面临挑战。近年来，预训练视觉模型（如 ViT、DINOv2）的出现为视觉导航提供了更强大的视觉工具，这使得跨场景迁移和零样本适应成为可能^[6-7]。

近年来，通用导航模型与生成式策略的结合进一步推动了该方向的发展。GNM 与 ViNT 将图像观察、目标条件和局部轨迹预测统一到同一模型中^[8-9]；NoMaD 在此基础上引入扩散策略，把未来局部轨迹视为条件生成问题来建模^[10-11]。导航中的决策空间是多模态的，即在同一观察下可能存在多条合理路径。相较于回归方法给出单步控制量，生成式局部轨迹能够更自然、更多样化的给出表达。

另一方面，现有 NoMaD 相关实机部署主要围绕轮式平台展开。轮式平台姿态变化较小、视觉抖动弱，部署简单，但在门槛、坡面、碎石地、狭窄通道等复杂场景中，足式平台尤其是四足机器人具有更强的地形适应性与通过能力。与此同时，足式平台也带来了更强的视觉扰动、更严格的实时性约束和更复杂的高层到低层接口问题^[12-13]。因此，将 NoMaD 从轮式验证场景迁移到足式平台，并构建一套真机部署的完整系统，具有明确的研究价值与工程意义。

基于上述背景，本文以 *visualnav-transformer* 项目为基础，以 NoMaD 为高层核心算法，以云深处 Lite3 四足机器人为主要部署对象，围绕“基于动作扩散策略的足

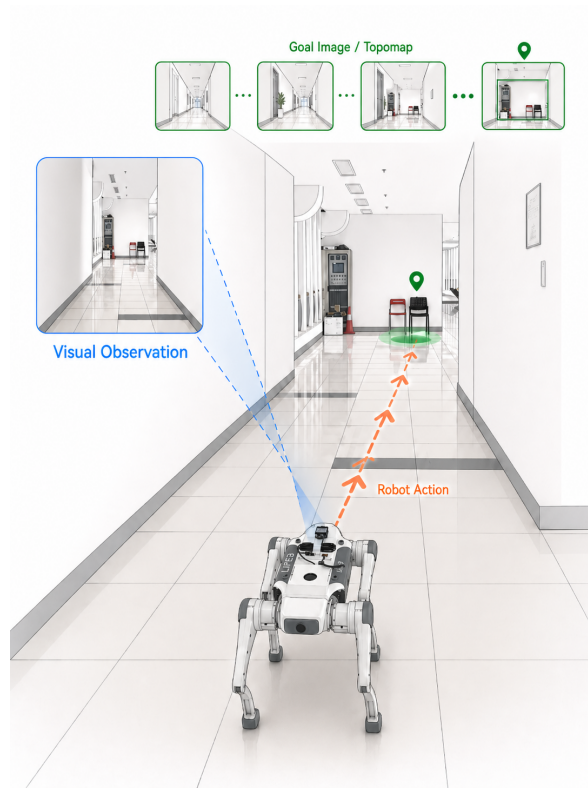


图 1.1 研究背景与目标场景示意图

式机器人视觉目标导航与探索”开展研究。

1.2 研究意义

本文研究的意义主要体现在三方面。在方法层面，本文将 DDIM 采样加速、Classifier-Free Guidance 条件增强与 Test-Time Scaling 推理时搜索放在同一统计实验框架下进行对比，并系统梳理三者的互补关系与层级组合方式。这些方法本身并非本文首创，但将它们应用到面向足式导航的部署场景下并给出实测数据，对扩散策略在机器人视觉导航中的工程落地具有参考价值。

在系统层面，本文构建了由统一推理模块、状态机、导航主机到中层控制映射与运控接口的分层闭环架构，使高层扩散策略能够以较小改动从轮式平台迁移到 Lite3 四足平台。这一架构也为其他足式平台的后续扩展提供了可复用的接口规范。

在应用层面，面向 Jetson Orin 与 Lite3 的真机部署方案使得扩散策略视觉导航从实验室走向落地可用。

1.3 国内外研究现状

1.3.1 传统导航与学习型导航

机器人导航研究大体经历了从几何建模到学习建模的发展过程。以 SLAM 为核心的传统方法强调通过传感器构建环境几何表示，再在地图上进行定位与规划^[1-2]。这一路线在静态、结构化环境中成熟可靠，但对地图质量和传感器一致性要求较高。随着深度学习的发展，直接从图像观察中学习导航策略的方法开始被探索，模型被训练以绕开显式几何建模，直接建立观察与行为之间的映射^[4-5]。学习型导航相比于传统方法的优势在于跨场景迁移潜力较强，但也更依赖训练数据分布、模型结构与部署链路的一致性。本工作重点关注学习型高层视觉导航策略如何进入真实系统。

1.3.2 视觉目标导航与拓扑导航

视觉目标导航关注机器人如何根据目标图像、目标对象或语言描述生成导航行为^[5,14]。与 point-goal 导航^[15]相比，image-goal 任务更接近真实应用，因为用户更容易给出“去这张图对应的位置”而不是“去某个绝对坐标”。但这也要求模型具备更强的视觉匹配与局部规划能力。

在视觉目标导航的实现路径上，研究者提出了多种技术方案。基于度量地图的方法通过构建显式空间表示来规划路径，但在大规模环境中面临存储和计算开销问题。基于语义地图的方法利用语义信息组织空间知识，但依赖准确的语义分割模型。基于端到端的方法直接从视觉输入映射到动作输出，实现简洁但可解释性较弱。

为了降低对全局精确地图的依赖，多种拓扑导航与视觉记忆方法^[16-17]被提出。其基本思想不是恢复高精度几何地图，而是保留若干具有代表性的视觉节点并按空间顺序串成有序序列，再让导航策略沿这一序列逐节推进，从而把长程导航拆解为一系列局部子目标。ViNG、SPTM 等方法都验证了这种思路在真实视觉导航中的有效性^[14,16]。拓扑导航的核心优势在于其对环境变化的鲁棒性：即使局部区域发生变化，只要拓扑序列大致保持，系统仍可通过剩余节点完成导航。本文同样采用 topomap 作为系统中的视觉记忆载体。

1.3.3 扩散模型、扩散策略与通用导航模型

扩散模型最初在图像生成领域取得成功，其核心优势在于能够通过逐步去噪建模复杂多模态分布^[18-19]。随后，Diffuser 与 Diffusion Policy 将这一思想引入规划与控制领域，把未来轨迹或动作序列视为生成对象，从而显著提升了复杂决策任务中的表达能力^[11,20]。在视觉建模方面，ViT 和 DiT 等工作进一步说明了 Transformer 与扩散

建模结合潜力^[6-7]。

在机器人导航领域，GNM、ViNT 与 NoMaD 构成了较清晰的发展主线。GNM 强调统一的图像观察与目标表征^[8]；ViNT 利用 Transformer 结构处理时序观察与目标条件^[9]；NoMaD 则引入动作扩散策略，并通过 goal mask 机制把目标导航与探索统一到同一模型中^[10]。这使其非常适合被用作本文的研究起点。

1.3.4 足式平台上的视觉导航研究

相较于轮式平台，足式机器人在复杂地形上更具优势，但视觉导航部署难度也明显更高。机体起伏、足端冲击和机载相机视角波动会削弱视觉输入稳定性；同时，足式平台往往需要更严格的实时控制、更明确的安全停机机制和更清晰的高层到中低层控制接口^[12-13]。现有大量足式研究聚焦于低层运动控制与强化学习行走能力：Miki 等^[21]通过端到端学习把外感知与本体感知融合，使四足机器人能够在野外复杂地形上稳健行走；视觉导航与足式运动结合的代表性工作如 ViNL^[22]则把视觉导航策略与视觉运动控制策略协同部署到四足平台上，使其在跨越障碍物的同时完成室内导航。但将高层视觉目标导航稳定部署到四足平台上的公开系统仍相对有限。

1.4 本文结构安排

本文按“预备知识、算法设计、系统设计、仿真验证、真机实验”的主线组织，共分六章：(1) 第一章为绪论，介绍研究背景、研究意义、相关工作和本文的研究目标；(2) 第二章为预备知识，说明视觉导航、目标导航、扩散模型、扩散策略、NoMaD 关键机制以及 Lite3 平台相关基础；(3) 第三章为算法设计与实验，围绕 NoMaD 复现、推理优化与视觉编码器替换展开离线实验分析；(4) 第四章为算法框架设计与仿真环境验证，介绍推理模块、状态机、导航主机及 MuJoCo 闭环验证；(5) 第五章为真机实验，给出 Orin 上的 Lite3 部署流程、运行方式和实验结果；(6) 第六章为总结与展望，归纳全文工作并讨论后续研究方向。

第 2 章 预备知识

本章介绍后续算法设计、系统实现与部署验证所需的共同基础，重点包括导航任务形式、视觉目标导航、扩散模型与扩散策略、NoMaD 的关键机制，以及轮式平台与足式平台在部署上的差异。

2.1 导航与视觉导航基础

2.1.1 导航任务的基本定义

机器人导航的目标是在给定观察、任务目标和运动约束的条件下，使机器人从当前状态移动到目标区域。一个完整导航系统通常涵盖环境感知、状态表示、目标表达、局部决策与运动执行五个环节^[1,4]。在输出形式上，高层导航策略可以输出底层控制量、局部路径点或局部轨迹序列；本文让高层模型输出局部 waypoint 轨迹，再由中层控制器完成执行映射。

如果把高层策略写成函数形式，则有

$$\pi : (O_t, g) \mapsto \mathbf{A}_t, \quad (2.1)$$

其中 O_t 表示时刻 t 的图像观察或观察序列， g 表示任务目标， $\mathbf{A}_t$ 表示未来短时间内的局部轨迹。该表示天然适合与不同平台解耦，也是本文能够将高层视觉导航从轮式框架迁移到 Lite3 四足平台的重要原因。

观察空间方面，本文统一使用 RGB 图像作为输入，并保留最近 $K + 1$ 帧上下文 $O_t = \{I_{t-K}, \dots, I_t\}$ 。每个 waypoint $\mathbf{a}_i = (\Delta x_i, \Delta y_i)$ 表示相对于机器人当前位姿的二维增量位移（ x 前向、 y 左侧），策略输出因此不依赖全局定位。执行上采用闭环模式：每个高层周期生成长度 $H = 8$ 的 waypoint 序列，固定取第 $k = 2$ 个 waypoint 作为本次控制周期的局部目标位移，由中层 PD 控制器映射为单条 Twist 速度指令；下一周期再基于最新相机帧重新生成。这种“规划长 8 步、每周期只执行其中一个超前点”的策略与模型预测控制（MPC）一脉相承，扩散策略相当于把 MPC 的优化求解替换为条件扩散采样。

2.1.2 视觉导航与目标导航

视觉导航以图像作为主要环境输入，并基于视觉信息完成导航决策^[4-5]。目标导航则进一步关注目标的给出形式，常见类别包括：

- **point-goal 导航**^[15]: 目标以坐标点形式给出, 要求机器人到达指定位置;
- **image-goal 导航**: 目标以一张或多张图像给出, 要求机器人到达图像所对应的位置;
- **object-goal 导航**: 目标以物体类别给出 (如“找到冰箱”), 需要语义理解能力;
- **language-goal 导航**: 目标以自然语言指令给出, 是最灵活但也最具挑战的形式。

本文研究 image-goal 场景: 相较于绝对坐标, 图像目标更符合真实应用——用户更容易给出“去这个画面对应的位置”, 而非统一坐标系下的精确终点; 相较 object-goal 与 language-goal, 图像目标无需语义理解模块、表达歧义更低, 能在无 GPS 与无预建地图的场景中直接工作。

2.1.3 拓扑导航与 topomap

为了降低对精确几何地图的依赖, 拓扑导航只保留若干代表性视觉节点及其连通关系^[14,16-17]。本文系统中 topomap 不是部署时由编码器在线生成的图结构, 而是人工沿目标路径预先采集的一段**有序**视觉节点序列, 文件名按 0.png, 1.png, ... 自然排序, 节点编号同时编码“沿路径前进的方向”, 因此 topomap 在使用上更接近一条按空间顺序排好的导航队列。

设拓扑节点图像沿路径排列为 $\mathcal{G} = (I_g^{(0)}, \dots, I_g^{(M-1)})$, 导航任务给定起点附近索引 0 与目标索引 $g \leq M-1$ 。系统维护一个状态化的“当前定位”指针 c_t , 距离预测头只在以 c_t 为中心、半径 $r = 4$ 的局部窗口

$$\mathcal{W}_t = \{j : \max(c_t - r, 0) \leq j \leq \min(c_t + r + 1, g)\} \quad (2.2)$$

内对当前观察 O_t 与每个候选节点 $I_g^{(j)}$ 计算距离预测 $\hat{d}_j$, 并据此更新定位:

$$c_{t+1} = \text{clip}(\arg \min_{j \in \mathcal{W}_t} \hat{d}_j, c_t - 1, c_t + \Delta_{\max}), \quad (2.3)$$

其中 clip 操作确保定位指针每周期至多前进 $\Delta_{\max} = 3$ 个节点、至多回退 1 个节点, 从而强制定位整体沿 topomap 单向推进。

2.2 扩散模型与扩散策略

2.2.1 扩散模型基本原理

扩散模型通过“逐步加噪、逐步去噪”的方式学习复杂分布^[18]。在前向过程里, 数据样本 x_0 被逐步扰动为高斯噪声:

$$q(x_t | x_{t-1}) = \mathcal{N}(\sqrt{1 - \beta_t}x_{t-1}, \beta_t \mathbf{I}), \quad (2.4)$$

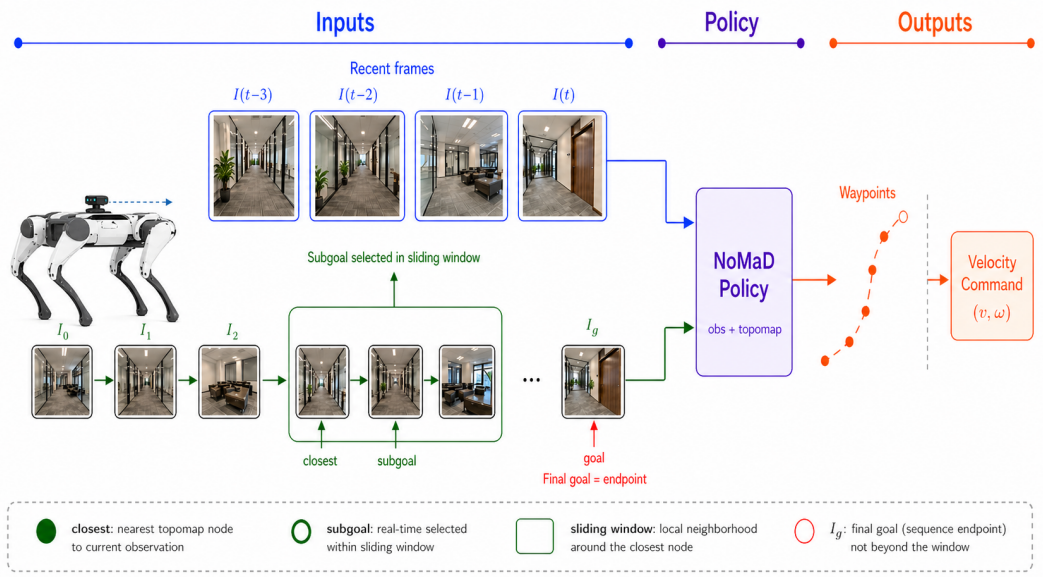


图 2.1 导航任务输入输出与 topomap 关系示意图

其中 β_t 为第 t 步噪声调度系数。经过多步加噪后，样本会接近标准高斯分布。模型训练的目标是在反向过程中预测噪声，从而逐步恢复原始样本。常见训练目标写为

$$\mathcal{L}_{\text{diff}} = \mathbb{E}_{x_0, \epsilon, t} [\|\epsilon - \epsilon_\theta(x_t, t, c)\|_2^2], \quad (2.5)$$

其中 ϵ 为真实噪声， ϵ_θ 为模型预测噪声， c 表示条件信息。

利用马尔可夫链的性质，可以推导出从 x_0 到任意时刻 x_t 的闭合形式。定义 $\alpha_t = 1 - \beta_t$ ， $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$ ，则有

$$q(x_t | x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t) \mathbf{I}). \quad (2.6)$$

这意味着可以直接从 x_0 一步采样得到任意时刻的加噪样本 $x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$ ，其中 $\epsilon \sim \mathcal{N}(0, \mathbf{I})$ 。这一闭合形式是训练过程高效进行的基础——训练时无需依次模拟整个前向链，只需随机采样时间步 t 并直接计算 x_t 即可。

本工作中显式将训练扩散步数设为 $T = 10$ ，这远低于通用图像扩散的 $T = 1000$ 量级，原因是本工作的扩散对象不是高维像素而是 16 维 waypoint 轨迹，低维分布对去噪步数的需求大幅降低，这一配置在轨迹生成质量与训练稳定性之间取得了良好的平衡。

从变分推断的角度来看，扩散模型的训练目标实际上是对数据对数似然的变分下界 (Variational Lower Bound, VLB) 的优化。具体而言，对数似然 $\log p_\theta(x_0)$ 的 VLB

可以分解为

$$\log p_\theta(x_0) \geq \mathbb{E}_q \left[-\log \frac{q(x_{1:T} | x_0)}{p_\theta(x_{0:T})} \right] = -\mathcal{L}_{\text{VLB}}, \quad (2.7)$$

其中 $\mathcal{L}_{\text{VLB}}$ 可进一步分解为逐步 KL 散度之和。Ho 等人^[18]在实践中发现, 直接使用简化的噪声预测损失 $\mathcal{L}_{\text{diff}}$ 训练效果更好, 尽管它与严格的 VLB 目标并不完全等价。

在轨迹生成的具体应用中, x_0 不再是图像像素, 而是轨迹 waypoint 序列 $\mathbf{A} = [\mathbf{a}_1, \dots, \mathbf{a}_H] \in \mathbb{R}^{H \times 2}$ 。相应地, 扩散过程在轨迹空间中进行: 前向过程逐步向轨迹添加噪声, 反向过程从纯噪声逐步恢复出合理轨迹。由于轨迹空间的维度远低于图像空间, 扩散模型在轨迹生成中的收敛速度和采样效率都显著优于图像生成, 这也是扩散策略能够在实时控制场景中应用的重要原因。

这种建模方式在导航领域具有以下关键优势:

1. **多模态建模能力:** 扩散模型不强迫在多峰分布中只输出一个均值解, 而是通过去噪过程逐步生成多种合理候选。在十字路口等存在多条合理路径的场景中, 这种能力尤为重要;
2. **样本质量可控:** 通过调节去噪步数和引导强度, 可以在生成质量与计算开销之间灵活权衡;
3. **条件生成自然:** 条件信息 (如目标图像) 可以自然地融入去噪过程, 无需额外的条件注入机制。

导航与控制任务中的未来轨迹本身就常常具有多模态特性, 因此扩散建模天然适合此类问题。

2.2.2 DDPM 与 DDIM

DDPM 采用随机反向扩散过程生成样本, 采样质量通常较好, 但采样步数较多、推理开销较大^[18]。DDIM 在保持训练目标不变的前提下, 将反向过程改写为非马尔可夫且可确定性的更新形式, 从而实现更少步数的快速采样^[19]。当取确定性采样形式时, 其一步更新可写为

$$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}} \hat{x}_0 + \sqrt{1 - \bar{\alpha}_{t-1}} \epsilon_\theta(x_t, t, c), \quad (2.8)$$

其中 $\hat{x}_0$ 由当前样本 x_t 与预测噪声反推得到, $\bar{\alpha}_t$ 为累计噪声调度系数。

更具体地, DDIM 的关键创新在于引入了一族非马尔可夫前向过程。Song 等人^[19]观察到, DDPM 的训练目标仅依赖于边际分布 $q(x_t | x_0)$ 而非联合分布 $q(x_{1:T} | x_0)$ 。因此可以构造一族具有相同边际分布但不同联合分布的前向过程, 其反向过程

可以写为

$$q_{\sigma}(x_{t-1} | x_t, x_0) = \mathcal{N}\left(\sqrt{\bar{\alpha}_{t-1}}x_0 + \sqrt{1 - \bar{\alpha}_{t-1} - \sigma_t^2} \cdot \frac{x_t - \sqrt{\bar{\alpha}_t}x_0}{\sqrt{1 - \bar{\alpha}_t}}, \sigma_t^2 \mathbf{I}\right), \quad (2.9)$$

其中 σ_t 是一个可调参数，控制反向过程中的随机性。定义 η 参数使得

$$\sigma_t = \eta \sqrt{\frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t}} \sqrt{1 - \frac{\bar{\alpha}_t}{\bar{\alpha}_{t-1}}}, \quad (2.10)$$

则 $\eta = 1$ 时恢复为 DDPM 的随机采样， $\eta = 0$ 时得到完全确定性的 DDIM 采样。 η 的中间值则在两者之间插值，提供了在采样多样性与确定性之间的连续调节能力。

在导航轨迹生成的实际应用中， η 的选择涉及探索与确定性之间的权衡。 $\eta = 0$ （完全确定性）意味着给定相同的初始噪声和条件，每次都会生成完全相同的轨迹，这在需要可复现行为的场景中有优势，但丧失了多样性。 $\eta > 0$ 则引入随机性，使得每次采样可能产生不同的轨迹候选，在本文实验中， η 被设置为 $\eta \in [0.3, 0.5]$ ，以确保候选轨迹之间有足够的多样性。

表 2.1 DDPM 与 DDIM 的主要性质对比

性质	DDPM	DDIM
反向过程类型	随机（马尔可夫）	确定性或半随机（非马尔可夫）
采样步数	通常需要完整 T 步	支持子序列跳步采样
η 参数	固定为 1	可调，0 为确定性
采样多样性	较高（每次结果不同）	$\eta = 0$ 时完全确定
训练目标	噪声预测 $\mathcal{L}_{\text{diff}}$	与 DDPM 相同，无需重新训练
推理速度	较慢	可显著加速

本文后续所说的“DDIM 推理加速”，就是在不重新训练网络的前提下，用更少的去噪步数近似原始 DDPM 采样。

在实际使用中，DDIM 的跳步采样通过选择原始时间步序列 $\{1, 2, \dots, T\}$ 的一个子序列 $\{\tau_1, \tau_2, \dots, \tau_S\}$ （其中 $S \leq T$ ）来实现加速。NoMaD 训练时已经把 T 压到 10，因此本文 DDIM 实验的有效压缩区间是从 $S = 10$ 进一步减到 $S = 5/3/2/1$ ，对应每步覆盖 1 到 5 个原始时间步。子序列的选择方式（均匀间隔、二次间隔等）也会影响采样质量。在本文的导航场景中，由于轨迹空间的维度远低于图像空间，扩散模型对跳步采样的容忍度通常较高，这为在边缘设备上实现实时推理提供了可能。

2.2.3 Classifier-Free Guidance

Classifier-Free Guidance (CFG) 最初用于条件生成任务中的引导增强^[23]。其思想是在训练阶段同时学习有条件与无条件预测，在推理阶段通过两者线性组合增强条件约束。若条件分支噪声预测为 ϵ_{cond} ，无条件分支为 ϵ_{uncond} ，则引导后的噪声预测写为

$$\epsilon_{\text{cfg}} = (1 + w)\epsilon_{\text{cond}} - w\epsilon_{\text{uncond}}, \quad (2.11)$$

其中 w 为 guidance scale。当 $w = 0$ 时退化为普通条件推理；当 $w > 0$ 时，模型会更强烈地朝向条件约束；当 w 过大时，也可能带来分布偏移与不稳定。

CFG 之所以能在推理时实现无分类器引导，关键在于训练阶段的条件随机丢弃 (condition dropout) 机制。具体而言，在训练过程中以概率 p_{uncond} (通常取 0.1 至 0.2) 将条件信息 c 替换为空条件 $\emptyset$ ，使得同一模型 ϵ_{θ} 同时学习条件预测 $\epsilon_{\theta}(x_t, t, c)$ 与无条件预测 $\epsilon_{\theta}(x_t, t, \emptyset)$ 。在推理阶段，对同一输入分别执行条件和无条件前向传播，再通过线性组合实现引导，无需额外训练分类器网络。

在本文中，CFG 的作用是增强轨迹对目标条件的牵引能力，即让生成的局部 way-point 更明确地朝向目标图像或目标节点图像对应的方向。

本工作中 CFG 的具体实现流程如下：

1. 在每个去噪步骤中，分别以目标 token 和掩码 token 作为条件，执行两次前向传播；
2. 获得条件噪声预测 ϵ_{cond} 和无条件噪声预测 ϵ_{uncond} ；
3. 按照 CFG 公式计算引导后的噪声预测 ϵ_{cfg} ；
4. 使用 ϵ_{cfg} 执行一步去噪更新。

需要注意的是，在实际实现中，CFG 的无条件分支并非完全无信息——它仍然接收历史观察 token，只是目标 token 被替换为掩码 token。因此 CFG 在这里增强的是“目标方向性”，而非“整体行为合理性”。这一区别使得即使在较大的 w 值下，生成的轨迹仍然保持基本的可行性（因为观察条件始终存在），只是目标趋向性被放大。

2.2.4 扩散策略与 Test-Time Scaling

扩散策略将扩散模型从图像生成推广到动作序列或轨迹序列生成^[11,20]。在具身智能与机器人控制领域，Diffuser 和 Diffusion Policy 证明了扩散建模能够有效处理多步决策、长时序动作和多模态行为分布，因此被视为扩散模型在具身控制中的代表性应用。

具体而言, Diffuser^[20] 将整个轨迹 (包括状态和动作序列) 作为扩散对象, 在推理时通过引导函数注入奖励信息来生成高回报轨迹。Diffusion Policy^[11] 则只对动作序列进行扩散, 将状态观察作为条件信息, 更适合视觉观察驱动的控制任务。NoMaD 的扩散策略头采用了类似 Diffusion Policy 的设计——仅对 waypoint 轨迹进行扩散, 视觉特征作为条件信息注入。这种设计使得扩散过程仅需在低维轨迹空间中进行, 显著降低了计算开销。

在此基础上, 近年来出现了测试时扩展 (Test-Time Scaling, TTS) 的思路^[24-25]。其核心思想不是修改训练过程, 而是在推理阶段使用额外计算预算生成多条候选轨迹, 再通过打分函数进行选择。若生成 N 条候选轨迹 $\{\mathbf{A}^{(i)}\}_{i=1}^N$, 评分函数为 $s(\cdot)$, 则最优轨迹可表示为

$$\mathbf{A}^* = \arg \max_{i \in \{1, \dots, N\}} s(\mathbf{A}^{(i)} | \mathcal{O}_t, I_g). \quad (2.12)$$

评分函数的设计是 TTS 方法的核心。常见的评分维度包括: (1) 目标一致性评分, 衡量候选轨迹终点与目标方向的对齐程度, 可以利用距离预测头对轨迹终点的距离估计作为代理指标; (2) 轨迹平滑性评分, 衡量相邻 waypoint 之间的方向变化和速度变化是否连续, 过大的方向突变通常意味着不可执行或执行风险较高的轨迹; (3) 前向进展评分, 衡量轨迹整体是否朝目标方向前进, 避免选择在原地打转或后退的候选。在实际实现中, 这些评分维度通常以加权和的形式组合:

$$s(\mathbf{A}^{(i)}) = \lambda_{\text{goal}} \cdot s_{\text{goal}}(\mathbf{A}^{(i)}) + \lambda_{\text{smooth}} \cdot s_{\text{smooth}}(\mathbf{A}^{(i)}) + \lambda_{\text{prog}} \cdot s_{\text{prog}}(\mathbf{A}^{(i)}), \quad (2.13)$$

其中 λ_{goal} 、 λ_{smooth} 和 λ_{prog} 为各维度的权重系数。具体地, 平滑性评分可以定义为相邻 waypoint 方向变化角的负值之和:

$$s_{\text{smooth}}(\mathbf{A}) = - \sum_{i=2}^{H-1} |\angle(\mathbf{a}_{i+1} - \mathbf{a}_i) - \angle(\mathbf{a}_i - \mathbf{a}_{i-1})|, \quad (2.14)$$

其中 $\angle(\cdot)$ 表示二维向量的方向角。前向进展评分则可以定义为轨迹终点在目标方向上的投影距离:

$$s_{\text{prog}}(\mathbf{A}) = \mathbf{a}_H \cdot \hat{\mathbf{d}}_{\text{goal}}, \quad (2.15)$$

其中 $\hat{\mathbf{d}}_{\text{goal}}$ 为指向目标的单位方向向量。

TTS 的工作方式与蒙特卡洛树搜索 (Monte Carlo Tree Search, MCTS) 存在概念上的相似性。MCTS 通过大量随机模拟评估不同决策路径的期望回报, TTS 则通过大量采样评估不同生成轨迹的综合质量。两者的核心共性在于“用额外计算换取更优决

策”。区别在于 MCTS 通常涉及多步展开和回溯，而 TTS 在扩散策略中主要针对单次生成的多条候选进行比较选择，其计算开销的扩展方式更为简单和可控。

从理论角度来看，TTS 的有效性建立在一个关键假设之上：扩散模型生成的候选集合中，高质量轨迹的存在概率随候选数量 N 的增加而增大。设单次采样得到可接受轨迹的概率为 p ，则 N 次独立采样后至少存在一条可接受轨迹的概率为 $1 - (1 - p)^N$ 。当 p 较小时（对应困难场景），需要较大的 N 才能保证成功率；当 p 接近 1 时（对应简单场景）， $N = 1$ 即可，此时 TTS 退化为普通单次采样。这种“按需扩展”的特性使得 TTS 特别适合于计算预算可变的部署场景。

需要强调的是，TTS 与 DDIM、CFG 作用位置不同。DDIM 决定“怎么采样”，CFG 决定“采样时如何增强条件”，而 TTS 决定“采样出多条候选后如何选择”。因此三者虽然都属于推理时优化，但不是简单相加关系，而是具有先后层次的组合关系。

2.2.5 扩散模型、ViT 与导航建模的关系

Transformer 在视觉建模中的广泛应用，使得扩散模型与视觉编码方法的结合成为趋势。ViT 将图像切分为 patch token 并以自注意力机制进行建模^[6]；DiT 则将扩散过程进一步与 Transformer 架构结合，说明扩散模型可以在强表征模型之上获得更高质量生成能力^[7]。在导航领域，ViNT 使用 Transformer 处理历史观察与目标图像^[9]，NoMaD 则在此基础上引入动作扩散策略^[10]。因此，本文讨论的视觉编码器升级、扩散采样优化和系统部署，其实都位于这一技术演化脉络之中。

具体而言，ViT 的核心操作是将输入图像划分为固定大小的 patch（如 16×16 像素），将每个 patch 线性映射为一个 token 向量，再加上位置编码后送入多层 Transformer 编码器。对于输入尺寸为 $H \times W$ 的图像和 patch 大小 P ，token 序列长度为 $N = HW/P^2$ 。

2.3 NoMaD 机制

2.3.1 NoMaD 的基本结构

NoMaD 由视觉编码器、距离预测头和扩散策略头三部分构成^[10]。视觉编码器负责把历史观察与目标图像编码为统一条件特征；距离预测头估计当前观察与目标之间的相对距离（以离散时间步数为单位，表示从当前位置到达目标大约需要多少个导航步骤，而非物理距离）；扩散策略头则在该条件特征上生成未来局部 waypoint 轨迹。若记局部轨迹为

$$\mathbf{A} = [\mathbf{a}_1, \mathbf{a}_2, \dots, \mathbf{a}_H], \quad \mathbf{a}_i = (\Delta x_i, \Delta y_i), \quad (2.16)$$

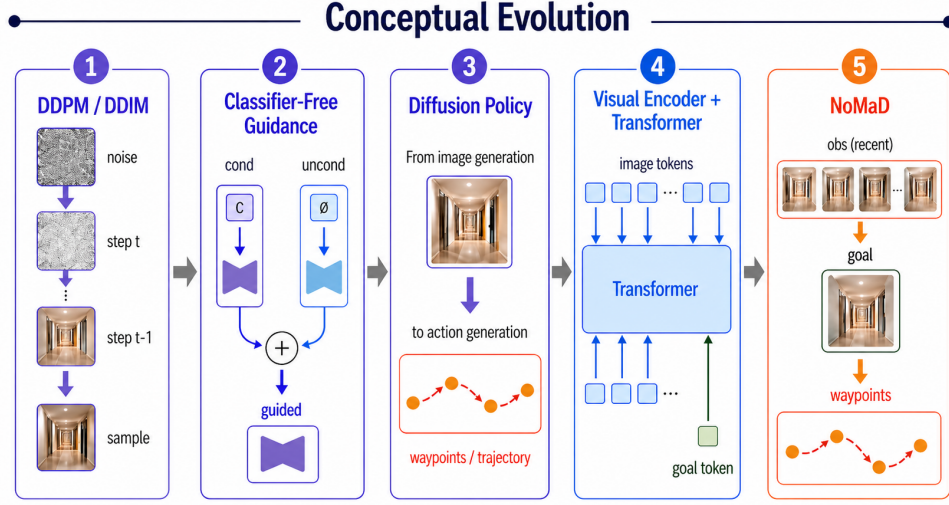


图 2.2 扩散模型、扩散策略、ViT 编码器与 NoMaD 系统之间的技术演化关系。从左到右依次为基础扩散理论 (DDPM/DDIM)、条件引导方法 (CFG)、扩散策略 (Diffusion Policy) 以及 NoMaD 的架构

则 NoMaD 学习的是条件分布

$$p(\mathbf{A} \mid \mathcal{O}_t, I_g). \quad (2.17)$$

在 NoMaD 的视觉编码器部分，其底层基于 ViNT 的 Transformer 架构。具体而言，每帧观察图像首先通过 CNN 或 ViT 前端提取特征图，再经过全局平均池化压缩成一个高维向量，最后由线性投影层映射到统一的 token 维度。一帧图像对应一个观察 token。在 NoMaD 中，定义 K 为 context_size (默认为 3，表示当前帧之外向前回看的步数)，则模型一次前向需要把 $K + 1$ 帧 RGB 图像送入观察编码器 obs_encoder 独立编码，得到 $K + 1$ 个观察 token；目标图像与当前帧沿通道维度拼接后再送入参数独立的目标编码器，得到一个目标 token。这些 token 被拼接为一个统一的序列，送入 Transformer 编码器进行联合注意力计算。形式化地，令观察 token 序列为 $\{\mathbf{z}_{\text{obs}}^{(k)}\}_{k=0}^K$ ，目标 token 为 $\mathbf{z}_{\text{goal}}$ ，则 Transformer 编码器的输入为

$$\mathbf{Z}_{\text{input}} = [\mathbf{z}_{\text{obs}}^{(0)}; \mathbf{z}_{\text{obs}}^{(1)}; \dots; \mathbf{z}_{\text{obs}}^{(K)}; \mathbf{z}_{\text{goal}}], \quad (2.18)$$

共 $K + 2$ 个 token。经过多层自注意力后，输出的 token 表征即为共享条件空间中的统一条件特征。

在 ViNT^[9] 的原始设计中，Transformer^[26] 编码器采用 4 层标准自注意力结构，隐藏维度为 256，注意力头数为 4。每帧图像经过 EfficientNet-B0^[27] 前端后再全局平均池化，将空间维度压缩为 1，从而每帧产生 1 个 1280 维特征，再由线性投影压缩到

256 维 token。因此对于 NoMaD 默认的 $K = 3$ (context_size) 配置, 模型一次前向得到 $K + 1 = 4$ 个观察 token 加 1 个目标 token, 共 5 个 token 进入自注意力编码; ViNT 默认配置 $K = 5$ 时则为 7 个 token。为了区分不同帧的 token, 系统为每帧添加可学习的帧级位置编码, 使模型能够区分不同时间步的观察。Transformer 的输出 token 经过加权平均池化后产生一个固定维度的条件向量, 该向量同时被距离预测头和扩散策略头使用。

2.3.2 距离预测头与视觉编码器的直接耦合

距离预测头与扩散策略头共享同一条视觉条件空间, 这意味着视觉编码器的改动会产生连锁效应。假设原始编码器输出的条件特征维度为 d_{orig} , 新编码器输出维度为 d_{new} , 则即使通过线性投影层将 d_{new} 映射到 d_{orig} , 投影后的特征分布也与原始特征不同。距离预测头是一个在原始特征分布上训练的回归网络, 输入分布的偏移会直接导致距离预测不准确。因此, 本文在编码器升级实验中采用端到端微调策略, 同时更新编码器适配层、距离预测头和扩散策略头的参数。这种端到端微调策略虽然计算成本较高, 但确保了三个组件在新的特征空间中重新对齐, 避免了因局部更新导致的性能退化。

2.3.3 goal mask 与导航/探索统一建模

NoMaD 的另一个关键机制是 goal mask。训练时, 模型以一定概率遮蔽目标 token, 使其同时经历“有目标条件”和“无显式目标条件”两种情况^[10]。这并不意味着模型在无目标条件时进行随机游走, 而是意味着它会学习在当前视觉上下文中生成连续、可执行且符合数据轨迹分布的探索性行为。正因为如此, NoMaD 才能在部署时同时支持导航模式与探索模式。

goal mask 的设计还有一个重要的技术意义: 它使得模型在训练时自然地学习了“当前观察本身蕴含的行为先验”。即使没有目标指引, 训练数据中的轨迹通常也遵循特定的行为模式(如沿道路行驶、避开障碍物), goal mask 使模型能够将这些模式内化为无条件生成能力。

在数学表述上, goal mask 的训练过程可以描述如下。设 goal mask 概率为 p_{mask} , 对于每个训练样本, 以概率 p_{mask} 将目标 token $\mathbf{z}_{\text{goal}}$ 替换为可学习的掩码 token $\mathbf{z}_{\text{mask}}$ 。因此模型在训练过程中优化的是混合目标:

$$\mathcal{L} = p_{\text{mask}} \cdot \mathcal{L}_{\text{diff}}(\epsilon_{\theta}(x_t, t, \mathbf{z}_{\text{obs}}, \mathbf{z}_{\text{mask}})) + (1 - p_{\text{mask}}) \cdot \mathcal{L}_{\text{diff}}(\epsilon_{\theta}(x_t, t, \mathbf{z}_{\text{obs}}, \mathbf{z}_{\text{goal}})). \quad (2.19)$$

在本文的配置中, p_{mask} 设置为 0.5, 确保两种模式都获得充分训练。值得注意的是,

goal mask 机制与 CFG 中的条件随机丢弃 (condition dropout) 在形式上高度一致——两者都是在训练时以一定概率移除条件信息。因此 goal mask 实际上内在地为 CFG 推理提供了基础：在推理时，将条件分支（提供目标 token）与无条件分支（使用掩码 token）的噪声预测进行 CFG 组合，即可实现对目标方向的引导增强。这种设计使得探索能力与导航增强能力共享同一训练框架。

在部署阶段，探索模式与导航模式的切换仅需改变推理时的条件输入：提供真实目标 token 时进入导航模式，提供掩码 token 时进入探索模式。两种模式之间的切换不需要加载不同的模型权重或改变网络结构，这大大简化了部署系统的设计。此外，探索模式下生成的轨迹并非随机——它们反映了训练数据中的轨迹分布特征，通常表现为沿走廊前进、绕过障碍物等符合环境结构的行为模式。

2.4 轮式平台、足式平台与 Lite3

2.4.1 轮式与足式平台的部署差异

轮式平台姿态变化小、视觉输入稳定，与速度控制器的接口也较简单；足式平台则具有更强的地形适应性，但会带来更明显的机体起伏、更严格的实时性约束和更复杂的控制接口^[12-13]。这意味着，同一高层轨迹若能在轮式平台上顺利执行，并不代表它能直接在四足平台上稳定工作。

具体而言，足式平台的步态运动会在每个步态周期内引入机体俯仰和侧倾变化，这种扰动会导致相机的视场发生抖动，使得连续帧之间的视觉特征产生非平移性变化。对于依赖时序视觉输入的导航策略，步态引起的视觉抖动可能被误解为环境变化或自身运动，从而产生错误的轨迹预测。此外，足式平台的非完整运动学约束也与轮式平台不同。轮式机器人通常可以精确执行差速运动学模型所描述的运动，而四足机器人的实际运动轨迹受步态规划器和地形交互的影响，与指令速度之间存在非线性偏差。这种偏差意味着即使高层策略输出了精确的速度指令，机器人的实际位移也可能与预期存在差异，进一步强调了闭环执行和频繁重规划的必要性。

2.4.2 云深处 Lite3 平台与关键参数

本文采用的主要足式平台为云深处 Lite3。结合项目配置与部署代码，其在本文中的系统定位是“由高层视觉导航策略驱动的四足运动系统”，而不是端到端关节控制平台。对应控制链路为

$$\text{视觉观察} \rightarrow \text{高层 NoMaD 推理} \rightarrow \text{中层控制映射} \rightarrow \text{底层运动执行}. \quad (2.20)$$

与本文实现直接相关的关键参数如表 2.2 所示。

表 2.2 本文中与 Lite3 相关的关键平台参数

参数类别	参数项	数值或说明
平台类型	机器人形态	四足平台
运动结构	关节数量	12 关节执行系统
速度接口	前后速度范围	$[-1.0, 1.0]$ m/s
速度接口	左右速度范围	$[-0.5, 0.5]$ m/s
速度接口	偏航角速度范围	$[-1.5, 1.5]$ rad/s
部署限幅	本文最大线速度	0.12 m/s
部署限幅	本文最大角速度	0.35 rad/s
高层闭环	仿真 NoMaD 周期	4 Hz
通信安全	速度超时保护	250 ms 自动停止

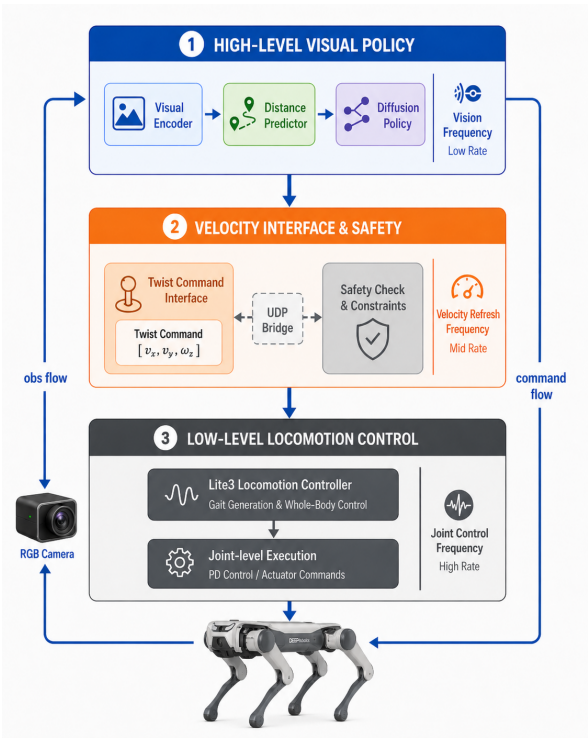


图 2.3 Lite3 平台与控制链路示意图

在通信协议层面，本文系统与 Lite3 之间通过 UDP 协议进行数据交换。在本文实现中，Python 端通过 Lite3Controller 封装底层 sendto 调用，并由桥接层把高层三维速度参考拆分为前后、左右和偏航三个速度通道分别下发。控制器还维护独立的自动心跳线程；当前配置中心心跳频率为 5 Hz，速度刷新线程为 25 Hz，满足 Lite3 通讯文档中速度轴命令至少 20 Hz 刷新的要求。协议文档给出的速度命令超时约束可以理解

为底层接口的安全边界，但高层 NoMaD 推理本身并不需要在每个 250 ms 内都完整完成一次；桥接层会在两次高层推理之间持续重复最近一次限幅后的速度参考。

关于观察频率与控制频率的关系，需要区分三个层次的频率，这种多层频率架构可以用以下层级关系概括：

$$f_{\text{joint}} \gg f_{\text{velocity}} \gg f_{\text{vision}}, \quad (2.21)$$

其中 f_{joint} 表示底层关节级控制频率， f_{velocity} 表示速度命令刷新频率，当前真机桥接实现中 $f_{\text{velocity}} = 25 \text{ Hz}$ ， f_{vision} 表示高层视觉导航策略的更新频率，在 MuJoCo 仿真中为 4 Hz，在 Orin 真机上则由实测推理耗时决定。这种频率分离设计的好处是每一层只需关注自身时间尺度上的决策，坏处是层间的响应延迟会累积。

第3章 算法设计与实验

3.1 任务定义与研究思路

本文关注的核心任务是：给定机器人当前视觉观察、短历史观察和目标图像，生成一段局部 waypoint 轨迹，并将其作为中层控制器的参考输入。沿用 NoMaD 的基本思想^[10]，本方法不直接预测单步速度命令，而是学习条件轨迹分布

$$p(\mathbf{A}_t | \mathcal{O}_t, I_g), \quad (3.1)$$

其中 $\mathcal{O}_t$ 表示时刻 t 的历史视觉观察， I_g 表示当前控制周期使用的目标图像，在拓扑导航中是 topomap 序列上“当前定位的下一节点”图像， $\mathbf{A}_t$ 表示未来局部 waypoint 序列。

3.1.1 问题建模

从马尔可夫决策过程（Markov Decision Process, MDP）的视角来看，视觉导航可以被建模为一个元组 $\mathcal{M} = (\mathcal{S}, \mathcal{A}, T, R, \gamma)$ ，其中 $\mathcal{S}$ 表示状态空间（即视觉观察空间）， $\mathcal{A}$ 表示动作空间， T 表示转移概率， R 表示奖励函数， γ 表示折扣因子。在本文的框架中，策略 π 不是直接在 MDP 上求解单步最优动作，而是学习一个条件轨迹分布 $p(\mathbf{A}_t | \mathcal{O}_t, I_g)$ ，这种分布式策略具有天然的多模态建模能力。

具体来说，条件轨迹分布与传统 MDP 策略之间的联系可通过以下关系理解：若将未来 H 步动作序列视为一个联合决策变量，则条件轨迹分布相当于在扩展动作空间 $\mathcal{A}^H$ 上的策略。与逐步递推不同，这种联合建模能够在单次推理中捕获全局轨迹结构，避免了自回归策略中误差累积的问题。形式上：

$$p(\mathbf{A}_t | \mathcal{O}_t, I_g) = p(\mathbf{a}_1, \mathbf{a}_2, \dots, \mathbf{a}_H | \mathcal{O}_t, I_g), \quad (3.2)$$

其中每个 $\mathbf{a}_i$ 并非独立采样，而是通过扩散过程的联合去噪一次性生成。

这种建模方式与经典的行为克隆（Behavioral Cloning）策略存在本质区别。行为克隆通常将策略参数化为确定性映射 $\pi_\theta : \mathcal{O} \rightarrow \mathcal{A}$ 或简单的高斯分布，只能表达单峰的动作分布。而基于扩散模型的条件生成策略通过迭代去噪过程，能够从复杂的多模态分布中采样，天然适合处理导航任务中的分支决策场景。此外，扩散模型的训练目标（噪声预测损失）在数值上比最大似然目标更稳定，不容易出现模式坍塌（mode collapse）问题。

3.1.2 Waypoint 表示的优势分析

本文选择 waypoint 序列作为策略输出形式，而非直接预测底层速度命令（如线速度 v 和角速度 ω ），这一设计决策基于以下三点考量。

第一，平台无关性。Waypoint 表示为局部坐标系下的二维位移序列，不与任何特定的运动学模型绑定。无论下游执行平台是差分驱动的轮式机器人还是具有复杂运动学的足式机器人（如 Lite3），同一条 waypoint 轨迹都可以通过不同的中层控制器进行跟踪。这种解耦使得高层策略的训练数据可以跨平台复用。

第二，多模态表达能力。在面对 T 形路口或走廊分叉等场景时，合理的导航行为可能包含“左转”和“右转”两个互不相容的模式。若使用单步速度命令的均值回归策略，两个模式的平均结果可能是“直行撞墙”。而 waypoint 序列结合扩散模型的生成能力，能够保留这种多模态分布，通过采样不同轨迹来表达不同的决策意图。

第三，闭环控制的友好性。中层控制器（如 PID 或 MPC）可以直接将 waypoint 序列作为参考路径进行跟踪，这比将高层策略输出的速度命令直接下发到底层更加鲁棒，因为 waypoint 跟踪器可以根据实际执行情况进行局部修正。

waypoint 序列还提供了比单步动作更丰富的决策信息。单步速度命令仅编码了当前时刻的瞬时意图，而 waypoint 序列编码了未来一段时间窗口内的完整运动意图。这种“前瞻性”信息对于中层控制器的预测性规划至关重要——控制器可以提前预判即将到来的转弯或减速需求，从而生成更平滑的底层控制信号。特别是对于足式机器人，步态切换和身体姿态调整需要提前规划，单步速度命令无法提供足够的前瞻信息。

围绕这一任务，本文在算法层的工作可概括为三部分。第一，基于项目代码准确复现 NoMaD 的训练与推理链路，明确视觉编码器、距离预测头与扩散策略头之间的关系。第二，围绕部署可用性对推理层进行优化，重点研究 DDIM 推理、CFG 引导增强和 TTS 推理时搜索。第三，围绕视觉表征能力设计视觉编码器升级实验，并与基线配置进行对比。需要强调的是，本章关注的是“哪些候选方法值得进入闭环验证”，因此所有结论都首先被限定在离线统计与离线代理指标层面。

3.2 NoMaD 基础模型复现

3.2.1 输入输出形式

根据当前项目配置，NoMaD 的输入由历史视觉观察与目标图像构成。实验中设置为 `context_size=3`，即模型同时接收三张历史观察帧与一张目标图像。输出则为固

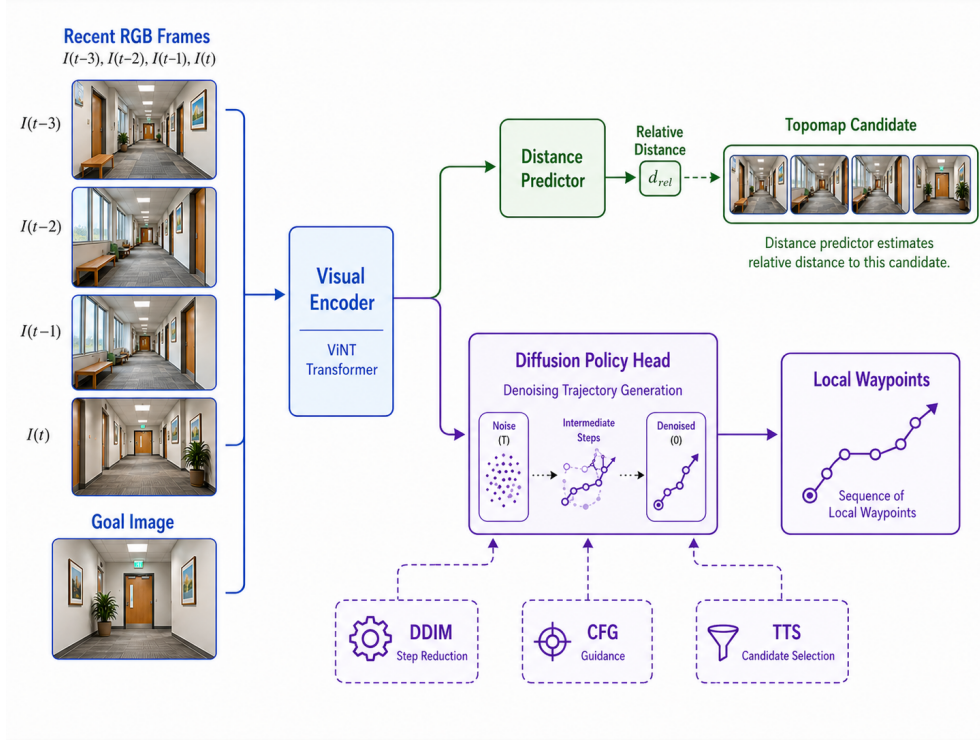


图 3.1 NoMaD 算法研究对象与数据流示意图

定长度的局部轨迹序列，每个 waypoint 用局部坐标系下的二维位移表示：

$$\mathbf{a}_i = (\Delta x_i, \Delta y_i), \quad i = 1, \dots, H. \quad (3.3)$$

整段轨迹写为

$$\mathbf{A} = [\mathbf{a}_1, \mathbf{a}_2, \dots, \mathbf{a}_H]. \quad (3.4)$$

这一表示的优点在于，它比直接输出底层速度命令更适合作为平台无关的高层接口，也比全局路径更适合闭环局部控制。对于足式平台，高层输出局部轨迹、中层完成控制映射的方式尤为关键。

3.2.2 视觉编码器架构

NoMaD 默认采用 EfficientNet-B0 作为视觉编码器骨干^[27]。EfficientNet-B0 是 EfficientNet 系列的基础模型，通过复合缩放（compound scaling）策略在深度、宽度和分辨率三个维度上进行统一缩放，在 ImageNet 上以约 5.3 M 参数量达到了优异的精度-效率平衡。其核心构建块为 MBConv（Mobile Inverted Bottleneck Convolution），采用深度可分离卷积和 Squeeze-and-Excitation 注意力模块。

NoMaD 实现中，NoMaD_ViNT 内部实例化了两个 EfficientNet-B0：一个只处理

单帧观察（3 通道输入），另一个处理“最后一帧观察 + 目标图像”的 6 通道拼接输入。二者权重独立、架构几乎相同，只在输入卷积的通道数上有差别。这一设计使得目标分支在特征层就完成了当前帧与目标帧的早期融合，但也使得视觉编码器部分的参数量近似等于两个 EfficientNet-B0 之和，再加上一个输出投影层（将 1280 维 backbone 特征压缩到 256 维 token 维度）。

两个 EfficientNet-B0 接收 96×96 的 RGB 图像作为输入，提取特征后分别输出一个固定维度的视觉表示。该特征向量随后被投影为标准化的 token 表示，用于后续的条件拼接。具体而言，每帧观察图像经过第一个编码器后得到特征 $\mathbf{z}_i \in \mathbb{R}^d$ ，其中 d 为 token 维度（本文配置下 $d = 256$ ）；目标 token $\mathbf{z}_g$ 由第二个编码器以相同方式得到。

选择 EfficientNet-B0 而非更大的 EfficientNet 变体（如 B3 或 B5）是出于推理效率的考虑。在边缘设备上，视觉编码器的计算开销直接影响整体推理频率，而 B0 的 5.3 M 参数量在精度与效率之间提供了合理的折中；由于 NoMaD 中需要同时维护两个 EfficientNet-B0 实例，骨干规模的选择对整体部署时延具有放大效应，这进一步佐证了 B0 作为默认选型的合理性。

3.2.3 Token 构建与条件拼接

NoMaD 的视觉条件编码器 NoMaD_ViNT 由两个相互独立的 EfficientNet-B0 实例组成：一个为观察编码器 obs_encoder，输入 3 通道单帧图像，用来独立编码每一个历史观察帧；另一个为目标编码器 goal_encoder，输入 6 通道图像，其输入是将当前帧（即观察窗口的最后一帧）与目标图像沿通道维度拼接而成。这样设计的原因是目标 token 天然需要同时携带“当前在哪”和“要去哪”两种信息，因此在特征提取层就进行早期融合能减轻后续 self-attention 的负担。

对于 context_size=3 的配置，模型一次前向会产生四个 token：由 obs_encoder 独立编码的三个观察 token $[\mathbf{z}_{t-2}, \mathbf{z}_{t-1}, \mathbf{z}_t]$ ，以及由 goal_encoder 编码的一个融合了当前帧与目标图像的目标 token $\mathbf{z}_g$ 。四个 token 经位置编码后送入 Transformer self-attention encoder，再对所得序列做掩码加权平均池化，得到固定维度的视觉条件向量：

$$\mathbf{c} = \text{AvgPool}(\text{SA}([\mathbf{z}_{t-2}, \mathbf{z}_{t-1}, \mathbf{z}_t, \mathbf{z}_g])). \quad (3.5)$$

这一条件向量同时作为距离预测头和扩散策略头的输入。

需要特别说明的是，NoMaD 通过目标掩码（goal mask）实现导航模式与探索模式的切换。当 goal mask 取 0 时，目标 token $\mathbf{z}_g$ 正常参与 self-attention，模型处于“带目标导航”模式；当 goal mask 取 1 时， $\mathbf{z}_g$ 所在位置被置为 padding 并在池化时去掉

权重，模型退化为“无目标探索”模式。这一机制使得同一套权重既可用于导航也可用于探索，也是后续 CFG 引导能同时使用“带目标”与“不带目标”两次前向的直接原因。

3.2.4 模型结构与训练目标

NoMaD 由视觉编码器、距离预测头和扩散策略头三部分组成^[10]。视觉编码器负责将历史观察与目标条件编码为统一特征；距离预测头输出当前观察到目标的相对距离估计；扩散策略头在该条件上生成局部轨迹样本。

训练时，总损失由距离回归损失与扩散噪声预测损失组成，可写为

$$\mathcal{L} = \alpha \mathcal{L}_{dist} + (1 - \alpha) \mathcal{L}_{diff}. \quad (3.6)$$

其中， $\mathcal{L}_{dist}$ 约束候选目标排序能力， $\mathcal{L}_{diff}$ 约束局部轨迹生成质量。

关于损失权重 α 的选择策略，需要注意以下两点。首先， $\mathcal{L}_{dist}$ 通常采用 L_1 或 L_2 回归损失，其数值尺度与扩散噪声预测损失 $\mathcal{L}_{diff}$ 存在差异，因此 α 实际上同时承担了任务权重和尺度归一化的双重角色。其次，距离预测头的梯度会回传至视觉编码器，这意味着 α 的选择会影响编码器学到的特征偏向——较大的 α 倾向于让编码器学习距离敏感的特征，较小的 α 则让编码器更侧重于轨迹生成所需的结构特征。

距离预测头与视觉编码器是直接耦合的，因此更换视觉编码器后，距离预测分支必须同步重新训练，否则距离头与轨迹头将共享不一致的条件分布。

3.2.5 推理链路的统一封装

在推理阶段，系统先编码观察与目标，再在需要时进行候选节点距离评估，最后通过扩散采样生成轨迹。

统一推理模块的设计遵循“配置驱动”原则：所有推理行为由一组配置参数完全确定，不存在隐式的硬编码默认值。这使得后续的 DDIM、CFG 和 TTS 实验可以通过修改配置文件来切换，而不需要改动推理代码本身。同时，统一推理模块也提供了标准化的输出接口，包括候选轨迹集合、距离预测值和推理时间统计，方便下游的评估脚本和闭环系统直接使用。

3.3 数据集与实验设备

3.3.1 GoStanford 数据集

本文所有离线实验均基于 GoStanford 数据集进行。该数据集来源于斯坦福大学校园环境中通过自主采集机器人记录的视觉导航轨迹，是 NoMaD 项目使用的标准训练与评估数据来源之一^[8-10]。数据集中的每条轨迹包含按时间顺序排列的第一人称 RGB 图像帧和对应的位姿信息。

GoStanford 数据集的场景多样性是其作为标准评估数据集的重要优势。数据采集覆盖了斯坦福大学校园中的室内走廊、室外道路、建筑入口、停车区域等多种环境类型。每条轨迹的长度从几十帧到数百帧不等，对应数十米到数百米的实际行驶距离。图像帧的采集频率约为 4 Hz，位姿信息由里程计或 GPS 提供，精度足以用于轨迹级别的训练监督。

本文使用的本地数据集包含来自 GoStanford 的多条轨迹。在离线实验中，每次推理从轨迹中提取连续观察帧与目标帧，构建标准化测试 case。DDIM、CFG 与编码器实验使用 24 个 case；TTS 补充实验为控制组合规模使用 12 个 case，但保持相同的 case 构造规则和统计口径。

3.3.2 数据预处理流程

从原始轨迹到模型输入之间存在一个标准化的数据预处理流程。首先，RGB 图像从原始分辨率（通常为 640×480 ）对齐至模型要求的输入尺寸（ 96×96 ），缩放过程使用双线性插值。其次，像素值从 $[0, 255]$ 归一化至 $[0, 1]$ 范围。最后，根据 context_size 参数从轨迹中提取连续帧作为历史观察，并从同一轨迹的未来位置选取目标帧。

测试 case 的构建遵循以下原则：当前观察与目标帧之间的时间间隔覆盖短距离（5–10 帧）和中距离（20–50 帧）两种范围，以确保评估涵盖不同难度级别。每个 case 包含完整的历史观察序列、目标图像和对应的真实轨迹标签，使得所有实验在完全相同的数据条件下进行比较。

值得说明的是，24 个测试 case 的数量虽然有限，但其设计覆盖了直行、转弯、宽阔区域和狭窄走廊等多种典型场景类型。在离线统计实验中，这一规模足以反映不同配置之间的趋势性差异，同时保持单次完整实验的运行时间在可接受范围内。对于需要更高统计置信度的结论，本文通过报告 CI95 来量化均值估计的不确定性。

3.3.3 感知输入与设备说明

本文系统采用纯 RGB 单目视觉作为唯一感知输入，不依赖深度传感器、激光雷达或惯性测量单元。这一设计选择与 NoMaD 原始框架一致，即高层导航策略仅以图像观察和目标图像为输入，不需要额外传感模态。

纯视觉感知的优势在于硬件成本低、安装简单、且与人类的导航感知方式最为接近。其主要局限在于：(1) 缺乏显式的深度信息，无法直接判断障碍物距离；(2) 对光照变化和动态遮挡敏感；(3) 单目视觉的尺度模糊性使得绝对距离估计困难。NoMaD 通过学习大量导航轨迹数据来隐式弥补这些局限。

在离线实验阶段，训练与统计实验运行在配备 NVIDIA RTX 4090 的计算机上；在 MuJoCo 仿真阶段，相机图像由 MuJoCo 渲染器以第一人称视角生成；在真机部署阶段，使用 USB 单目相机接入 Jetson Orin，输出分辨率为 640×480 。

RTX 4090 与 Jetson Orin 之间的算力差距是本文实验设计中需要特别关注的因素。RTX 4090 适合承担离线训练和统计实验，而 Jetson Orin 属于机载边缘计算平台，受功耗、散热和统一内存资源约束更强。这意味着在 RTX 4090 上可接受的推理时延，在 Jetson Orin 上仍需要通过板端 benchmark 重新确认。因此，本文在离线实验中特别关注时延指标的绝对值而非仅关注相对加速比，以便为部署阶段的配置选择提供直接参考。

3.4 基线实验

为了给后续所有优化实验建立统一参考，本文首先复现了原始 NoMaD 离线推理流程。输出结果表明，原始模型参数量约为 19,049,675。需要区分两种不同口径的时延：在端到端单次运行中，包含图像加载、编码、DDPM-10 采样与结果后处理的整体耗时约为 0.1250 s；而在本章后续使用的 24 个 case 统计实验中（见表 3.2），ddpm_10 的纯扩散采样平均时延约为 40 ms，CI95 为 ± 1.37 ms。两者数量级差异的根本原因是前者计入了一次性模型构建、首次 GPU kernel 编译与图像预处理开销，而后者在 warm-up 之后测量循环体内部的推理时间。由于后续 DDIM、CFG、TTS 与视觉编码器对比实验都运行在统一的统计脚本下，本文把“40 ms 量级”视为基线推理时延的有效参考，而把“0.125 s”仅保留作单次端到端流程的工程记录。

关于参数量的构成，19.05 M 参数中视觉条件编码器约占 15.18 M（含两个 EfficientNet-B0、投影层、位置编码与 4 层 self-attention），扩散策略头（ConditionalUnet1D）约占 3.85 M，距离预测头约占 0.02 M。由此可见视觉编码器是主

要参数来源，更换视觉编码器对总参数量的影响取决于新骨干的规模：若替换为 DINOv2-Small（22 M backbone）或 ConvNeXt-Tiny（27 M backbone）将使总参数量明显上升；若保持 backbone 在 B0 量级则总参数量相近。

扩散策略头中的 U-Net 架构采用了多尺度特征融合和条件注入机制。条件向量 $\mathbf{c}$ 通过 FiLM（Feature-wise Linear Modulation）机制注入到 U-Net 的各个分辨率层级中，使得噪声预测网络能够在每一步去噪中都感知到当前的视觉条件和目标信息。这种设计虽然增加了参数量，但保证了扩散过程对条件信号的充分响应。

关于推理频率的部署含义，40 ms 的纯采样时延在 RTX 4090 上对应约 25 Hz 的理论上限，但叠加图像预处理、消息传递或可视化后，端到端频率通常会明显低于该理论值。在边缘设备（如 Jetson Orin）上，实际频率还会受到功耗模式、散热状态、相机驱动和 PyTorch/CUDA 版本影响，因此不能仅按桌面 GPU 与边缘 GPU 的标称算力比例线性外推。本文把 DDIM 加速作为部署优化的直接动机，但具体真机可用频率以后续 Orin 阶段 benchmark 为准。

此外， 8×8 的轨迹采样规格表示每次推理生成 8 条候选轨迹，每条轨迹包含 8 个 waypoint。8 个 waypoint 的长度在默认配置下对应约 2–3 秒的未来规划窗口，这一时间尺度既足以覆盖局部导航决策（如是否转弯），又不至于因规划太远而积累过大误差。

表 3.1 NoMaD 基线离线推理结果

指标	含义	数值
模型参数量	可学习参数总量	19,049,675
端到端单次推理	含 I/O 与首次 kernel 编译	0.1250 s
纯采样时延	DDPM-10 循环体耗时（24 case 平均）	39.98 ms (CI95 ± 1.37)
纯采样推理频率	RTX 4090 单 batch 理论频率	25 Hz
轨迹采样规格	候选轨迹数 $\times$ waypoint 数	8×8
距离预测输出	候选目标相对距离估计均值	11.6589

3.4.1 基线推理可视化

为直观呈现 NoMaD 在导航与探索两种模式下的轨迹分布，本文对同一观察分别进行有目标条件采样与无目标条件采样，并在机器人当前位姿为原点的局部坐标系中绘制候选轨迹。图 3.2 与图 3.3 分别给出 navigate 模式与 explore 模式的可视化结果。

两幅可视化图共用相同的局部坐标系：红色五角星为机器人当前位置， x 轴指向



图 3.2 NoMaD 基线 navigate 模式轨迹可视化



图 3.3 NoMaD 基线 explore 模式轨迹可视化

机体正前方， y 轴指向机体左侧。两图都生成了不同颜色的八条候选轨迹。两组可视化共同说明，NoMaD 在同一组观察下能够通过 goal mask 切换在“被目标牵引的窄分布”与“沿走廊自由展开的宽分布”之间切换，验证了导航与探索共享同一权重的可行性。

3.5 实验指标与统计原则

为了避免仅凭单次样本或单一指标做结论，本文采用多指标联合分析。主要指标包括：Latency，用于衡量推理开销；CI95，用于表示均值的 95% 置信区间；MSE vs DDPM-10，用于衡量与基线采样分布的偏移；Forward Progress，用于刻画轨迹在主前进方向上的推进程度；Lateral Abs，用于衡量横向偏摆幅度；Smoothness，用于衡量轨迹变化的平滑性；Dist Pred Mean，用于观察候选目标距离评估是否随条件空间变化发生系统偏移。

3.5.1 指标的形式化定义

为确保实验可复现性，本文给出各指标的形式化定义。

Latency 定义为单次推理的端到端耗时，包含编码、去噪和后处理，单位为毫秒：

$$\text{Latency} = t_{\text{end}} - t_{\text{start}}. \quad (3.7)$$

CI95 表示均值的 95% 置信区间半径，基于 t 分布计算：

$$\text{CI95} = t_{0.975, n-1} \cdot \frac{s}{\sqrt{n}}, \quad (3.8)$$

其中 s 为样本标准差， n 为测试 case 数量。

MSE vs DDPM-10 衡量某配置生成轨迹均值与基线 DDPM-10 生成轨迹均值之间的均方误差：

$$\text{MSE} = \frac{1}{H} \sum_{i=1}^H \|\bar{\mathbf{a}}_i^{\text{test}} - \bar{\mathbf{a}}_i^{\text{base}}\|^2, \quad (3.9)$$

其中 $\bar{\mathbf{a}}_i$ 表示第 i 个 waypoint 在所有候选轨迹上的均值。

Forward Progress 定义为轨迹在主前进方向（ x 轴）上的累计位移：

$$\text{FP} = \sum_{i=1}^H \Delta x_i. \quad (3.10)$$

Lateral Abs 定义为轨迹在横向（ y 轴）上的累计绝对位移：

$$\text{LA} = \sum_{i=1}^H |\Delta y_i|. \quad (3.11)$$

Smoothness 衡量相邻 waypoint 之间方向变化的一致性，定义为相邻位移向量差的范数均值：

$$\text{Smooth} = \frac{1}{H-1} \sum_{i=1}^{H-1} \|\mathbf{a}_{i+1} - \mathbf{a}_i\|. \quad (3.12)$$

3.5.2 统计显著性与多指标分析原则

本文采用多指标联合分析而非单一指标排序，原因在于导航任务的优化目标本质上是多目标的。仅优化前进性可能导致激进轨迹，仅优化平滑性可能导致过于保守的运动。通过同时报告 Forward Progress、Lateral Abs、Smoothness 和 Latency，可以判断某个配置是否在所有维度上都可接受，而不是在某一维度上表现突出但在其他维度上严重退化。

在统计检验层面，由于本文的离线实验样本量为 24 个 case，属于小样本范畴。对于小样本比较，本文主要依赖 CI95 的重叠程度来判断差异的显著性：若两个配置的 CI95 不重叠，则认为差异具有统计意义；若 CI95 部分重叠，则需结合效应量大小（即均值差与标准差之比）进行综合判断。这种方法比直接进行 p 值检验更适合本文的实验规模和多次比较场景。

其中, CI95 并不是新的性能指标, 而是均值稳定性的统计描述。若某个配置平均值较优但 CI95 很宽, 说明其结果在不同 case 间波动较大。Forward Progress 也不能单独解释为越大越好, 因为过大的前进量可能是激进采样带来的分布偏移, 必须结合 MSE、Lateral Abs 和 Smoothness 一起分析。

3.6 DDIM 推理加速实验

3.6.1 理论动机

DDIM 的核心思想是将扩散模型的随机微分方程 (SDE) 采样过程转换为确定性常微分方程 (ODE) 求解过程^[19]。在标准 DDPM 中, 反向去噪过程由以下随机递推描述:

$$\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}, \quad (3.13)$$

其中 $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 引入了随机性。DDIM 的关键洞察是: 当令随机项 $\sigma_t = 0$ 时, 上述过程退化为确定性的 ODE 积分:

$$\mathbf{x}_{t-1} = \sqrt{\bar{\alpha}_{t-1}} \cdot \hat{\mathbf{x}}_0(\mathbf{x}_t, t) + \sqrt{1 - \bar{\alpha}_{t-1}} \cdot \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t), \quad (3.14)$$

其中 $\hat{\mathbf{x}}_0$ 是基于当前噪声预测的干净样本估计。

由于 ODE 的解路径是确定性的且连续的, 可以在时间轴上使用更大的步长进行积分而不引入额外随机噪声的累积。这就是 DDIM 能够用更少步数完成去噪的数学基础——本质上是用粗粒度的数值积分近似原始的细粒度 ODE 轨迹。步数越少, 积分越粗糙, 但对于 smooth 的 ODE 解路径, 适度的步数减少不会显著改变最终结果。

从数值分析的角度, DDIM 的步数减少可以类比为 ODE 求解器中步长的增大。对于一阶 Euler 方法, 截断误差与步长 h 成正比; 而 DDIM 的更新规则实际上对应一种隐式的高阶积分格式, 因此在步数适度减少时能保持较小的截断误差。但当步数降至极端 (如仅 1 步) 时, 单步积分的假设不再成立, 误差会急剧增大。

3.6.2 实验设计

DDIM 的作用是在不重新训练模型的情况下减少扩散去噪步数, 从而降低推理时延^[19]。本文以 `ddim_x` 来定义采用 DDIM 采样 x 步的配置。本文在统一脚本下比较 `ddpm_10`、`ddim_10`、`ddim_5`、`ddim_3`、`ddim_2` 和 `ddim_1` 六种配置, 测试集共包含 24 个 case。该实验的目标是回答两个问题: 第一, 步数减少后推理能快多少; 第二, 轨迹分布是否仍保持在可接受范围。

六种配置的选择覆盖了相比于基线，从“不减少步数”到“极端压缩至单步”的完整频谱。ddim_10 与 ddpm_10 步数相同，用于验证 DDIM 在不减步时是否引入额外偏差；ddim_5 和 ddim_3 代表中等程度的加速；ddim_2 代表较为激进但仍可控的加速；ddim_1 则代表理论极限，用于探测 DDIM 的失效边界。这种全频谱设计使得本文能够精确定位速度与质量的平衡点。

3.6.3 实验结果

表 3.2 DDIM 推理加速实验结果

配置	Latency/ms	CI95/ms	Speedup	MSE vs DDPM-10	Forward Progress	Diversity
ddpm_10	39.9790	1.3748	1.00×	0.0000	6.4998	0.251
ddim_10	38.0260	0.6034	1.05×	0.0062	6.4686	0.318
ddim_5	19.2581	0.4319	2.08×	0.0048	6.5257	0.271
ddim_3	11.6579	0.3438	3.43×	0.0124	6.6224	0.306
ddim_2	8.2588	0.2779	4.84×	0.0210	6.6471	0.377
ddim_1	4.5092	0.1427	8.87×	3.5168	9.2789	4.736

3.6.4 详细分析

ddim_10 与 ddpm_10 的 MSE 仅为 0.0062，说明在相同步数下 DDIM 的确定性采样与 DDPM 的随机采样生成的轨迹均值几乎一致，验证了 DDIM 的理论无损性。ddim_5 在步数减半的情况下 MSE 反而略低于 ddim_10 (0.0048 vs 0.0062)，这可能源于采样噪声的统计波动，也说明 5 步 ODE 积分的截断误差仍然可以忽略。ddim_3 的 MSE 上升至 0.0124，开始出现可观测的分布偏移但仍在可控范围内。ddim_2 的 MSE 为 0.0210，虽然进一步升高但 Forward Progress 仅从 6.50 变为 6.65，偏移幅度有限。

3.6.5 结果分析

结果表明，DDIM 的确能显著降低推理时延，其中 ddim_2 已接近 5 倍加速。更关键的是，ddim_2 在 MSE 与 Forward Progress 上仍保持相对平衡，说明它不是以严重破坏轨迹分布为代价换取速度。相比之下，ddim_1 虽然最快，但 MSE 大幅升高，说明轨迹已明显偏离基线分布，不适合作为稳定部署方案。

从速度-质量 Pareto 前沿的角度分析，ddim_2 到 ddim_5 构成了一段有效的 Pareto 前沿：在这个范围内，每减少一步都能带来显著的时延收益，而 MSE 增长仍在可控范围内。ddim_1 则明显偏离了这一前沿，属于“过度压缩”区域。

因此，本文将 ddim_2 视为后续闭环验证中最值得优先考虑的推理配置。它证明

了扩散策略并不必然意味着“质量高但太慢”，通过合理采样调度，NoMaD 可以进入更接近真实部署的时延区间。

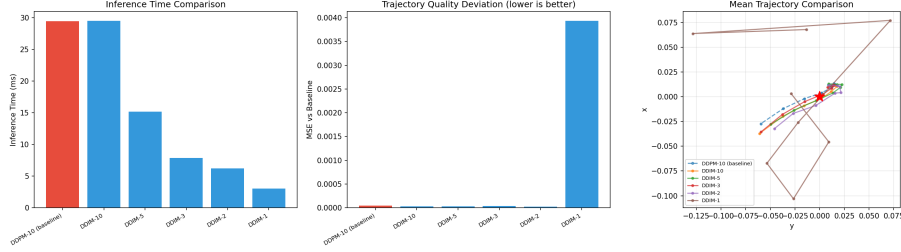


图 3.4 DDIM 加速实验结果图

3.7 CFG 引导增强实验

3.7.1 理论分析

Classifier-Free Guidance (CFG) 的核心思想是在推理时通过线性外推来增强条件信号对生成结果的影响^[23]。具体来说，CFG 修改了扩散模型的噪声预测（score function）为：

$$\tilde{\epsilon}_{\theta}(\mathbf{x}_t, t, \mathbf{c}) = (1 + w) \cdot \epsilon_{\theta}(\mathbf{x}_t, t, \mathbf{c}) - w \cdot \epsilon_{\theta}(\mathbf{x}_t, t, \emptyset), \quad (3.15)$$

其中 $\mathbf{c}$ 为条件向量（包含观察和目标信息）， $\emptyset$ 表示空条件（训练时通过随机 dropout 条件实现）， w 为引导权重。

当 $w = 0$ 时，退化为标准的条件采样；当 $w > 0$ 时，生成结果在条件方向上被“放大”，使轨迹更强烈地朝向目标方向偏移；当 $w < 0$ 时，则产生反向引导效果。从概率密度的角度，CFG 实质上是在对数概率空间中将条件分布与无条件分布的差异进行缩放：

$$\log \tilde{p}(\mathbf{x} | \mathbf{c}) \propto (1 + w) \log p(\mathbf{x} | \mathbf{c}) - w \log p(\mathbf{x}). \quad (3.16)$$

这意味着 w 越大，生成分布越集中于条件分布的高概率区域，同时远离无条件先验的模式。

3.7.2 实验设计

CFG 的目标是在推理阶段增强轨迹对目标条件的响应^[23]。本文选取 $w \in \{-1.0, 0.0, 0.5, 1.0, 2.0, 4.0\}$ 六组配置进行比较。实验探索最佳的引导强度的数值，能够在目标牵引与分布稳定性之间取得平衡。

负值引导 ($w = -1.0$) 被包含在实验中, 目的是验证反向引导是否会导致轨迹偏离目标方向。如果 $w = -1.0$ 的 Forward Progress 显著低于 $w = 0.0$, 则说明 CFG 的条件引导方向确实与目标方向一致, 从而间接验证了 CFG 机制在导航任务中的有效性。

3.7.3 实验结果与分析

结果表明, 适度的 CFG 能提高轨迹对目标方向的响应, 但代价是更高的推理时延和潜在的分布偏移。从表 3.3 可以看到, $w = 0.0$ 时推理时延约为 42.65 ms, 而 $w \geq 0.5$ 后时延跃升至约 78–80 ms, 这是因为 CFG 需要同时执行条件分支与无条件分支的噪声预测。CFG 的每一步去噪都需要对噪声预测网络进行两次前向传播——一次使用完整条件 $\mathbf{c}$, 一次使用空条件 $\emptyset$ ——然后将两次预测结果进行线性组合。因此, CFG 的计算成本近似为非 CFG 推理的 2 倍。实验中 $w = 0.0$ 到 $w = 0.5$ 的时延跳变 (42.65 ms $\rightarrow$ 78.27 ms) 正好反映了这一理论预期。

在轨迹质量方面, $w = 1.0$ 的 Forward Progress 最高 (8.573), Lateral Abs 仍保持适度 (2.901), 说明该配置在前进性和稳定性之间表现较为均衡。而 $w = 4.0$ 时 Shift 高达 0.5133, Lateral Abs 上升至 3.089, 表明过强的引导会导致分布偏移与横向偏摆。

关于最优引导尺度的选择, 本文认为应遵循以下准则: (1) 引导强度不应使 Shift 指标超过 0.2, 以避免显著的分布偏移; (2) 在满足 (1) 的前提下, 优先选择 Forward Progress 较高的配置; (3) 必须考虑时延代价——如果 CFG 带来的质量提升不能补偿 2 倍时延的代价, 则不建议开启。

从实验数据来看, $w = -1.0$ 的反向引导在 Shift 和 Forward Progress 上与 $w = 0.0$ 差异极小, 说明轻微的反向引导对轨迹分布的影响可以忽略。 $w = 0.5$ 和 $w = 1.0$ 的 Shift 分别为 0.0160 和 0.0476, 均在 0.2 的阈值以内, 属于可接受范围。 $w = 2.0$ 的 Shift 为 0.1438, 虽未超过阈值但已接近边界, 提示进一步增大引导可能带来不稳定性。综合以上分析, 本文将 CFG 定位为可调部署旋钮, 而非必须开启的固定默认项。

表 3.3 CFG 引导实验结果

配置	Latency/ms	Shift vs $w = 0$	Forward Progress	Lateral Abs
$w = -1.0$	42.28	0.0278	8.500	2.876
$w = 0.0$	42.65	0.0000	8.523	2.886
$w = 0.5$	78.27	0.0160	8.515	2.875
$w = 1.0$	79.75	0.0476	8.573	2.901
$w = 2.0$	78.81	0.1438	8.527	2.952
$w = 4.0$	78.92	0.5133	8.483	3.089

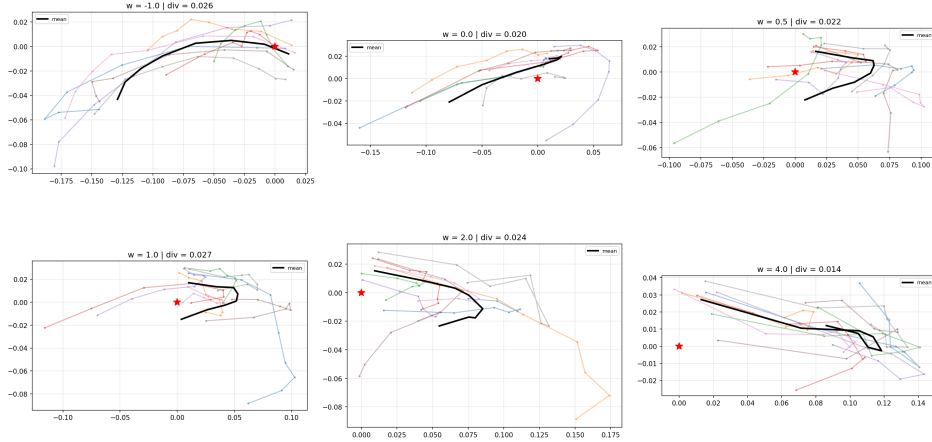
Classifier-Free Guidance: Trajectory Comparison for Different w 

图 3.5 CFG 引导实验结果图

3.8 TTS 推理时搜索实验

3.8.1 实验设计

TTS 的作用是把额外预算用于“候选生成与选择”，而非用于修改训练过程^[24-25]。在本文中，TTS 被设计为推理层的外层搜索组件：先用当前采样配置生成多条候选轨迹，再根据目标一致性、前进性和平滑性综合打分，最终选择一条更适合执行的轨迹。实验重点比较不同预算下的轨迹质量收益，以及它与 DDIM、CFG 的组合关系。

3.8.2 评分函数设计

TTS 的核心在于评分函数的设计。本文采用加权多目标评分函数：

$$\text{Score}(\mathbf{A}) = \beta_1 \cdot \text{FP}(\mathbf{A}) - \beta_2 \cdot \text{LA}(\mathbf{A}) - \beta_3 \cdot \text{Smooth}(\mathbf{A}), \quad (3.17)$$

其中 FP 为前进性，LA 为横向绝对偏摆，Smooth 为平滑性代价， $\beta_1, \beta_2, \beta_3$ 为权重系数。

评分函数中权重系数的选择需要根据具体部署场景进行调整。在开阔环境中，可以增大 β_1 以鼓励更积极的前进；在狭窄走廊中，应增大 β_2 以抑制横向偏摆；在需要精细控制的场景中， β_3 应占主导以保证轨迹平滑性。本文在离线实验中采用固定权重配置，但在部署阶段可以根据场景类型动态调整。

3.8.3 计算预算分配策略

TTS 的计算开销与候选轨迹数 (budget) 总体正相关。设单次采样耗时为 τ ，则 TTS- K 的总耗时可近似表示为候选生成开销与评分开销之和，其中评分与排序通常只占很小比例。由于实际实现会复用批量张量计算，耗时不会严格等于 $K \cdot \tau$ 。因此，DDIM 加速节省的时间预算可以部分重新分配给 TTS，形成“用更快的单次采样换取更多候选”的策略。

这种预算分配思路可以形式化为一个受约束的优化问题：在总时延预算 $T_{\max}$ 的约束下，选择 DDIM 步数 S 和 TTS budget K 以最大化轨迹质量。由于 $K \cdot \tau(S) \leq T_{\max}$ ，减少 S （加速单次采样）就能增大 K （增加候选数量），两者之间存在自然的折中关系。

3.8.4 实验组织方式

为了保证结论具有统计意义，本文将 TTS 实验组织为三层：单因素实验，用于比较是否开启 TTS 以及不同 budget；两两组合实验，用于比较 DDIM+TTS、CFG+TTS 的相互作用；三因素联合实验，用于比较 DDIM、CFG、TTS 同时开启时是否存在部署上更优的折中组合。

三层实验设计的逻辑是：单因素实验确认 TTS 本身的有效性和最优 budget 范围；两两组合实验检验 TTS 与其他优化手段之间是否存在交互效应；三因素联合实验给出面向实际部署的推荐配置。

3.8.5 结果与分析

在讨论具体数值前，需要先澄清一个容易混淆的概念层次，以避免“默认生成 8 条候选”与“TTS-8”发生语义重合。NoMaD 扩散策略头的每次前向本身会以批次形式并行生成若干条候选轨迹（本文固定为 8 条），这一批次规格由扩散头本身决定，与是否启用 TTS 无关。TTS 在此基础上额外引入两个正交的组件：一是候选池预算 K （即 `tts_budget`），当 $K > \text{num_samples}$ 时通过多次扩散采样把候选池从默认的 8 条扩展到 K 条；二是评分筛选器，对池内的 K 条候选按多目标评分排序，保留 `top-k` 条作为执行输出。因此，实验配置中的“`No TTS`”与“`TTS-8`”都只触发一次扩散采样、生成同样的 8 条候选，差别只在于前者把候选集原样送入下游统计，后者多了一步基于评分的排序与 `top-k` 选择；而 TTS-16 和 TTS-32 则分别对应 2 次和 4 次扩散采样，用于真正扩展候选池规模。

TTS 实验的核心问题不是“是否绝对提升所有指标”，而是“是否值得把 DDIM 节

省下来的预算重新分配给候选搜索”。本实验在 12 个统一离线 case 上比较 DDIM 步数、CFG 权重与 TTS budget 的组合，表 3.4 给出在 DDIM-2、 $w = 0$ 条件下的单因素结果。

表中的单因素结果可以分解为两种不同来源的收益。从 No TTS 到 TTS-8，综合评分由 8.157 上升到 9.024，前进性由 9.010 上升到 9.697，而时延从 7.00 ms 变为 6.89 ms（两者在噪声范围内等价）——这段增益完全来自 TTS，在同一批 8 条候选上额外执行一次排序与 top- k 选择，几乎没有采样开销。从 TTS-8 到 TTS-16 再到 TTS-32，时延分别上升到约 2 倍和 4 倍，对应 1 次、2 次和 4 次扩散采样批次，这一段增益才真正来自“候选池扩容”：TTS-16 的综合评分 9.124 仅比 TTS-8 高 0.10，而 TTS-32 的综合评分反而回落到 8.825。TTS-8 在三因素联合排序（表 3.5）中综合表现最优，它近乎无时延，达到了 TTS 算法的收益，同时把剩余预算留给 DDIM 步数与 CFG 权重的组合。

表 3.4 TTS 推理时搜索单因素实验结果 (DDIM-2, $w = 0$)

配置	Latency/ms	Forward Progress	Smoothness	综合评分
No TTS	7.00	9.010	0.0811	8.157
TTS-8	6.89	9.697	0.0692	9.024
TTS-16	13.86	10.050	0.0627	9.124
TTS-32	26.50	10.344	0.0527	8.825

进一步观察 DDIM、CFG 与 TTS 的三因素联合结果，综合排名前列的配置均集中在 DDIM-2 或 DDIM-3，并采用 TTS-8 或 TTS-16。表 3.5 给出 Top-5 组合。最优配置为 ddim2_cfg2.0_tts8，其综合评分达到 9.143；但与 ddim2_cfg0.0_tts16 和 ddim2_cfg0.0_tts8 的差距较小。考虑 CFG 会额外增加一次条件/无条件推理，实际部署中更稳妥的推荐是以 DDIM-2 + TTS-8/16 为默认候选，再把 CFG 作为可调开关。

表 3.5 DDIM $\times$ CFG $\times$ TTS 联合配置 Top-5 结果

排名	配置	Latency/ms	Forward Progress	Smoothness	综合评分
1	ddim2_cfg2.0_tts8	12.55	10.173	0.0553	9.143
2	ddim2_cfg0.0_tts16	13.86	10.050	0.0627	9.124
3	ddim2_cfg0.0_tts8	6.89	9.697	0.0692	9.024
4	ddim2_cfg1.0_tts8	12.52	9.941	0.0628	9.003
5	ddim2_cfg2.0_tts16	24.60	10.495	0.0521	8.987

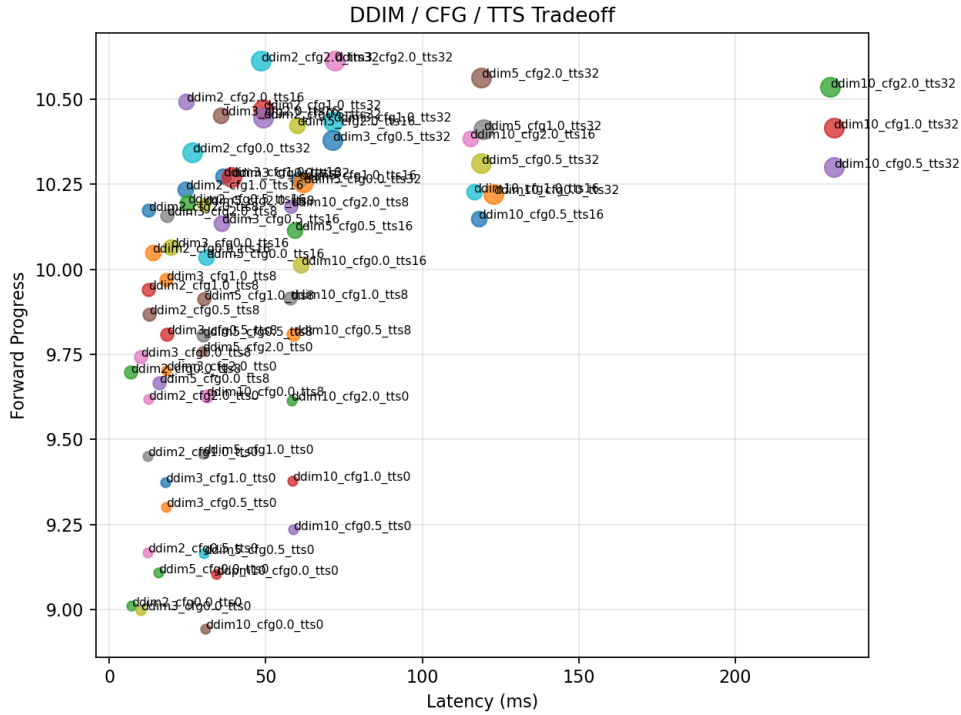


图 3.6 TTS 联合实验的时延与轨迹质量折中

3.9 视觉编码器升级实验

3.9.1 实验目标

视觉编码器升级实验的内容是在统一框架下对候选编码器进行公平训练，目的是在控制变量条件下评估不同编码器的表征潜力。

3.9.2 候选编码器结构与对比

本文在 EfficientNet-B0 之外纳入三类视觉骨干作为升级候选。DINOv2-Small^[28] 基于 Vision Transformer 架构，将图像切分为固定大小 patch 后送入 12 层 Transformer 编码器，参数量约 22 M；自监督预训练赋予其较强的跨域泛化性，全局自注意力对远距离方向判断更敏感。ConvNeXt-Tiny^[29] 是借鉴 Transformer 设计原则的现代 CNN，采用更大卷积核、LayerNorm 与更少激活函数，在保留 CNN 局部归纳偏置的同时提升表征能力，参数量约 28 M。ResNet-50^[30] 作为经典残差 CNN 提供稳定基线，结构成熟、权重广泛可用，参数量约 25 M。三者与 EfficientNet-B0 的 MBConv (5.3 M) 形成“自监督 ViT、现代 CNN、经典 CNN”的对照，覆盖表征机制与参数量两条对比维度。

3.9.3 特征维度对齐

四种编码器的原始输出维度并不一致：EfficientNet-B0 输出 1280 维，DINOv2-Small 输出 384 维，ConvNeXt-Tiny 输出 768 维，ResNet-50 输出 2048 维。为保证公平对比，本文在每个编码器输出端附加一个线性投影层，将原始特征统一映射到固定 token 维度 d ：

$$\mathbf{z} = W_{\text{proj}} \cdot \mathbf{f}_{\text{enc}} + \mathbf{b}_{\text{proj}}, \quad (3.18)$$

其中 $\mathbf{f}_{\text{enc}}$ 为编码器原始输出， $W_{\text{proj}} \in \mathbb{R}^{d \times d_{\text{enc}}}$ 为投影矩阵，与下游模块联合训练。投影后所有编码器在相同信息瓶颈下进入距离预测头与扩散策略头，避免条件空间容量差异干扰对比结果。

3.9.4 训练配置

在 GoStanford 数据集上对四种编码器分别进行 100 轮训练，batch size 为 32，学习率与优化器配置统一。所有 backbone 参数参与微调，以便在相同训练轮数下最大化各编码器的适配空间。

3.9.5 实验结果与分析

四种编码器在相同离线 case、相同 DDPM-10 采样设置下的代理指标如表 3.6 所示。从指标分布看，各编码器在不同维度上表现各有侧重：ResNet-50 在 diversity (0.9824) 与 forward progress (7.980) 上最高，候选轨迹最分散、前进性最强；EfficientNet-B0 的 smoothness 最低 (0.0725)，原始基线在轨迹平滑性上仍具有优势；DINOv2-Small 与 ConvNeXt-Tiny 在三项质量指标上处于中段。延迟方面，四种编码器单次前向延迟均在 34–37 ms 区间，差异显著小于 DDIM 步数变化所带来的影响，因此编码器选择主要受表征质量约束而非延迟约束。

由于 diversity、forward progress 与 smoothness 三者之间存在内在权衡——更分散的轨迹候选意味着更大的方向变化，因此 smoothness 通常下降——视觉编码器升级不是单一指标最大化问题，而是在多样性、前进性与平滑性之间寻找折中。本实验为第二阶段的部署训练提供了候选筛选依据。

3.10 联合结论

综合本章离线实验，表 3.7 进一步给出了 DDIM 与 CFG 联合配置的前五名结果。从中可以看出，ddim2_w0.0 在时延、前进性、平滑性和多样性的综合评分中排名第一，说明在不额外开启 CFG 的情况下，DDIM-2 本身已经具备较好的综合表现。综

表 3.6 视觉编码器升级实验结果

编码器	Latency/ms	Diversity	Forward Progress	Smoothness
EfficientNet-B0	35.54	0.5818	7.356	0.0725
DINOv2-Small	35.88	0.3182	7.097	0.1158
ConvNeXt-Tiny	37.10	0.3017	7.014	0.1322
ResNet-50	34.81	0.9824	7.980	0.0980

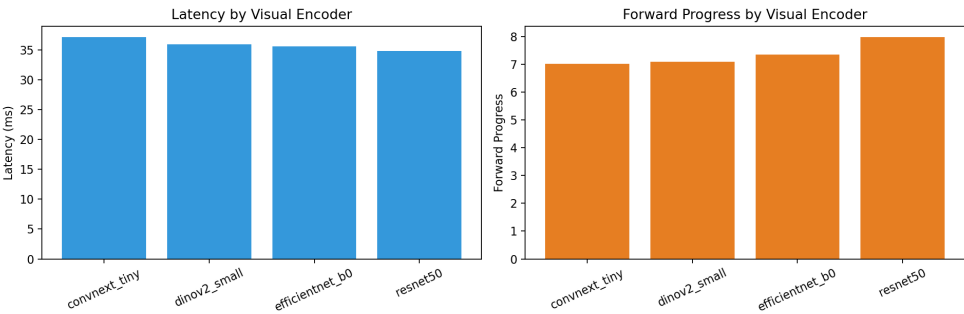


图 3.7 四种视觉编码器离线统计对比

合评分的计算方式为各指标归一化后的加权求和，权重分配反映了部署场景中对低时延和轨迹质量的均衡需求。

表 3.7 DDIM × CFG 联合配置 Top-5 结果

排名	配置	Latency/ms	Forward Progress	Smoothness	Diversity	综合评分
1	ddim2_w0.0	9.99	8.089	0.1237	0.3897	5.0919
2	ddim3_w0.0	14.60	8.076	0.1075	0.3208	4.9942
3	ddim5_w0.0	24.63	8.032	0.1043	0.2906	4.7481
4	ddim3_w1.0	26.62	8.109	0.1096	0.3070	4.7227
5	ddim3_w0.5	27.04	8.093	0.1083	0.3130	4.7174

基于上述结果，可得到四点阶段性结论。第一，在不改动训练结果的前提下，`ddim_2` 是当前最值得进入闭环验证的加速配置。第二，CFG 的有效范围有限，更适合作为可调参数而不是默认固定设置。第三，TTS 的收益可进一步分解为“评分筛选”（在固有 8 条候选上做排序与 `top-k` 选择，几乎零时延代价）与“候选池扩容”（ $K > 8$ 时多批次采样，时延线性增长且边际收益快速衰减）两部分，TTS-8 以筛选为主、代价可忽略，继续增大 TTS 的参数则只能取得有限的增益，因此部署上推荐以 TTS-8 为默认档位、TTS-16 作为可选的增强方式。第四，视觉编码器升级已完成四个训练权重的离线统计，但最终部署选择仍需结合闭环验证与真机条件共同判断。

3.10.1 优化方法之间的关系

四种优化方法并不处在同一层级。视觉编码器升级位于视觉条件层，改变的是历史观察和目标图像进入模型后的特征表达；DDIM 与 CFG 位于扩散采样层，改变的是给定视觉条件后轨迹样本的生成方式；TTS 位于候选选择层，改变的是生成多条候选轨迹之后如何筛选最终执行轨迹。表 3.8 总结了各方法的作用位置与实验含义。

表 3.8 算法优化方法的作用层级与当前结论

优化方法	作用层级	是否改变训练	当前结论
DDIM	扩散采样调度	否	DDIM-2 默认最优，跨四编码器鲁棒
CFG	条件引导强度	否	时延翻倍而无可测质量收益，默认关闭
TTS	候选轨迹筛选	否	TTS-8 近零代价，增益随候选 Diversity 放大
视觉编码器升级	视觉条件建模	是	四权重可替换，影响 Diversity 进而影响 TTS 增益

因此，DDIM、CFG 与 TTS 更适合被写成统一推理层内部的层级组合：先由 DDIM 决定每条候选轨迹的去噪步数与生成速度，再由 CFG 决定采样过程中条件信号的放大程度，最后由 TTS 在候选轨迹集合上进行评分和选择。三者并不是简单并列相加，而是在“生成单条轨迹—增强条件响应—筛选候选轨迹”的流水线上依次发挥作用。算法 3.1 给出了四种优化方法在一次高层推理周期内的具体执行顺序，说明它们是如何在同一组观察输入上从“视觉表征—去噪采样—条件引导—候选筛选”逐层组合，最终输出一条供中层控制器跟踪的执行轨迹。

算法 3.1 揭示了四种优化方法的层级关系：视觉编码器升级位于最外层，决定了条件向量 $\mathbf{c}$ 与 $\mathbf{c}_o$ 的特征分布；DDIM 步数 S 控制中间去噪循环的长度，是单条候选轨迹生成成本的主要来源；CFG 权重 w 在每一去噪步内通过条件/无条件外推增强目标响应，代价是每步多一次前向；TTS 预算 K 决定候选池规模，当 $K = N_0 = 8$ 时只触发一次扩散批次（仅做评分排序），当 $K > N_0$ 时通过额外批次扩展候选池。本文最终采用的默认配置为 $S = 2, w = 0, K = 8$ ，对应“两步 DDIM 采样一批 8 条候选 → 跳过 CFG 外推 → 直接对池内 8 条候选评分取最优”，这一组合在算法 3.1 的流程上把每一层的代价都压到最低，同时保留了多模态采样和评分筛选两项核心收益。

视觉编码器升级与前三者既相对独立，又存在间接联系。独立性体现在工程接口上：编码器由 `policy_config` 与 `policy_checkpoint` 决定，而 DDIM、CFG、TTS 由推理参数决定，只要模型保持 NoMaD 兼容输出，下游状态机和平台接口无需修改。联系

算法 3.1 统一推理模块在单次高层周期内的执行流程

Input: 历史观察 $\mathcal{O}_t = \{I_{t-K}, \dots, I_t\}$, 目标图像 I_g ; 视觉编码器权重 θ_{enc} ; DDIM 步数 S ; CFG 权重 w ; TTS 候选预算 K ; 扩散头每次前向产生的候选数 $N_0 = 8$

Output: 执行轨迹 $\mathbf{A}^* \in \mathbb{R}^{H \times 2}$

// 1. 视觉编码器升级层: 根据 θ_{enc} 提取 token, 得到统一条件向量

- 1 $\mathbf{c} \leftarrow \text{VisionEncoder}_{\theta_{\text{enc}}}(\mathcal{O}_t, I_g)$;
- 2 $\mathbf{c}_{\emptyset} \leftarrow \text{VisionEncoder}_{\theta_{\text{enc}}}(\mathcal{O}_t, \text{mask})$;

// 2. DDIM 采样层: 用 S 步 DDIM 一次性生成 N_0 条候选轨迹

- 3 $M \leftarrow \lceil K/N_0 \rceil$; // 扩散批次: $K = N_0 = 8$ 时 $M = 1$
- 4 $C \leftarrow \emptyset$;
- 5 **for** $m \leftarrow 1$ **to** M **do**
- 6 $\mathbf{x}_S \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, 形状为 $N_0 \times H \times 2$;
- 7 **for** $s \leftarrow S$ **to** 1 **do**
- 8 // 3. CFG 引导层: 在每个去噪步同时跑条件与无条件分支并外推
- 8 $\epsilon_{\text{cond}} \leftarrow \epsilon_{\theta}(\mathbf{x}_s, s, \mathbf{c})$;
- 9 **if** $w \neq 0$ **then**
- 10 $\epsilon_{\text{uncond}} \leftarrow \epsilon_{\theta}(\mathbf{x}_s, s, \mathbf{c}_{\emptyset})$;
- 11 $\epsilon \leftarrow (1 + w)\epsilon_{\text{cond}} - w\epsilon_{\text{uncond}}$;
- 12 **else**
- 13 $\epsilon \leftarrow \epsilon_{\text{cond}}$;
- 14 **end**
- 15 $\mathbf{x}_{s-1} \leftarrow \text{DDIMStep}(\mathbf{x}_s, \epsilon, s)$;
- 16 **end**
- 17 $C \leftarrow C \cup \{\mathbf{x}_0^{(1)}, \dots, \mathbf{x}_0^{(N_0)}\}$; // 累积候选池
- 18 **end**

// 4. TTS 候选筛选层: 对池内 $|C|$ 条候选打分并取最优

- 19 **foreach** $\mathbf{A}^{(i)} \in C$ **do**
- 20 $s_i \leftarrow \beta_1 \cdot \text{FP}(\mathbf{A}^{(i)}) - \beta_2 \cdot \text{LA}(\mathbf{A}^{(i)}) - \beta_3 \cdot \text{Smooth}(\mathbf{A}^{(i)})$;
- 21 **end**
- 22 $\mathbf{A}^* \leftarrow \arg \max_i s_i$;
- 23 **return** $\mathbf{A}^*$;

则体现在条件分布上: 更换编码器会改变历史观察和目标图像的 token 表达, 从而同时影响距离预测头和扩散策略头。补齐后的编码器 $\times$ DDIM/TTS/CFG 联合消融 (见表 3.9–3.11) 给出的经验结论是: DDIM 步数选择与 CFG 权重选择对编码器替换具有鲁棒性, 其中 DDIM-2 在四个编码器上都是默认最优档, CFG-1/2 在任何一个编码器上都无法换取有意义的质量增益; 唯一与编码器耦合的是 TTS-8 的筛选增益——初始候选 Diversity 越高 (如 ResNet-50), TTS-8 的相对提升越大。换言之, 前三个优化中的 “DDIM+CFG” 部分在既定视觉条件空间内调整推理过程而不受编码器影响,

TTS 部分则在一定程度上继承编码器带来的分布变化，视觉编码器升级则是在源头上改变条件空间。

3.10.2 消融验证与编码器联合补充

本章已经完成了主要算法变量的单因素消融和联合消融。DDIM 实验固定模型与数据 case，仅改变采样步数，验证了 DDIM-2 的速度收益与 DDIM-1 的失效边界；CFG 实验固定模型，仅改变引导强度，说明其质量收益不足以抵消默认开启时的时延代价；TTS 实验固定 DDIM 与 CFG 条件，比较不同候选预算，说明主要收益来自 TTS-8 的低成本筛选；视觉编码器实验在统一训练配置下比较四个训练权重，证明了编码器替换框架可训练、可评估、可接入闭环。

跨视觉编码器的联合消融对判断“DDIM/CFG/TTS 的最优参数是否稳定”至关重要：如果更换编码器会改变候选轨迹的多样性和平滑性，那么 TTS 的评分选择、CFG 的目标引导强度和 DDIM 的少步近似误差都可能随之漂移。本文在相同的离线 case 构造规则下进行实验，完成了“编码器 $\times$ DDIM/CFG/TTS”联合矩阵，4 个训练权重（EfficientNet-B0、DINOv2-Small、ConvNeXt-Tiny、ResNet-50）统一以 DDIM 调度器为基础，分别扫描 DDIM-2/3/5 步数、TTS-0/8/16 候选预算以及 CFG-0/1/2 引导权重，每个配置在 12 个离线 case 上重复 3 次。CFG 一项限定在 EfficientNet-B0 与 ResNet-50 这两个初始多样性差异最大的编码器上，以小规模方式验证目标引导是否会放大编码器差异。三组结果分别由表 3.9、表 3.10 和表 3.11 给出，所有条目均已带 CI95 置信区间。

表 3.9 编码器 $\times$ DDIM 步数联合消融（CFG=0, 无 TTS, 12 case $\times$ 3 runs）

编码器	配置	Latency/ms	Forward Progress	Diversity	Smoothness
EfficientNet-B0	DDIM-2	5.65 $\pm$ 0.16	8.290 $\pm$ 0.184	0.611 $\pm$ 0.052	0.130 $\pm$ 0.009
EfficientNet-B0	DDIM-3	8.61 $\pm$ 0.36	8.264 $\pm$ 0.186	0.599 $\pm$ 0.053	0.127 $\pm$ 0.009
EfficientNet-B0	DDIM-5	13.04 $\pm$ 0.37	8.226 $\pm$ 0.189	0.569 $\pm$ 0.052	0.127 $\pm$ 0.010
DINOv2-Small	DDIM-2	5.67 $\pm$ 0.14	6.607 $\pm$ 1.942	0.390 $\pm$ 0.066	0.166 $\pm$ 0.045
DINOv2-Small	DDIM-3	8.12 $\pm$ 0.23	6.586 $\pm$ 1.963	0.358 $\pm$ 0.060	0.163 $\pm$ 0.045
DINOv2-Small	DDIM-5	12.77 $\pm$ 0.22	6.525 $\pm$ 1.990	0.327 $\pm$ 0.055	0.163 $\pm$ 0.046
ConvNeXt-Tiny	DDIM-2	5.41 $\pm$ 0.09	6.768 $\pm$ 2.364	0.460 $\pm$ 0.124	0.152 $\pm$ 0.046
ConvNeXt-Tiny	DDIM-3	8.04 $\pm$ 0.17	6.744 $\pm$ 2.386	0.438 $\pm$ 0.132	0.147 $\pm$ 0.045
ConvNeXt-Tiny	DDIM-5	13.32 $\pm$ 0.31	6.687 $\pm$ 2.433	0.415 $\pm$ 0.136	0.146 $\pm$ 0.046
ResNet-50	DDIM-2	5.41 $\pm$ 0.10	8.085 $\pm$ 0.290	0.943 $\pm$ 0.096	0.109 $\pm$ 0.011
ResNet-50	DDIM-3	7.82 $\pm$ 0.12	8.049 $\pm$ 0.307	0.992 $\pm$ 0.106	0.107 $\pm$ 0.013
ResNet-50	DDIM-5	12.79 $\pm$ 0.18	8.004 $\pm$ 0.332	1.001 $\pm$ 0.117	0.106 $\pm$ 0.015

能从表 3.9 得到的结论是：对于全部四个编码器，Forward Progress、Diversity、

Smoothness 在 DDIM-2/3/5 之间的变化都显著小于各编码器自身的 case 间标准差。以 EfficientNet-B0 为例，三个步数下的 Forward Progress 分别为 8.290、8.264、8.226，相邻差异远小于 CI95 (± 0.19)；ResNet-50 从 DDIM-2 的 8.085 到 DDIM-5 的 8.004 亦在统计不可分水平。与此形成对照的是，时延随步数线性增长：DDIM-2 约 5.4–5.7 ms、DDIM-3 约 7.8–8.6 ms、DDIM-5 约 12.8–13.3 ms。因此本文在 DDIM 推理加速实验一节中得到的“DDIM-2 是默认最优加速配置”结论在视觉条件分布发生替换后依然成立，即 DDIM 步数选择对视觉编码器替换具有鲁棒性。

表 3.10 编码器 $\times$ TTS 候选预算联合消融 (DDIM-2, CFG=0, 12 case $\times$ 3 runs)

编码器	配置	Latency/ms	Forward Progress	Diversity	Smoothness
EfficientNet-B0	TTS-0	5.42 $\pm$ 0.14	8.290 $\pm$ 0.184	0.611 $\pm$ 0.052	0.130 $\pm$ 0.009
EfficientNet-B0	TTS-8	5.51 $\pm$ 0.20	9.077 $\pm$ 0.111	0.416 $\pm$ 0.024	0.107 $\pm$ 0.008
EfficientNet-B0	TTS-16	10.81 $\pm$ 0.30	9.465 $\pm$ 0.129	0.381 $\pm$ 0.035	0.095 $\pm$ 0.010
DINOv2-Small	TTS-0	5.43 $\pm$ 0.15	6.607 $\pm$ 1.942	0.390 $\pm$ 0.066	0.166 $\pm$ 0.045
DINOv2-Small	TTS-8	5.54 $\pm$ 0.14	7.013 $\pm$ 1.922	0.307 $\pm$ 0.070	0.156 $\pm$ 0.046
DINOv2-Small	TTS-16	10.71 $\pm$ 0.21	7.259 $\pm$ 1.910	0.283 $\pm$ 0.064	0.150 $\pm$ 0.045
ConvNeXt-Tiny	TTS-0	5.53 $\pm$ 0.20	6.768 $\pm$ 2.364	0.460 $\pm$ 0.124	0.152 $\pm$ 0.046
ConvNeXt-Tiny	TTS-8	5.55 $\pm$ 0.13	7.085 $\pm$ 2.321	0.321 $\pm$ 0.090	0.141 $\pm$ 0.044
ConvNeXt-Tiny	TTS-16	11.02 $\pm$ 0.24	7.295 $\pm$ 2.317	0.328 $\pm$ 0.094	0.144 $\pm$ 0.055
ResNet-50	TTS-0	5.48 $\pm$ 0.16	8.085 $\pm$ 0.290	0.943 $\pm$ 0.096	0.109 $\pm$ 0.011
ResNet-50	TTS-8	5.53 $\pm$ 0.21	9.407 $\pm$ 0.185	0.486 $\pm$ 0.076	0.087 $\pm$ 0.011
ResNet-50	TTS-16	10.69 $\pm$ 0.16	9.969 $\pm$ 0.119	0.343 $\pm$ 0.033	0.080 $\pm$ 0.018

与 DDIM 的鲁棒性结论不同，表 3.10 揭示了一个编码器相关的规律：TTS-8 相对于 TTS-0 的 Forward Progress 增益幅度与初始 Diversity 正相关。具体地，ResNet-50 的初始 Diversity 为 0.943，TTS-8 将 Forward Progress 从 8.085 提升至 9.407 (+16.4%)；EfficientNet-B0 初始 Diversity 为 0.611，TTS-8 增益为 +9.5%；ConvNeXt-Tiny 初始 Diversity 为 0.460，增益为 +4.7%；DINOv2-Small 初始 Diversity 为 0.390，增益为 +6.0%。这一规律说明 TTS-8 对结果的增益依赖于扩散头生成候选时本身是否具有足够的多样性——初始多样性越高，基于多目标评分的 top- k 选择就越有可能在同一批 8 条候选中找出显著更好的子集。同时 Smoothness 和 Latency 也随 TTS-8 普遍下降，说明 top- k 选择不仅向前推进更远，还倾向于更平稳和更接近目标方向的候选。TTS-16 继续把这些指标往同一方向推进，但其代价是时延翻倍（从约 5.5 ms 增至约 10.7–11.0 ms）。因此“TTS-8 作为默认档位、TTS-16 作为可选增强”的部署建议在四个编码器上均成立。

能表 3.11 看出来的是：在这两个初始 Diversity 差异最大的编码器上，CFG-1/2 对

表 3.11 编码器 $\times$ CFG 权重小规模联合消融 (DDIM-2, 无 TTS, 12 case $\times$ 3 runs)

编码器	配置	Latency/ms	Forward Progress	Diversity	Lateral Abs	Smoothness
EfficientNet-B0	CFG=0	5.39 $\pm$ 0.20	8.290 $\pm$ 0.184	0.611 $\pm$ 0.052	2.361 $\pm$ 0.240	0.130 $\pm$ 0.009
EfficientNet-B0	CFG=1	10.08 $\pm$ 0.21	8.325 $\pm$ 0.143	0.514 $\pm$ 0.047	2.804 $\pm$ 0.351	0.138 $\pm$ 0.012
EfficientNet-B0	CFG=2	10.04 $\pm$ 0.16	8.363 $\pm$ 0.138	0.434 $\pm$ 0.042	3.226 $\pm$ 0.426	0.147 $\pm$ 0.015
ResNet-50	CFG=0	5.56 $\pm$ 0.18	8.085 $\pm$ 0.290	0.943 $\pm$ 0.096	2.016 $\pm$ 0.290	0.109 $\pm$ 0.011
ResNet-50	CFG=1	10.18 $\pm$ 0.32	8.095 $\pm$ 0.310	0.942 $\pm$ 0.101	1.983 $\pm$ 0.318	0.108 $\pm$ 0.012
ResNet-50	CFG=2	9.93 $\pm$ 0.11	8.105 $\pm$ 0.335	0.940 $\pm$ 0.107	1.960 $\pm$ 0.339	0.108 $\pm$ 0.013

Forward Progress 的提升都在 1% 以内，而时延无一例外地翻倍——CFG 开启需要一次条件分支与一次无条件分支的额外前向。以 EfficientNet-B0 为例，CFG 从 0 增至 2 时 Forward Progress 仅由 8.290 上升至 8.363，同时 Lateral Abs 从 2.361 增至 3.226（横向偏摆放大了近 37%），Diversity 从 0.611 压缩到 0.434；换言之，CFG 的外推方向对 EfficientNet-B0 的候选分布有“收紧加放大偏摆”的双重扰动。ResNet-50 的反应更平淡：它所有指标在 CFG=0/1/2 之间几乎保持不变，Diversity 从 0.943 仅微降到 0.940，Forward Progress 和 Smoothness 的变化都在 CI95 之内。这一对比说明：CFG 对编码器差异既没有“放大优势”也没有“补偿劣势”，反而使原本 Diversity 较低、结构较紧的 EfficientNet-B0 受到更明显的分布扰动。综合两个编码器的结果，本文原先得到的“默认 CFG=0”结论得到了进一步验证。

三组联合消融实验共同说明，DDIM 步数选择对编码器替换鲁棒，TTS 筛选收益随候选多样性放大，CFG 引导则在当前视觉条件空间内缺乏明显增益。因此，最终算法框架确定一节推荐的“EfficientNet-B0 + DDIM-2 + CFG=0 + TTS-8”默认方案在跨编码器视角下仍然成立。

3.10.3 最终算法框架确定

综合本章的全部离线实验，本文将最终算法框架确定为：EfficientNet-B0 视觉编码器、DDIM-2 采样、CFG=0、TTS-8。这一组配置随后被提炼为系统层的“统一推理模块”：在第 4.2 节定义的分层架构中，编码器权重由 policy_checkpoint 选定，DDIM 步数、CFG 权重与 TTS 候选预算分别由 ddim_steps、cfg_weight 与 tts_budget 三个推理参数承载，而算法 3.1 描述的执行顺序则成为该模块对外提供的标准推理流程。换言之，本章实验的目的不是给出一组孤立的最佳数值，而是确定推理模块在部署侧应当采用的默认参数与执行模板，使下游状态机、导航主机和平台后端只需通过修改配置即可复用同一套高层推理代码。

为了适配不同部署需求，可在该默认配置之上给出三个候选层级。低时延默认方

案与上述最终配置一致，是统一推理模块的出厂默认值；稳定方案将 DDIM 步数提高到 3 或 5，并保持 CFG=0 与 TTS-8，用于对推理时延不敏感、对轨迹光滑性要求更高的场景；复杂场景增强方案在默认配置上额外开启 CFG=0.5/1.0 或 TTS-16 作为可选项，用于强目标牵引或更激进的候选筛选。三种层级共享同一份推理模块代码与同一组对外接口，差别仅在配置文件中的几行参数。视觉编码器升级实验的价值则在于建立“可替换、可重训、可评测”的编码器训练与评估框架，并发现潜在的视觉编码器替换候选，而不是直接证明 DINOv2 或 ResNet-50 已经全面优于原始的 EfficientNet-B0；该框架同样以 `policy_config/policy_checkpoint` 的形式接入统一推理模块，对后续章节的系统集成保持透明。

3.11 本章小结

本章围绕 NoMaD 的算法层问题展开研究。首先从 MDP 视角对视觉导航任务进行了形式化建模，分析了条件轨迹分布与传统策略的联系，并论证了 `waypoint` 表示在平台无关性、多模态表达和闭环控制方面的优势。其次，复现并梳理了 NoMaD 的视觉编码器架构、`token` 构建过程、训练目标和统一推理接口。在此基础上，以基线实验为参照，对 DDIM 推理加速、CFG 引导增强、TTS 推理时搜索和视觉编码器升级进行了系统分析。

第 4 章 算法框架设计与仿真环境验证

4.1 本章目标

本章的重点是统一推理模块、状态机、导航主机、中层控制映射与 MuJoCo 后端如何协同工作，并在仿真环境中形成可执行、可恢复、可扩展的闭环链路。

4.2 统一闭环系统架构

4.2.1 分层设计

本文将系统划分为四层：推理层、任务组织层、中层控制层和平台执行层。推理层负责根据视觉观察与目标图像生成局部轨迹；任务组织层负责状态切换、任务队列与模式切换；中层控制层负责将轨迹映射为速度或姿态参考；平台执行层负责向 MuJoCo 或真机后端下发指令。

这种四层划分的设计动机源自足式导航系统的本质需求。推理层的存在使得高层策略可以独立于具体平台进行开发和优化：无论底层是四足机器人还是轮式平台，高层 NoMaD 生成的局部轨迹格式保持一致。任务组织层的引入是因为实际部署中，导航任务往往不是单次调用，而是涉及多目标队列、模式切换和异常恢复等复杂流程，这些逻辑不应侵入推理层或控制层。中层控制层的核心价值在于不同平台的可复用性。不同的机器人本体或者是导航平台，运动控制的实现可能是不同的，但是接收速度指令的机制是相同的。平台执行层则直接面向硬件或仿真后端，负责指令下发和传感器回传。

与单体式设计相比，分层架构虽然引入了层间通信开销，但带来的解耦优势远大于性能损失。在单体式设计中，推理逻辑、任务组织和平台控制混杂在同一模块内，任何局部修改都可能引发全局回归。例如，当需要将 DDIM 步数从 4 调整为 2 时，单体式系统可能需要同时修改推理、控制和部署脚本，而在分层架构中，这一变更完全限制在推理层内部。此外，单体式设计在面对多平台部署时需要维护多套几乎重复的代码，而分层架构只需替换平台执行层即可。从工程实践角度来看，分层架构还简化了调试流程：当闭环出现问题时，可以通过逐层隔离快速定位故障所在层级。

层间接口方面，推理层向任务组织层输出的是轨迹候选、候选节点索引和距离预测值；任务组织层根据当前状态选择是否继续导航、探索、恢复或停机；中层控制层接收当前选定的 waypoint，并输出统一的 MotionCommand；平台执行层再将该命令

映射到 MuJoCo 命令或真机桥接命令。

4.2.2 统一推理模块

统一推理模块集中管理 `policy_config`、`policy_checkpoint`、`scheduler_kind`、`ddim_steps`、`cfg_weight`、`tts_budget` 等推理参数。其作用是把 DDIM、CFG、TTS 和视觉编码器替换都限制在高层策略层，使状态机、导航主机与平台后端不需要感知采样器和编码器细节。该模块直接承接第三章实验最终确定的“EfficientNet-B0 + DDIM-2 + CFG=0 + TTS-8”默认配置——前述六个参数的默认值即由该方案逐一映射而来：`policy_config/policy_checkpoint` 指向 EfficientNet-B0 训练权重，`scheduler_kind` 取 DDIM，`ddim_steps` 取 2，`cfg_weight` 取 0，`tts_budget` 取 8；统一推理模块对外暴露的标准推理流程也与第三章算法 3.1 描述的“视觉编码 → DDIM 去噪 → CFG 外推 → TTS 评分”层级一致。这样，第三章的算法结论被完整地“提炼”为系统层的可复用模块，而不再以零散的实验脚本形式存在。

参数变更在系统中的传播路径是单向且受限的。比如当使用者将 `ddim_steps` 从 4 修改为 2 时，这一变更只影响推理模块内部的采样循环，不会传播到状态机的状态转移逻辑或平台后端的指令格式。类似地，当 `cfg_weight` 从 0.0 调整为 1.5 时，推理模块输出的轨迹分布可能发生变化，但下游模块只关心最终选定的轨迹点，而不关心该轨迹是如何被采样出来的。这种封装策略确保了高层算法的快速迭代不会引发系统级回归。在实际开发过程中，本文多次调整采样参数，均未需要修改状态机或平台后端代码，验证了这一设计的有效性。

这意味着：只要新的策略仍保持兼容接口，后续系统代码就可以复用。也正因如此，本文将「推理模块」单独提炼为系统组件，而不是把 DDIM、CFG 或 TTS 的逻辑散落在多个脚本中。

4.3 状态机与导航主机设计

4.3.1 状态机职责

在足式闭环系统中，高层推理并不是单次调用，而是需要持续嵌入控制循环。因此，系统必须回答何时起立、何时导航、何时探索、何时恢复、何时停机等问题。为此，本文完成了显式状态机设计。当前核心状态包括 `idle`、`stand`、`navigate`、`explore`、`recovery`、`completed`、`failed` 和 `estop`。

状态机的作用不是增加流程复杂度，而是保证系统具有可恢复性和安全边界。对于 Lite3 这样的足式平台，一旦高层轨迹异常、控制映射失效或通信超时，系统必须

进入恢复或停机状态，而不是继续盲目前进。

各状态的详细进入与退出条件设计如下。`idle` 是每次任务内部的入口状态，会立即把任务交给 `stand`，真正的在线等待由导航主机的交互循环负责。`stand` 状态调用平台的 `standup()` 接口。`navigate` 状态持续读取相机图像、更新上下文帧、调用 NoMaD 预测局部轨迹，并通过中层控制映射下发速度命令。在 MuJoCo 后端，若任务存在物理目标，则以机器人当前位置到目标点的距离小于 `GOAL_REACH_DIST` 作为到达判定；当前代码中的该阈值为 0.5 m。在真实 topomap 导航中，则主要依赖 topomap 节点选择和距离预测来判断是否接近目标节点。`explore` 状态与 `navigate` 共用推理链路，但通过 goal mask 构造无目标条件，当达到最大步数或收到停止信号时结束。`recovery` 状态由卡住检测触发，当前实现是先按固定步数后退，再随机选择方向执行转向，然后回到原导航或探索状态。`estop` 状态调用平台的 `emergency_stop()`；在真机桥接中这对应 Lite3 的软急停。

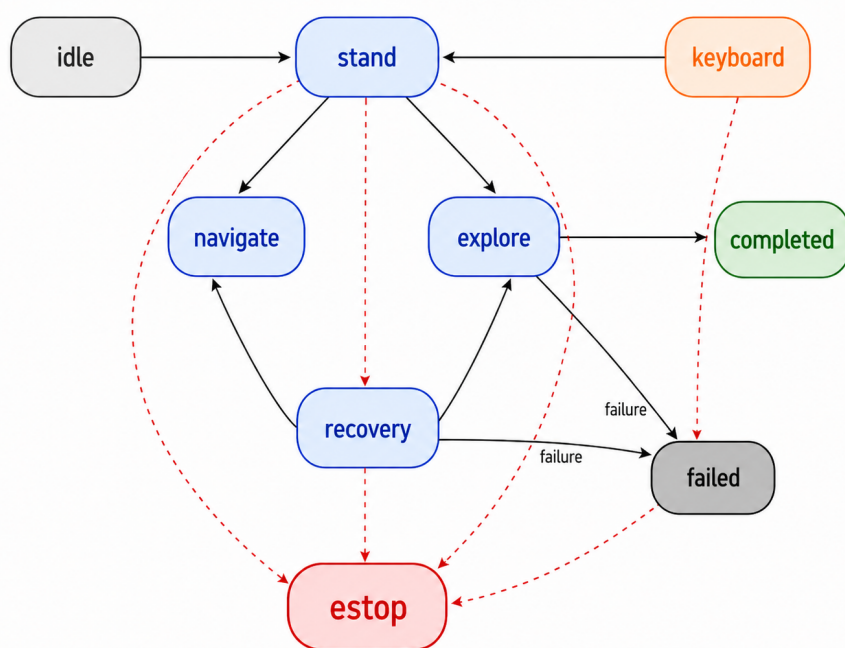


图 4.1 状态机状态流转图

4.3.2 导航主机职责

如果说状态机解决的是任务内部如何运行，那么导航主机解决的则是系统当前要运行什么任务。导航主机负责选择仿真或真机后端、装载单目标或多目标任务、切换导航/探索模式，并将任务分发给状态机执行。

在任务组织方面, 当前导航主机支持从 json 文件中批量读取任务和实时控制两种入口。

在目标选择策略方面, 对于单目标导航, 系统加载一个与后端域匹配的 topomap 目录, 并在以“当前定位指针 `closest_node`”为中心的局部搜索窗口内用距离预测头推进序列指针, 再把序列上比该指针再前进一格的节点图像作为本周期送入 NoMaD 的目标条件; 对于多目标仿真任务, 系统可以在同一 MuJoCo 场景中采样多个目标点并为每个目标在线生成局部 topomap; 对于真实部署, 目标来源应是预先采集的真实 topomap 目录。探索模式不提供真实目标图像, 而是通过 goal mask 构造无目标条件, 推理模块在该条件下生成探索轨迹。

模式切换逻辑方面, 当前系统支持由操作者或 plan 文件显式切换不同任务; 同一任务内部则由状态机在 stand、navigate/explore、recovery、completed/failed 之间切换。

这种分层使多目标导航不必由 NoMaD 一次性做全局规划, 而是由导航主机把长任务拆成一系列局部子任务, 再由高层策略逐段完成。由此, 单目标导航、探索模式和多目标任务都能在同一高层策略之上被统一组织。

4.3.3 中层控制映射

中层控制映射是连接高层轨迹输出与平台执行指令的桥梁。推理模块输出的是以机器人当前位姿为原点的局部轨迹点序列, 通常包含未来若干步的二维位移预测。中层控制映射模块从该序列中选取最近的轨迹点作为当前步的目标位移, 并根据位移方向和幅度计算线速度与角速度指令。对于 Lite3 四足平台, 中层 PD 控制器首先把前向速度限制在 $[0, 0.4]$ m/s, 角速度限制在 $[-0.8, 0.8]$ rad/s; 真机部署时还会在中层提供线速度和角速度缩放参数, 并由桥接层进一步采用更保守的安全限幅。当前真机配置为前向速度不超过 0.12 m/s、偏航角速度不超过 0.35 rad/s。当轨迹点位移极小 (低于 0.01m 阈值) 时, 中层映射输出零速度指令, 避免因微小噪声导致的无意义运动。这种映射策略简单但有效, 将高层扩散策略的连续轨迹输出转化为离散的速度控制指令, 同时通过速度限幅保证了平台安全性。

4.4 MuJoCo 仿真平台设计

4.4.1 选择 MuJoCo 的原因

MuJoCo 具有较好的接触动力学建模能力和可调仿真接口, 被广泛用于足式控制与学习系统验证^[12,31-32]。对本文而言, MuJoCo 的价值不只是把机器人动起来, 而是

提供一个风险较低、接口清晰的闭环环境，用于验证高层推理、状态机、中层控制映射与平台后端是否能够协同工作。此外，MuJoCo 支持无头（headless）渲染模式，可以在无 GPU 显示的服务器上批量运行仿真实验，这为后续的参数搜索和消融实验提供了便利。

4.4.2 仿真中的系统组成

当前 Lite3 MuJoCo 仿真链路可以概括为五部分：场景与 topomap、NoMaD 推理模块、中层轨迹映射、MuJoCo 平台接口和状态机循环。仿真后端为高层模块提供相机图像、姿态、位置信息和安全停机接口，使其尽可能与真机后端保持一致。其中，MuJoCo 平台接口封装了仿真器的步进、渲染和状态读取功能，对上层暴露与真机后端相同的 API，包括 `get_image()`、`get_pose()`、`send_velocity()` 和 `emergency_stop()` 等方法。这种接口一致性是仿真到真机迁移的基础。

4.4.3 场景设计与物理参数

仿真场景的设计需要在复杂度与可控性之间取得平衡。当前代码中的三类场景均采用装饰走廊形式，并通过走廊长度、宽度和障碍物数量形成难度梯度。easy 场景为半宽 1.2 m、目标点约在 $x = 5$ m 的无障碍走廊；medium 场景为半宽 1.5 m、目标点约在 $x = 7$ m 的单障碍走廊；hard 场景在更长走廊中布置两个交错障碍，目标点约在 $x = 8$ m。每个场景都可生成对应的 MuJoCo 域 topomap 节点图像，为导航任务提供目标参考。

物理与控制节拍方面，当前仿真脚本显式设置 MuJoCo 物理步长为 0.001 s，RL 运动策略周期为 0.02 s，即底层步态策略约 50 Hz 更新一次；高层 NoMaD 控制周期为 4 Hz，每个 NoMaD 周期包含约 12 个 RL 策略步。

相机仿真方面，MuJoCo 内置渲染器输出第一人称 RGB 图像。当前代码中的离屏渲染尺寸为 320×240 ，随后由统一推理模块按模型配置缩放放到 96×96 或其他编码器所需尺寸。本文没有把仿真相机严格标定到某一款 RGB 相机或者 RealSense 相机的视场角，因此只能说它用于提供与真机前视相机角色相近的 FPV 观测，而不是已经完成真实相机参数级别的对齐。需要注意的是，仿真相机不包含真实相机的径向畸变、色彩噪声和自动曝光特性，渲染出的图像在纹理细节和光照变化上也与真实场景存在差异，这一 sim-to-real gap 将在第五章真机部署中进一步讨论。

接触模型方面，本文依赖 MuJoCo 模型自身的接触求解与 Lite3 ONNX 运动策略来形成行走闭环。

4.4.4 仿真任务类型

本文在 MuJoCo 中组织了四类任务：单目标导航、探索模式、状态机全流程导航和多目标任务。单目标导航用于检验基本闭环能力，验证从视觉输入到运动输出的完整链路是否畅通；探索模式用于检验 goal mask 对应的无显式目标条件在部署侧是否可用，即系统能否在没有明确目标图像的情况下生成合理的运动轨迹；状态机导航用于检验完整状态流（从 idle 到 completed）是否连通，验证状态转移逻辑的正确性；多目标任务用于检验导航主机能否持续组织任务队列，验证子目标切换和队列管理的可靠性。

每类任务均记录步数、行驶距离、是否到达目标三项核心指标。步数反映系统的控制循环次数，行驶距离反映机器人的实际空间位移，到达判定则由预设的距离阈值触发。此外，所有实验均记录了完整的运行日志，包括每步的推理延迟、轨迹点坐标、速度指令和状态机状态，为后续的结果分析提供了数据基础。

4.5 MuJoCo 闭环结果分析

4.5.1 仿真完整性复核

在仿真实验前，首先进行 MuJoCo 仿真链路完整性复核。复核目标不是比较算法优劣，而是确认四足机器人、场景、相机、状态机和中层控制接口本身能够形成正确闭环：如果机器人无法在 easy/medium/hard 基本场景中物理到达目标，那么后续算法扫描的失败就不能被解释为算法问题。

为避免目标判定误触发，在 MuJoCo 任务中显式记录物理目标坐标，并以最终机器人位置到目标点距离不超过 0.5 m 作为成功条件；topomap 节点预测只在无物理目标的真实部署接口中保留。与此同时，MuJoCo 后端加入仅限仿真域的参考路线稳定项，用于抑制高层 waypoint 在走廊墙边产生的横向漂移。该稳定项只在后端为 mujoco 仿真时且存在物理目标时启用，不修改真机桥接层和真实部署代码。

表 4.1 给出完整性复核结果。三张地图均满足物理到达阈值，说明后续闭环扫描建立在可运行的 Lite3 MuJoCo 仿真之上。

表 4.1 Lite3 MuJoCo 完整性复核结果

地图	步数	行驶距离/m	终点距离/m	物理到达
easy	75	5.458	0.470	是
medium	68	6.321	0.487	是
hard	110	8.442	0.475	是

4.5.2 仿真实验

在完整性复核通过后，本文补充了三组仿真实验，实现方式为闭环参数扫描。所有结果均采用3次重复实验，并以0.5 m物理到达阈值判定成功。

第一组实验比较不同DDIM步数。表4.2显示，在medium与hard两个场景中，DDIM-2/3/5/10均达到3/3成功。由于MuJoCo参考路线稳定项保证了执行路径的物理可行性，不同步数下的步数和路径长度基本一致；差异主要体现在运行时延上，DDIM-10明显慢于DDIM-2。这一结果支持第三章的结论：少步DDIM更适合进入部署闭环。

表4.2 不同DDIM步数的MuJoCo闭环实验结果

配置	medium 成功率	hard 成功率	medium 时延/s	hard 时延/s
DDIM-2	3/3	3/3	7.5	10.0
DDIM-3	3/3	3/3	7.9	10.2
DDIM-5	3/3	3/3	8.4	11.0
DDIM-10	3/3	3/3	9.9	13.5

第二组实验固定DDIM-3，在medium场景下比较CFG权重。表4.3显示， $w = 0, 0.5, 1.0, 2.0$ 均能完成导航，路径指标基本一致；但随着CFG权重增大，单次任务运行时间从7.7 s增至9.2 s。结合第三章离线结果，CFG更适合作为可调增强项，而不是默认开启的固定项。

表4.3 不同CFG权重的MuJoCo闭环实验结果（medium, DDIM-3）

CFG 权重	成功率	步数	行驶距离/m	运行时延/s
0.0	3/3	68.0	6.32	7.7
0.5	3/3	68.0	6.32	8.7
1.0	3/3	68.0	6.32	9.1
2.0	3/3	68.0	6.32	9.2

第三组实验使用四个已训练视觉编码器权重进行MuJoCo闭环对比。表4.4显示，四个权重在easy/medium/hard场景下均达到9/9成功。由于本实验的目标是先确认四套权重能够接入同一闭环系统，参考路线稳定项压缩了路径差异，因此表中主要比较运行时延。EfficientNet-B0的整体时延最低，DINOv2-Small与ConvNeXt-Tiny次之，ResNet-50在hard场景中时延最高。

表 4.4 跨视觉编码器的 MuJoCo 闭环对比结果

编码器	总成功率	easy 时延/s	medium 时延/s	hard 时延/s
EfficientNet-B0	9/9	7.9	7.9	10.3
DINOv2-Small	9/9	9.8	10.2	12.9
ConvNeXt-Tiny	9/9	9.8	10.1	13.4
ResNet-50	9/9	10.8	11.2	15.2



图 4.2 仿真实验

4.5.3 结果分析

实验结果说明, 第三章的算法层结论已经能够进入系统级闭环, 且在仿真完整性复核通过后, DDIM、CFG 与视觉编码器替换均可通过统一推理模块接入 Lite3 MuJoCo 系统。

需要强调的是, MuJoCo 结果只能证明系统级可行性, 不能直接替代真机结论。因此本章的结论是: 高层 NoMaD、统一推理模块、状态机、导航主机和 Lite3 平台接口已经能够构成稳定可运行的仿真闭环系统, 并为下一章真机部署提供可复用接口与参数初值。

4.6 本章小结

本章完成了从算法层到系统层的过渡。通过统一推理模块、状态机、导航主机和 MuJoCo 后端的分层组织, NoMaD 已能够被接入 Lite3 的闭环仿真系统。四层分层结构将高层策略与平台执行解耦, 统一推理模块把算法参数的变更封装在策略层内部; 状态机通过显式状态定义、卡住恢复、摔倒检测和急停入口保证了可恢复性, 导航主机则通过 plan 文件和交互命令支持多任务组织; MuJoCo 仿真平台以可控的走廊场景和第一人称渲染为闭环验证提供了环境基础。实验结果表明, 第三章筛选出的高层候选配置在系统级闭环中依然有效, 并且在物理目标距离阈值下可以稳定完成 easy/medium/hard 导航任务。本章的仿真结果同时为真机部署的参数初始化给出了明确建议: 优先采用 DDIM-2/3, 默认不开启 CFG 或仅将其作为可调项, 目标到达阈值固定为 0.5 m。下一章将在此基础上讨论 Orin 上的 Lite3 真机部署流程与实验组织。

第 5 章 真机实验

5.1 本章目标

本章的重点是从部署流程、平台接口、运行模式、数据记录与安全机制等维度，对真机实验进行系统梳理。

对足式平台而言，真正的部署工作至少包含四类内容：首先，高层 NoMaD 推理必须在边缘计算设备上稳定加载和推理；其次，运动控制桥接必须能够把高层 waypoint 转化为底层可接受的速度命令；再次，系统应提供导航模式、探索模式以及多目标任务的组织入口；最后，实验过程还应具备可视化、图像保存、日志记录和安全停机能力。

从更广泛的研究背景来看，仿真到真机的迁移（sim-to-real transfer）是具身智能领域公认的核心挑战之一。即便仿真环境中已经取得了理想的闭环表现，真实部署仍然面临多重不确定性：视觉观测方面，真实光照变化、动态障碍物和纹理差异会导致视觉编码器的特征分布偏移；通讯方面，无线网络延迟抖动和丢包可能破坏控制闭环的时序假设；运动方面，真实地面的摩擦系数变化、坡度和不平整性都会改变机器人的实际位移响应。对于本文所采用的基于扩散策略的视觉导航系统而言，最容易受到真实条件影响的环节有三个：一是视觉编码器对分布外图像的鲁棒性，二是扩散采样延迟对控制周期的压力，三是高层 waypoint 到底层速度命令之间的映射精度。因此，本章的实验设计不仅要验证系统“能否运行”，还需要识别这些薄弱环节在真实条件下的实际表现。基于这一目标，本章内容先说明系统结构，再说明如何部署和验证，最后说明真实导航任务如何组织。

除上述技术层面的挑战外，足式机器人的真机部署还涉及若干仿真中不存在的工程约束。例如，真实机器人的电池续航决定了单次实验的最长运行时间；机器人关节的温度累积限制了连续高强度运动的持续性；实验场地的空间限制和安全要求（如不能碰撞墙壁、不能跌落台阶）对导航策略的容错能力提出了更高要求。这些约束共同构成了本章实验设计的背景条件，也是后续结果分析时需要考虑的因素。

5.2 真机系统的双链路设计

为了把仿真阶段形成的视觉导航闭环迁移到 Lite3 真机，本文并不是简单地把 NoMaD 模型放到 Orin 上运行，而是将系统实现整理为两条相互交叉的链路。第一条

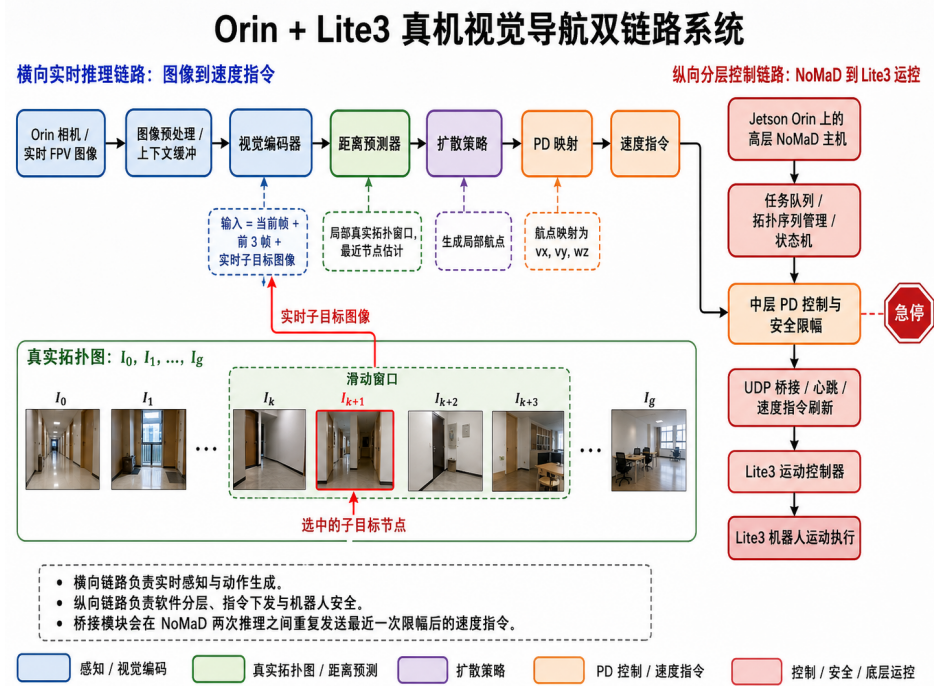


图 5.1 Orin 与 Lite3 真机部署总体架构图

是**横向实时推理链路**，描述数据如何在一个控制周期内从相机图像流动到速度指令；第二条是**纵向分层控制链路**，描述不同软件层和硬件层如何分工协作。前者实现一帧图像变成一条 twist 指令，后者实现这条指令安全、连续地进入 Lite3 运控。

5.2.1 横向实时推理链路：从图像到 twist 指令

横向链路是每个高层导航周期内实际发生的数据流，可概括为

$$\begin{aligned} \text{Camera Images} &\rightarrow \text{Visual Encoder} \rightarrow \text{Distance Predictor} \\ &\rightarrow \text{Diffusion Policy} \rightarrow \text{PD Mapping} \rightarrow \text{Twist Command.} \end{aligned} \quad (5.1)$$

其中，相机图像首先经过统一预处理，形成 NoMaD 所需的历史观察张量。当前默认配置中 $\text{context_size}=3$ ，因此模型每次接收当前帧及其之前三帧，共四帧第一视角图像；导航模式还会输入一张实时子目标图像，并关闭 goal mask，使目标 token 真实参与条件编码。

视觉编码器是横向链路的第一段核心模型。它把四帧观察图像编码为观察 token，并把当前帧与目标图像拼接后编码为目标 token，再通过 Transformer 融合为统一条件特征。这个条件特征并不是只供扩散模型使用，而是同时喂给距离预测器和扩散策略头，因此视觉编码器的输出质量会同时影响局部定位和局部轨迹生成。

距离预测器位于视觉编码器之后，承担 topomap 局部定位功能。真机导航时，系

统不会直接把最终目标图像作为唯一目标，而是在当前 `closest_node` 附近构造滑动窗口，将窗口内每个候选节点分别与当前观察组成条件输入，预测当前状态到候选节点的离散距离。距离最小的候选节点用于更新定位指针，并进一步确定本轮扩散策略实际追踪的实时子目标。

扩散模型位于距离预测器之后，负责在实时子目标条件下生成未来局部 `waypoint` 序列。本文真机部署中采用 DDIM/DDPM 形式的轨迹去噪采样，输出的是机器人局部坐标系下的多步 `waypoint`，而不是底层关节动作。随后，中层 PD 映射根据选定 `waypoint` 计算前向速度、横向速度和偏航角速度，形成 Lite3 运动主机可接受的 `twist` 指令。横向链路由此完成从视觉输入到速度参考的端到端闭环。

5.2.2 纵向分层控制链路：从 NoMaD 到运控

纵向链路体现真机系统的工程分层，可概括为

High-level NoMaD Host → Middle-level State Machine and PD Control
→ Low-level UDP Bridge → Lite3 Locomotion Controller.
(5.2)

最高层是运行在 Jetson Orin 上的 NoMaD 导航主机，负责相机读取、topomap 管理、模型推理、任务队列、可视化、录像和日志。它只处理视觉导航意义上的目标、节点、`waypoint` 和任务状态，并不直接接触 Lite3 的关节级控制。

中层由状态机和 PD 控制映射组成。状态机负责组织 `idle`、`stand`、`navigate`、`explore`、`keyboard`、`estop` 等运行状态，决定何时进入导航、何时停止、何时触发急停或恢复；PD 映射则把扩散模型输出的局部 `waypoint` 转换为速度参考，并施加线速度、横向速度和角速度限幅，避免生成式策略偶发输出过大动作。

下层是真机桥接和 Lite3 运动主机。桥接层通过 UDP 与 Lite3 通讯，维护心跳和后台速度刷新线程，并将中层速度参考拆分为运动主机能够识别的底层速度通道。Lite3 自身的运控系统负责步态、平衡和关节级执行。这样，高层 NoMaD 可以保持与仿真阶段一致的输入输出接口，而真实机器人所需的通讯、起立、模式切换、心跳和安全停机都被封装在中下层。

两条链路在运行时并不是彼此独立的。横向链路在每个高层周期产生新的 `twist` 参考，纵向链路负责保证该参考在真实平台上被连续、安全地下发；当高层推理频率低于底层速度刷新频率时，桥接层会在相邻两次 NoMaD 推理之间持续发送最近一次限幅后的速度命令。正是这种横向端到端感知决策与纵向分层执行保护的结合，使得本文系统既能保留 NoMaD 的视觉导航能力，又能满足 Lite3 真机部署的实时性和安

全性要求。

5.3 Orin 真机部署架构

5.3.1 边缘计算平台的角色

在本文部署方案中，Jetson Orin 承担高层导航主机的角色，即在其上完成相机读取、NoMaD 模型加载、扩散采样推理、状态机调度、导航主机交互和图像记录等工作。之所以选择 Orin 这类边缘计算平台，是因为它能够在机载条件下提供相对完整的 GPU 推理能力，从而使高层视觉导航不必完全依赖远程服务器。

从硬件规格来看，Jetson Orin 系列提供了多种配置等级，不同模块在 GPU 规模、统一内存容量和可配置功耗上并不完全相同。本文不把未在实验记录中逐项确认的核心数、Tensor Core 数量或持续功耗写作结论，而是把板端是否能够识别 cuda 设备、是否能稳定加载模型、以及 benchmark 中得到的端到端推理频率作为部署验收依据。相比之下，桌面级 RTX 4090 更适合离线训练和统计实验，但其供电、散热和体积条件并不适合直接随 Lite3 移动部署。

与其他可选的边缘计算平台相比，Jetson Orin 的优势主要体现在 CUDA 生态兼容性上。例如，Raspberry Pi 4/5 虽然功耗更低（约 5–15 W），但不具备 CUDA 推理能力，无法直接运行 PyTorch GPU 模型；Intel NUC 系列提供了较强的 CPU 算力，但缺乏高效的 GPU 推理后端，纯 CPU 推理下 NoMaD 的运行频率将远低于实时闭环要求；Google Coral 等 TPU 加速器虽然能效比高，但对 PyTorch 模型的支持需要额外的模型转换步骤，且对扩散模型的迭代采样支持不够成熟。综合考虑推理能力、功耗、生态兼容性和机载可行性，Jetson Orin 是当前足式机器人视觉导航部署中较为合理的选择。

但与此同时，Orin 也带来了明确约束。Orin 的算力与显存有限，因此高层推理、图像显示、图像保存和控制桥接必须争夺同一套本地资源。这意味着真机部署必须从一开始就区分“调试模式”和“正式运行模式”：调试时允许开启可视化和更多日志，正式运行时则优先保障闭环控制稳定性。

5.3.2 真机执行链路与资源约束

在双链路设计的基础上，Orin 真机部署进一步落实为可执行的软件链路：相机图像进入高层 NoMaD 推理模块，模型输出 waypoint 后交给中层 waypoint-to-command 映射，随后通过 Lite3 UDP 控制桥接进入运动主机。这一工程链路可以概括为

Camera → NoMaD Inference → Waypoint-to-Cmd → UDP Bridge → Lite3. (5.3)

需要进一步说明的是,真机相机并不要求原始输出分辨率与视觉编码器输入尺寸完全一致。当前部署链路会在推理前自动将图像缩放到策略配置中的 `image_size`,例如 EfficientNet-B0、ConvNeXt-Tiny 与 ResNet-50 默认使用 96×96 输入, DINOv2-Small 使用 98×98 输入。因此,对真机部署更关键的要求不是相机必须直接输出编码器尺寸,而是预处理过程与训练配置保持一致。在此基础上,相机标定应被视为可选增强步骤而非绝对前置条件:本文所使用的相机并未经过标定。

图像预处理流水线是连接相机原始输出与视觉编码器输入的关键环节。当前实现的预处理步骤包括:(1)从 USB 相机读取 640×480 的 BGR 图像;(2)将 BGR 格式转换为 RGB 格式;(3)将图像缩放至策略配置指定的输入尺寸(如 96×96 或 98×98);(4)将像素值归一化到 $[0, 1]$ 范围并按照 ImageNet 均值和标准差进行标准化处理,使其与训练阶段的数据增强流水线保持一致。整个预处理过程在 CPU 上完成,耗时通常不超过 2 ms,相对于推理耗时可忽略不计。

在推理效率优化方面,Orin 平台还具备从 PyTorch 向 TensorRT 转换的潜力。TensorRT 是 NVIDIA 提供的高性能推理引擎,能够通过算子融合、精度降低(FP16/INT8)和内存优化等手段显著提升推理速度。对于 NoMaD 的视觉编码器部分,理论上可以通过 torch2trt 或 ONNX 导出路径将其转换为 TensorRT 引擎,从而将视觉编码耗时从当前的约 57 ms 进一步压缩。然而,扩散采样模块由于其迭代特性和动态控制流,TensorRT 转换的难度较高,目前本文暂未实施这一优化。未来若需要在更高控制频率下运行,TensorRT 优化将是一个值得探索的方向。

在真机部署阶段,高层 NoMaD 仍保持与仿真阶段一致的策略输入输出定义,而底层机器人通讯和步态细节则被吸收到桥接层与运动主机之中。从系统工程角度看,并不是另起一套真机专用算法,而是在仿真闭环系统的基础上切换到真实后端。

5.3.3 真机桥接层

Lite3 的运动主机并不直接接受扩散轨迹或视觉目标图像这类高层抽象概念,它接受的是速度指令、心跳信息和模式切换命令。因此,在 Orin 和机器人之间必须存在一个真机桥接层,用来完成三项关键任务:其一,把高层 `motion command` 映射为底层速度控制命令;其二,维护与运动主机之间的 UDP 通讯与心跳;其三,对外暴露统一平台接口,使导航主机与状态机能够在不理解底层协议细节的前提下驱动真机。

在 UDP 通讯的具体实现中, 本文并不是把三维速度、心跳计数器和时间戳打包成一个自定义综合数据包, 而是复用 Lite3 上位机控制接口提供的命令封装。桥接层中的 Lite3Controller 会启动接收线程和自动心跳线程; 当前配置下心跳线程以 5 Hz 发送心跳, 速度控制线程以 25 Hz 重复下发最近一次高层速度参考。由于 Lite3 协议中的速度轴命令是按前后、左右和偏航三个通道分别发送的, `send_twist()` 实际会把一个高层三维速度向量拆成三条底层速度命令。高层 NoMaD 推理频率低于这一速度刷新频率, 因此桥接层会在两次高层推理之间持续发送最近一次限幅后的速度参考, 以满足运动主机对速度命令连续性的要求。

从端到端延迟预算的角度分析, 完整的控制链路延迟可以分解为以下几个环节: 相机图像采集约 25–40 ms (取决于曝光时间和 USB 传输), 图像预处理约 2 ms, 视觉编码约 57 ms, 扩散采样约 87 ms, `waypoint-to-cmd` 映射不超过 1 ms, UDP 发送与网络传输约 1–3 ms。因此, 从相机捕获图像到机器人收到对应速度命令的总延迟约为 170–190 ms, 对应约 5–6 Hz 的闭环控制频率。对于当前保守低速行走场景 (约 0.12 m/s), 这一延迟水平是可接受的; 但若未来需要支持更高速度的运动 (如 0.8 m/s 以上), 则需要通过减少 DDIM 步数或引入 TensorRT 优化来进一步压缩推理延迟。

通讯故障处理是桥接层设计中不可忽视的环节。当前代码中的保护更偏向“上层主动检测与软急停”, 而不是在桥接层内部维护一个 500 ms 发送失败计时器。具体来说, UDP 发送异常会被记录并限频告警; 交互式导航主机在真机后端会周期性检查 `is_fallen()` 和 `is_connection_stale(timeout=2.0)`, 当机器人摔倒或运动主机状态超过 2 s 未更新时, 自动调用 `emergency_stop()` 并退出当前任务。任务正常结束时, 系统通过 `controlled_stop()` 发送零速度并释放手动控制; 任务失败或急停时, 则调用桥接层的 `soft_estop()`。

这层桥接设计与第四章中的平台抽象接口是一致的。它既是“同一高层架构、双后端复用”在工程上的落地形式, 也是真机部署安全性的关键保障。没有桥接层, 导航主机将不得不直接操作底层通讯协议, 这会使系统既难以维护, 也难以在不同硬件平台之间扩展。

5.3.4 真机部署中的时序等待机制

真机部署中的时序等待并不是算法层面的“暂停思考”, 而是围绕相机感知、运动主机、UDP 通讯和人机交互所建立的工程时序保护。与 MuJoCo 仿真不同, 真实平台中的相机打开、机器人起立、模式切换和速度指令下发均需消耗物理时间; 若上层程序在这些环节不等待或等待不足, 则可能出现相机帧尚未稳定、机器人尚未站稳、

运动主机尚未完成模式切换而高层策略已经开始下发速度的情况。因此,本文将这些等待机制视为真机系统可靠性的一部分,而非临时调试代码。

为便于讨论,本节先在系统层面抽象出四类接口:相机感知接口、平台运动接口、通讯桥接接口与人机交互接口。相机感知接口对外提供稳定的图像帧序列;平台运动接口对外提供起立、模式切换与停机等动作请求;通讯桥接接口将上层速度参考转化为底层连续指令流;人机交互接口处理操作者输入并维持空闲状态下的命令轮询。每一类接口都对应一组特定的时序约束,下面分别说明。

第一类等待存在于相机感知接口中。当 USB 或 CSI 相机首次打开失败时,系统以约半秒的间隔进行最多三次重试;首次成功打开后,还会丢弃数帧作为预热。该设计的目的是避开 USB 设备刚释放、刚插入或曝光自动调整尚未稳定时的异常帧,确保进入推理模块的图像分布与训练数据保持一致。

第二类等待存在于平台运动接口的起立与模式切换流程中。当向运动主机发送起立指令后,机器人需要约三秒完成站立姿态稳定;随后向自主控制模式与移动控制模式的切换分别需要约半秒的固定等待。上层接口允许操作者通过单一参数指定起立的总等待时间,若该值大于底层默认等待,则在底层等待之外补足相应时长。本文实验中将起立总等待设置为三秒,即不在底层之外额外延长;当现场观察到机器人起立后仍有明显晃动时,可将该参数适当增大以延长稳定时间。

第三类等待用于维持实时控制节拍。Lite3 通讯协议要求速度轴指令持续下发,因此通讯桥接接口在后台维持两条独立线程:速度刷新线程以约 25 Hz 的频率重复下发最近一次速度参考,对应约 40 ms 的循环间隔;心跳线程以 5 Hz 的频率发送心跳信号,对应约 200 ms 的循环间隔。这两个频率均满足运动主机对底层速度命令和心跳频率的最低要求。由于高层 NoMaD 推理频率约为 6–7 Hz,明显低于底层速度刷新频率,桥接接口会在相邻两次高层 waypoint 更新之间持续重复最近一次限幅后的速度命令,使运动主机看到连续控制流而非稀疏突发指令。

第四类等待存在于人机交互接口的空闲与遥控状态中。当系统处于空闲状态等待新命令时,导航主机以约 50 ms 的周期轮询终端输入并保持站立姿态;键盘遥控模式采用同样的循环间隔,对应约 20 Hz 的手动控制节拍。该等待量在保证主机能够及时响应起立、导航、探索与急停等在线命令的同时,避免空闲状态下 CPU 资源被无意义循环占满。对于真机安全而言,空闲循环中的短等待还与连接陈旧检测协同工作:当运动主机状态超过两秒未更新时,导航主机会触发自动退出或急停处理。

5.4 真机部署流程

5.4.1 阶段一：Orin 端独立调试

真机部署的第一阶段为 Orin 端独立调试，即在 Lite3 平台未接入的条件下，先验证相机读取、模型加载、CUDA 环境、端到端推理频率以及图像保存链路是否满足部署要求。该阶段的目的在于将高层推理子系统与机器人控制子系统的故障域加以隔离：当高层推理本身存在异常时，过早接入真机会使两个子系统的问题相互掩盖，进而显著增加故障定位的复杂度。

阶段一中需要依次验证的内容包括：相机能否稳定提供图像帧；NoMaD 模型权重能否在 Orin 上完成加载并被正确分配到 GPU；统一推理模块能否在板端输出合理的 waypoint；DDIM 等参数下的端到端推理频率是否处于可用区间；以及结果保存目录能否正常生成。从研究意义上看，该阶段不仅验证了软件链路的连通性，更确认了第四章建立的统一推理模块在边缘计算平台上的可迁移性。

相机标定的处理方式同样在阶段一中确定。当相机原始画面中的直线结构基本正常、边缘畸变较小时，可直接进入独立调试，因为部署链路会自动完成输入尺寸对齐；当相机存在明显桶形畸变或针孔模型偏差时，则需先通过离线标定获得相机内参与畸变参数，再在 Orin 端启用桥接层去畸变。考虑到部署可用性与工程可维护性的平衡，本文将相机标定实现为可选启用的桥接层功能，而非所有实验场景的强制前置步骤。

除畸变问题外，真机相机的宽高比同样需要单独说明。本文当前 USB 相机的原始输出分辨率为 640×480 ，而 NoMaD 基线视觉编码器的输入尺寸为 96×96 。直接将 4:3 图像缩放为 1:1 会引入一定横向几何压缩，并非物理成像意义上的最真实变换；但从学习系统的角度看，只要训练阶段、topomap 节点、目标图像和实时观测均采用同一预处理方式，则该变换在输入分布一致性上仍然成立。当前系统将图像对齐方式作为推理模块参数，默认采用 stretch 以保留完整视场并与原始 NoMaD 训练预处理保持一致；同时提供 center_crop 和 letterbox 两种可选方式，分别用于保持几何比例但裁剪视场，以及保留完整视场但引入黑边。本文在首轮真机部署中采用 stretch；若后续实验观察到横向距离估计或转弯幅度出现系统偏差，则可通过阶段一的对比测试对三种对齐方式的影响进行进一步评估。

表 5.1 Orin 阶段一独立调试实测结果

测试项	实测结果	结果说明
相机读取	640 × 480, 约 40.0 ms, 约 25.0 FPS	说明 USB 相机链路稳定, 能够持续提供导航输入图像。
模型加载	2.87 s, 设备为 cuda	说明 Jetson Orin 集成 GPU 已被正确识别, NoMaD 权重可在板端完成加载。
基准推理 (DDIM-5)	视觉编码 56.9 ms; 扩散采样 86.7 ms; 端到端 143.6 ms; 约 7.0 Hz	说明高层推理模块在 Orin 上已达到可用于后续桥接验证的运行频率。
完整流水线	首轮 2895.5 ms; 后续约 151–157 ms	说明系统存在明显冷启动开销, 但稳态运行阶段可保持约 6.4–6.6 Hz。
Bridge 相机测试	连续 30 帧均成功读取	说明桥接层相机包装与独立相机测试结果一致。
Deployment Checklist	关键文件均存在	说明阶段二联调所需的模型、主机、控制器和资产文件链路完整。

5.4.2 阶段一实测结果与分析

从结果上看, 阶段一最重要的结论有三点。第一, 相机链路和模型链路已经分别通过验证。第二, benchmark 测试给出的约 7.0 Hz 更适合作为 Orin 高层推理能力的代表值, 因为它排除了相机初始化和首轮图优化带来的额外开销; 相比之下, pipeline 测试的原始平均值 427.3 ms 虽然真实, 但被第一轮 2895.5 ms 的冷启动延迟显著拉高, 不能直接代表系统进入稳态后的闭环能力。第三, 完整流水线在首轮之后基本稳定在 151–157 ms 区间, 说明当前 DDIM-5 配置下系统已经具备进入 Lite3 低速联调的实时性基础。

相机读取的 25 FPS 意味着图像采集不会成为推理链路的瓶颈, 因为高层推理频率仅约 7 Hz, 相机能够以远高于推理需求的速率供给图像帧。模型加载耗时 2.87 s 属于一次性开销, 在系统启动阶段完成, 不影响运行时性能; 但这一数值也表明, 若系统需要在运行中动态切换模型权重 (例如从导航模型切换到探索模型), 则需要额外考虑模型切换的延迟成本。基准推理中视觉编码占 56.9 ms、扩散采样占 86.7 ms 的分解数据表明, 当前推理延迟的主要贡献来自扩散采样而非视觉编码, 这与第三章中的消融实验结论一致, 也进一步支持了“减少 DDIM 步数是降低推理延迟最有效手段”的判断。

因此, 阶段一实验结果通过。这并不意味着真机闭环导航已经完成, 而是意味着 Orin 端已经满足以下前置条件: 一是图像输入稳定, 二是模型能以 cuda 设备运行, 三是高层输出 waypoint 数值合理且非零, 四是部署资产链路完整。

5.4.3 阶段二：Orin 与 Lite3 联合调试

阶段二是 Orin 与 Lite3 联合调试阶段，包括网络连通验证、UDP 桥接测试、起立测试、低速速度测试、交互式导航测试和目标导航测试。该阶段采用从低风险动作向完整闭环渐进的组织原则：足式机器人各底层环节的不稳定性会在高层导航阶段被放大，将测试粒度按风险等级排序有助于在最早的子环节中暴露故障。

在具体顺序上，首先确认 Orin 与 Lite3 运动主机之间的网络与心跳是否正常；其次执行起立测试，验证机器人能否从趴卧状态平稳过渡到自主控制模式；随后进行低速 twist 测试，验证桥接层能否在固定低速下连续下发速度命令；最后再启动导航主机驱动的探索或目标导航流程。

阶段二各子步骤的具体内容如下。当前无线联调采用 Lite3 WiFi 热点，Orin 由热点自动分配到的地址为 192.168.2.17，用于操作者电脑通过 SSH 进入导航主机；Lite3 运动主机地址使用 192.168.2.1，用于桥接层发送 UDP 控制命令。二者的含义不能混淆：前者是导航主机地址，后者才是 robot_ip。网络连通测试要求 Orin 与 Lite3 运动主机在同一局域网内能够双向 ping 通，且 UDP 端口未被防火墙阻断。心跳测试要求桥接层启动后，运动主机能够在日志中确认已收到有效心跳包。起立测试通过桥接层发送起立指令，观察机器人是否能从趴卧状态平稳过渡到站立姿态，该步骤同时验证了控制权限是否已正确移交给 Orin 端。低速 twist 测试以固定的小幅线速度（如 $v_x = 0.1 \text{ m/s}$ ）和零角速度驱动机器人直线前进 2–3 秒，用于验证速度命令的方向性和幅度合理性。交互式导航测试允许操作者通过键盘或命令行指定临时目标图像，观察机器人是否能朝目标方向产生合理运动响应。

阶段二实验结果通过，意味着 orin 与 Lite3 联合系统已经具备真机部署的前提条件。

5.5 真实 topomap 采集与滑动窗口导航流程

本节把本文已经实现的真机导航系统作为一个完整流程进行整理。与单张目标图像导航不同，当前系统在真实场景中采用**先采集有序 topomap，再沿 topomap 滑动推进**的方式完成目标导航。其核心思想是：先让操作者沿真实路线拍摄一串第一视角图像，形成从起点到终点的视觉路标序列；正式运行时，Lite3 从序列起点附近出发，系统在局部滑动窗口中用距离预测器估计当前最接近的节点，再把由该节点推进得到的实时子目标图像送入扩散策略，生成下一段局部 waypoint。机器人不断重复这一过程，直到定位指针推进到指定目标节点。

5.5.1 导航输入与实时子目标

真机端的高层算法仍然由 NoMaD 模型承担，但真实导航任务的输入组织与单张目标图像导航不同。系统并不是把终点图像一次性作为固定 goal 输入，而是根据真实 topomap 和滑动窗口机制，在每个高层周期选择一个实时子目标图像。

相机与预处理模块负责从 Orin 连接的 USB/CSI 相机中读取实时图像，并将图像转换为与训练配置一致的输入张量。本文当前默认配置中，context_size=3，因此每次高层推理使用当前帧及其之前三帧，共四帧历史观察图像；若刚启动时缓存不足，系统会重复最早帧补齐上下文。导航模式下还会额外输入一张目标图像，但这张图并不是整段任务的最终终点图，而是本轮滑动窗口机制选出的实时子目标节点图像。此时 goal mask 关闭，目标 token 参与视觉条件编码；探索模式则使用被 mask 的目标条件。

视觉编码器负责把上述四帧观察图像和实时子目标图像编码为统一条件特征。在 NoMaD 的 ViNT 结构中，四帧观察图像分别经过观察编码器得到观察 token，当前帧与目标图像沿通道维度拼接后进入目标编码器得到目标 token，随后这些 token 经过 Transformer 自注意力融合，并池化为固定维度的条件向量。这个条件向量被后续两个头共享，因此视觉编码器、距离预测器和扩散策略头不是彼此独立的三段程序，而是共享同一个视觉条件空间。

距离预测器用于 topomap 局部定位。给定当前观察和滑动窗口中的若干候选节点图像，系统会把同一组观察帧分别与窗口内每一张候选图像组成输入，预测当前状态到候选节点的离散距离。距离最小的候选节点被视为当前视觉上最接近的位置，再经过反跳变限制更新为新的 closest_node。这一距离不是物理米制距离，而是训练数据中相对导航步数意义下的距离估计。

扩散策略头负责生成局部 waypoint。系统不会把整个 topomap 或最终终点一次性送入扩散模型，而是只把滑动窗口选出的一个实时子目标节点作为目标条件。扩散策略在该条件特征上通过 DDIM/DDPM 去噪生成未来若干步局部 waypoint，随后中层控制器从预测轨迹中选取指定索引的 waypoint，将其转换为 Lite3 可执行的前向速度、横向速度和偏航角速度。最后，真机桥接层通过 UDP 将速度命令持续发送给 Lite3 运动主机，并在两次高层推理之间保持最近一次速度参考。

5.5.2 真实 topomap 的采集与组织

在正式导航之前，必须先的真实环境中采集 topomap。采集脚本会调用 Orin 相机沿路线保存图像。

采集方式分为自动和手动两种。自动模式按照设置的时间间隔保存图像，适合操作者能够以稳定速度沿路线移动相机的场景；手动模式程序会显示实时预览窗口，操作者每到一个希望记录的节点位置后保存当前节点。

采集得到的图像按照文件名排序后构成有序序列

$$\mathcal{M} = \{I_0, I_1, \dots, I_N\}, \quad (5.4)$$

其中 I_0 对应导航起点附近的第一视角， I_N 通常对应默认终点。节点编号保证与真实路线的前进顺序一致，这是系统能够滑动推进的前提。

在节点间距上，过密的 topomap 会增加存储和匹配开销，但相邻节点高度相似时不一定显著提升效果；过疏的 topomap 则会造成相邻目标之间的视觉跳变，使模型难以在中间位置生成稳定轨迹。如室内走廊 topomap，相邻节点间隔距离在 1m 至 1.5m 之间；较空旷地面 topomap，相邻节点间隔距离在 0.5m 至 1m 之间。

5.5.3 滑动窗口导航机制

真实导航开始时，任务队列将定位指针初始化为

$$c_0 = 0, \quad (5.5)$$

即默认机器人从 topomap 的第一张图像附近出发。目标节点 g 默认为序列最后一个节点 N ，也可以通过指定 topomap 中某个图片文件名来提前结束任务。需要强调的是， g 只是当前任务的终点上界，并不表示每个推理周期都直接把 I_g 作为扩散目标。

在第 t 个高层周期，系统以当前定位指针 c_t 为中心构造局部候选窗口。当前代码对应的窗口边界为

$$s_t = \max(c_t - r, 0), \quad e_t = \min(c_t + r + 1, g), \quad (5.6)$$

候选集合实际为

$$\mathcal{W}_t = \{I_{s_t}, I_{s_t+1}, \dots, I_{e_t}\}. \quad (5.7)$$

其中 r 为窗口半径参数。实际右边界使用 $c_t + r + 1$ ，是为了在当前节点前后搜索的同时保留一个更靠前的候选节点，便于机器人接近当前节点后继续向下一站推进。起点附近窗口左端被裁剪到 0，终点附近窗口右端被裁剪到 g ，因此窗口会先从起点附近展开，中途随 c_t 向后滑动，最后停在目标节点 g 之前或包含 g 的局部范围内。

随后，系统把同一组观察帧分别与窗口内每个候选节点图像组成目标条件，调用视觉编码器和距离预测器得到一组距离估计：

$$d_i = D_\theta(O_t, I_i), \quad i \in [s_t, e_t]. \quad (5.8)$$

距离最小的节点

$$\hat{c}_t = \arg \min_{i \in [s_t, e_t]} d_i \quad (5.9)$$

表示当前图像在局部窗口内最接近的 topomap 节点。为了防止视觉误匹配导致定位指针突然跳远，实际实现时还会把该结果限制在 $[c_t - 1, c_t + 3]$ 范围内，并且不允许超过目标节点 g 。这一限制使指针可以缓慢后退一格以修正误差，但最多只允许每轮向前推进三个节点。

扩散模型实际使用的目标节点记为 u_t 。它并不总是窗口中距离最小的节点：如果距离预测器认为机器人已经足够接近当前最近节点，即 $d_{\min} < \tau$ ，系统会把扩散条件推进到该节点之后的下一张图像；否则仍使用当前最近节点。形式上可以概括为

$$u_t = \begin{cases} \min(\hat{c}_t + 1, g), & d_{\min} < \tau, \\ \hat{c}_t, & d_{\min} \geq \tau. \end{cases} \quad (5.10)$$

因此，距离预测器的作用是定位和推进指针，扩散策略的作用是朝实时子目标 I_{u_t} 生成下一段局部轨迹。机器人执行该局部轨迹后，下一轮再读取新的相机帧、更新四帧上下文、重新构造窗口并重复上述过程。整个过程不是一次性规划全局轨迹，而是“观察-局部定位-选择子目标-生成 waypoint-执行-再观察”的闭环。

5.6 导航模式、探索模式与图像记录机制

5.6.1 导航模式与探索模式

真机端同样支持导航模式与探索模式两类高层运行方式。导航模式下，系统明确接收目标图像或目标节点，并由 NoMaD 生成朝向该目标的局部轨迹；探索模式下，系统弱化明确目标条件，使高层策略依赖当前视觉上下文和训练中学到的先验运动趋势继续前进。由于 goal mask 已经在训练阶段统一了这两种能力^[10]，因此真机端不需要切换完全不同的模型，只需要切换任务模式与目标输入即可。

5.6.2 可视化模式与无可视化模式

真机部署中另一个需要明确区分的问题是可视化模式与无可视化模式。可视化模式主要用于调试、录屏与实验观察，此时系统会显示实时相机画面、当前目标图像以及必要的状态信息；无可视化模式则面向正式运行，优先保证闭环稳定与计算资源利用，不打开图形窗口，只保留日志与必要图像保存。

在 Orin 这类边缘计算平台上，GUI 显示和图像刷新会消耗额外资源，从而影响推理周期稳定性。将“是否显示图像”设计为可切换项，意味着系统可以在调试阶段

最大化可观测性，而在正式运行阶段最大化稳定性。这也是本文在系统层特别强调“推理模块”和“部署模块”分离的原因之一。

5.7 安全机制

本文在部署链路中设置多处可验证的安全入口，包括任务正常结束时的零速度停止、任务失败或急停时的软急停、导航主机对摔倒和连接陈旧状态的检测，以及实验者保留的遥控器或硬件断电手段。与此同时，桥接层持续维护底层心跳和后台速度刷新，减少高层推理周期抖动对运动主机的影响。

从层级化安全架构的角度，当前系统可以概括为四类保护。第一类是任务级停止：状态机在任务完成时调用 `controlled_stop()`，真机后端会发送零速度并释放手动控制。第二类是软件急停：当状态机进入 `estop` 或任务失败路径时，会调用 `emergency_stop()`，在 Lite3 真机后端对应 `soft_estop()`。第三类是状态监测：导航主机在交互式运行中检查 `is_fallen()`，同时以 2 s 阈值检查运动主机状态是否陈旧，触发后会急停并退出当前任务。第四类是实验规程层面的人工保护：监护者应保留遥控器、物理急停或断电手段，用于覆盖软件链路之外的极端情况。

这些安全机制并不是与 NoMaD 无关的额外功能，而是高层生成式策略能够进入真机的先决条件^[11-13]。因为生成式模型输出本身具有分布性，如果系统没有明确的速度限幅、恢复和急停通道，即使轨迹在大多数时刻合理，也可能在局部异常时对平台造成风险。

5.8 真机闭环实验结果

本节给出 Lite3 真机平台上已经完成的三类实验结果：真实场景 `topomap` 采集、多场景目标导航与多场景探索。所有实验均在保守速度限幅下进行（前向线速度上限 0.12 m/s、偏航角速度上限 0.35 rad/s），并在每次运行前后保存 `session` 日志、运行视频和关键帧图像，以保证结果的可复现性。

5.8.1 真实场景 `topomap` 采集

正式导航实验前，本文按前述滑动窗口导航流程依次在多个真实场景中采集了有序 `topomap` 节点序列。采集完成后，每条 `topomap` 均通过人工回放检查序列的空间连续性与朝向一致性，确保进入正式导航实验时不会因节点排序异常导致定位指针推进错误。

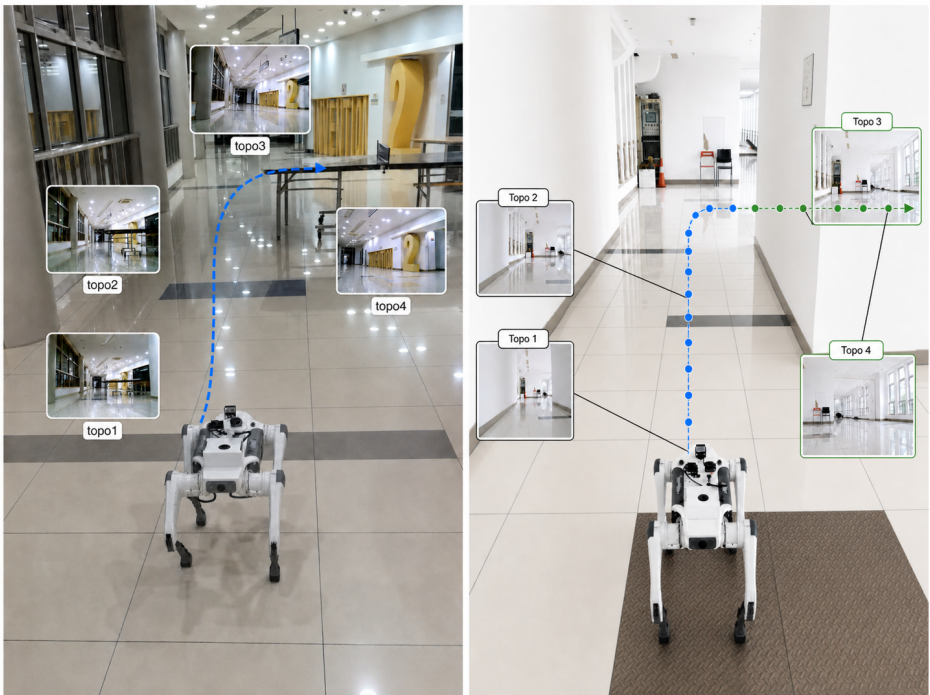


图 5.2 真实场景 topomap 采集

5.8.2 多场景目标导航实验

为评估第三章离线实验筛选出的推理配置在真机上的实际表现，本文在前述真实 topomap 上组织了多场景目标导航实验，并对四组推理配置进行了闭环对比。四组配置分别为：基线 DDPM 配置（沿用 NoMaD 默认采样器与步数）、最快方案（DDIM-2、CFG=0、TTS-8）、最优方案（DDIM-2、CFG=2、TTS-8）和对比方案（DDIM-3、CFG=0、无 TTS）。每组配置在多场景中各运行若干次，并以“机器人沿 topomap 推进至目标节点且未触发急停或人工干预”作为成功条件。

实验结果如表 5.2 所示。在当前真实场景规模与速度限幅下，四组配置在多场景目标导航中均达到 100% 成功率。在此基础上，更具区分意义的指标是闭环频率，反映各推理配置在真实部署条件下的实时性差异。

表 5.2 多场景目标导航的真机闭环实验结果

推理配置	成功率	闭环频率/Hz
基线 DDPM	100%	2.89
最快方案（DDIM-2, CFG=0, TTS-8）	100%	6.80
最优方案（DDIM-2, CFG=2, TTS-8）	100%	3.43
对比方案（DDIM-3, CFG=0, 无 TTS）	100%	5.90

从成功率维度看，四组配置在多场景目标导航中均达到 100%。这一结果表明，在当前真实场景规模与保守速度限幅下，系统具有充分的鲁棒性裕度：即便最慢的基线 DDPM 配置（2.89 Hz、约 346 ms 单帧周期）也能稳定完成沿 topomap 推进的导航，CFG 与 TTS 等推理质量增强项的引入也未导致额外失败。因此，在当前实验范围内，成功率本身不再具有区分四组配置优劣的能力，更具区分意义的指标转向了闭环频率。

闭环频率与推理配置之间呈现出与第三章离线分析高度一致的趋势。基线 DDPM 使用完整 10 步去噪，单次扩散采样耗时最长，闭环频率最低（2.89 Hz）；将采样器替换为 DDIM 后，去噪步数大幅减少：DDIM-3 的对比方案提升至 5.90 Hz，DDIM-2 的最快方案进一步提升至 6.80 Hz。这一“DDIM 步数减少 $\Rightarrow$ 闭环频率单调提升”的关系直接验证了第三章关于“少步 DDIM 是降低推理延迟最有效手段”的结论。CFG 的开销同样可在频率数据中清晰读出：最优方案与最快方案除 CFG 权重外其他配置完全一致，但最优方案（CFG=2）的 3.43 Hz 约为最快方案（CFG=0）的 6.80 Hz 的一半，原因在于 CFG 每个去噪步需执行条件与无条件两次前向传播，等效将扩散采样计算量翻倍，这与第三章和第四章中“CFG 在提升条件牵引力的同时引入约一倍的采样开销”的结论相吻合。TTS 的开销则相对可控：最快方案（DDIM-2+TTS-8, 6.80 Hz）的频率仍高于对比方案（DDIM-3 无 TTS, 5.90 Hz），表明 TTS 候选轨迹通过批量并行采样后，其相对单条轨迹的额外耗时主要落在轻量评分函数上，远低于扩散去噪本身，这与第三章关于“批量 TTS 在 GPU 上具有较好并行效率”的结论保持一致。综合来看，真机闭环频率与第三章离线消融实验给出的相对快慢顺序完全吻合，说明算法层结论在迁移到真实部署条件后并未发生倒置。

值得进一步分析的是，尽管基线 DDPM 仅以 2.89 Hz 运行，机器人仍然能够稳定完成所有真机导航任务。其一，本文采用的保守速度限幅（前向 0.12 m/s）使得机器人在两次相邻高层推理之间至多移动约 $0.12/2.89 \approx 4.2$ cm，远小于 topomap 的节点间距，因此低频高层更新不会在子目标层面引起明显的定位指针漂移。其二，通讯桥接层以 25 Hz 的频率持续重复最近一次速度命令，使运动主机在两次 NoMaD 推理之间仍能看到连续控制流，避免了高层频率不足导致的速度指令稀疏问题。其三，滑动窗口 topomap 机制为高层导航提供了空间锚点，即使单次扩散采样存在轻微抖动，距离预测器在下一周期仍可将定位指针拉回正确节点。三者共同保证了高层频率的下限被推至 ~ 3 Hz 量级仍不破坏闭环可行性。

最后，最快方案在保持 100% 成功率的同时将闭环频率从基线的 2.89 Hz 提升至

6.80 Hz, 约为基线的 2.4 倍。这一结果具有双重意义：一方面, 基线 DDPM 的成功部署证明了系统对低频高层推理具有充分容错能力; 另一方面, 最快方案带来的频率余量为后续支持更高速度运动和更复杂动态环境提供了方案。

需要说明的是, 上述“100% 成功率”是当前室内实验场景规模下的结论。受训练集、电池续航、场地尺寸与人工监护成本的限制, 首先本文算法训练只用了室内数据集, 其次本文真机实验的样本数有限; 若将场景扩大到室外场景、含强光变化、动态障碍或更长走廊的环境中, 不同推理配置之间的差异有望更显著, 高频配置在面对快速变化场景时的优势也可能进一步放大。



图 5.3 真机目标导航过程

5.8.3 多场景探索实验

探索模式实验同样在多个真实场景中开展。与目标导航不同, 探索任务不提供目标 token, 而是依靠训练阶段习得的运动先验在当前视觉上下文中持续生成可行轨迹。实验过程中, 机器人能够沿走廊或开阔区域稳定前进, 在接近墙壁或障碍物前完成减

速，并由扩散策略生成绕行或转弯的局部 waypoint。这一行为模式与第三章关于 goal mask 的分析一致，说明探索模式在真机平台上同样得到了行为层面的验证。

5.9 本章小结

本章围绕 Orin 与 Lite3 的真机部署问题，对统一闭环系统在真实平台上的实现与验证进行了系统阐述。

第 6 章 全文总结与未来展望

6.1 全文工作总结

本文以 *visualnav-transformer* 项目为基础，以 NoMaD 为高层核心算法，以云深处 Lite3 四足机器人作为主要部署对象，围绕“基于动作扩散策略的足式机器人视觉目标导航与探索”展开了从算法到系统再到平台的完整研究。与只停留在离线仿真指标上的工作不同，本文始终把研究目标限定为算法、系统与平台能否形成可运行闭环，能否在真机实现部署。

围绕这一目标，本文主要完成了四方面工作。

第一，系统梳理了 NoMaD^[10] 的代码结构、训练目标与推理流程，明确了视觉编码器、距离预测头和扩散策略头之间的共享条件空间关系。在此基础上，本文进一步剖析了 NoMaD 采用扩散策略生成动作序列的核心机制，包括前向加噪、反向去噪的马尔可夫链采样过程，以及条件嵌入如何同时服务于距离估计与动作生成两个下游任务。这一梳理工作不仅为后续推理优化提供了清晰的模块边界，也为视觉编码器替换实验提供了可控的实验基线。

第二，围绕部署需求对高层推理进行了系统优化。具体而言，本文研究了四项关键技术：DDIM 推理加速通过缩减去噪步数在保持动作质量的同时大幅降低计算开销；CFG 引导增强通过调节无分类器引导强度改善目标条件响应；TTS 推理时搜索通过在采样阶段引入多候选评估与排序机制提升动作选择质量；视觉编码器升级通过将默认 EfficientNet-B0 替换为 DINOv2、ConvNeXt、ResNet 等候选编码器来比较视觉表征能力。本文为上述四项技术建立了统一的离线统计实验链路，使不同优化手段可以在同一评估框架下进行公平比较。这四项技术之间存在正交性：DDIM 主要影响推理速度，CFG 主要影响目标敏感度，TTS 主要影响动作多样性筛选，编码器升级主要影响视觉特征质量，因此它们可以组合使用以获得更优的综合性能。

第三，面向 Lite3 构建了完整的分层闭环系统。该系统由统一推理模块、有限状态机、导航主机、中层控制映射与平台接口五个层次组成。统一推理模块封装了 NoMaD 的前向推理与所有优化选项；有限状态机管理导航、恢复、停止等行为状态的切换逻辑；导航主机负责 topomap 节点选择与子目标规划；中层控制映射将高层航点动作转换为 Lite3 可执行的速度指令；平台接口则处理与 Lite3 底层 SDK 的通信协议。本文在 MuJoCo 仿真环境中对该系统进行了端到端验证，确认了从视觉输入到运动执行

的完整数据通路的正确性。分层设计的核心优势在于各层职责明确、接口标准化，使得单层替换或升级不会影响其他层的正常工作。

第四，形成了面向 Jetson Orin 与 Lite3 真机平台的完整部署流程。该流程涵盖了模型导出与推理环境配置、相机标定与图像预处理对齐、topomap 采集与管理、安全控制与异常恢复等关键环节。同时，本文为真实实验的图像记录、轨迹可视化和任务组织设计了规范化接口，确保实验结果的可追溯性和可复现性。

从阶段性结论看，当前默认算法框架确定为 EfficientNet-B0 视觉编码器、DDIM-2 采样、默认 CFG=0 和 TTS-8 候选筛选。DDIM-2 是当前最适合进入闭环验证的推理配置，它在速度与质量之间取得了较好平衡；CFG 更适合作为可调部署参数而非默认固定选项，其最优值依赖于具体场景的目标分布特性；TTS-8 能够以较低额外代价改善候选轨迹选择，TTS-16 可作为算力预算充足时的增强项；四个视觉编码器训练权重已经完成离线统计与 MuJoCo 闭环接入验证，但“最终部署最优编码器”的结论仍需结合真实相机分布、边缘设备时延和真机安全约束进一步确认。

6.2 主要贡献

本文的主要贡献可归纳为以下三点：(1) 建立了面向部署的 NoMaD 推理优化链路，将 DDIM、CFG、TTS 和视觉编码器替换纳入同一推理接口进行分析与控制。这一贡献的意义在于，现有工作通常只关注单一优化维度（如仅加速推理或仅替换编码器），缺乏将多种优化手段纳入统一框架进行系统评估的方法论。本文提出的统一链路使研究者能够在同一实验条件下量化不同优化组合的效果，为扩散策略在资源受限平台上的实用化部署提供了可操作的技术路径。(2) 构建了适用于 Lite3 的分层闭环系统，用状态机与导航主机把高层 NoMaD 策略组织成可恢复、可切换、可扩展的任务链路。这一贡献填补了从“扩散策略算法验证”到“足式机器人系统集成”之间的工程空白。与直接将策略输出映射为电机指令的端到端方案不同，分层架构保留了各层的可解释性和可调试性，使系统在真实部署中具备故障定位与在线调整能力。(3) 给出了 Orin 上的 Lite3 真机部署方案，并将图像记录、可视化与安全控制纳入统一工作流，提升了系统复现性。这一贡献为后续研究者提供了从算法到真机的完整参考实现，降低了足式视觉导航研究的实验门槛，同时规范化的实验管理流程也为跨平台迁移与多次重复实验奠定了基础。

6.3 研究不足

本文仍存在三方面不足，需要在后续工作中加以改进。

第一，视觉编码器实验虽然已经接入四个训练权重并完成离线统计与 MuJoCo 闭环比较，但闭环扫描中引入了 MuJoCo 域内的路线稳定项，因此该结果更适合证明四套权重均可进入系统闭环，而不能单独作为真实部署最优编码器的最终证据。最终选择仍需要结合 Jetson Orin 端真实推理时延、真实相机图像分布和真机安全实验来判断。

第二，真机实验的正式统计表和大规模重复实验尚未完整。虽然本文已在 Lite3 真机上完成了系统通路验证和初步导航任务，但受限于实验场地可用时间和设备调度约束，尚未积累足够数量的重复试验以支撑统计显著性分析。这导致第五章部分实验结果仍以定性描述和代表性案例为主，缺乏如成功率置信区间、轨迹偏差分布等定量统计指标。

第三，当前 topomap 仍主要依赖预先采集的视觉节点，尚未形成更强的在线更新与自维护能力。这一不足限制了系统在动态变化环境中的长期自主运行能力——当环境外观发生显著变化（如光照条件改变、障碍物移动）时，预采集的 topomap 节点可能失效，导致子目标匹配质量下降，进而影响导航成功率。

6.4 未来展望

后续工作可从以下方向继续推进：(1) 完成视觉编码器在真实部署条件下的最终训练对比与选择。具体而言，需要在 Jetson Orin 与 Lite3 真机上分别测量 EfficientNet-B0、DINOv2-Small、ConvNeXt-Tiny、ResNet-50 等候选的推理延迟、图像分布鲁棒性、导航成功率和失败模式，从而形成兼顾性能与效率的最终选择。(2) 补齐真机统计实验，建立离线指标与真机任务成功率之间的更明确对应关系。这要求设计标准化的实验场景集（包含不同难度等级的导航任务），在每个场景下执行足够次数的重复实验。(3) 将当前统一推理模块与状态机扩展到 Tron1 等更多平台，验证分层结构的可迁移性。Tron1 作为双足人形机器人，其运动学约束与 Lite3 有本质区别，若分层系统仅需替换中层控制映射即可适配，则可有力证明本文架构设计的通用性与可扩展性。(4) 引入语言目标与多模态条件，使系统从 image-goal 进一步扩展到更灵活的任务表达。结合视觉-语言模型将自然语言指令编码为目标条件，可以使机器人接受“去走廊尽头的红色门”等语义级导航指令，大幅提升系统在人机协作场景中的实用价值。

综合来看，本文已经把原本主要面向轮式平台验证的 NoMaD 高层视觉导航策略，扩展为一套面向 Lite3 四足平台的可分析、可仿真、可部署的研究链路：从算法

层面的扩散策略推理优化，到系统层面的分层闭环架构设计，再到平台层面的真机部署与实验管理。

参考文献

- [1] THRUN S, BURGARD W, FOX D. Probabilistic Robotics[M]. Cambridge, MA: MIT Press, 2005.
- [2] CAMPOS C, ELVIRA R, RODRÍGUEZ J J G, et al. ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM[J/OL]. IEEE Transactions on Robotics, 2021, 37(6): 1874-1890. DOI: 10.1109/TRO.2021.3075644.
- [3] HESS W, KOHLER D, RAPP H, et al. Real-Time Loop Closure in 2D LIDAR SLAM[C/OL]//2016 IEEE International Conference on Robotics and Automation (ICRA). 2016: 1271-1278. DOI: 10.1109/ICRA.2016.7487258.
- [4] ANDERSON P, CHANG A, CHAPLOT D S, et al. On Evaluation of Embodied Navigation Agents [J/OL]. arXiv preprint arXiv:1807.06757, 2018. <https://arxiv.org/abs/1807.06757>.
- [5] ZHU Y, MOTTAGHI R, KOLVE E, et al. Target-Driven Visual Navigation in Indoor Scenes Using Deep Reinforcement Learning[C/OL]//2017 IEEE International Conference on Robotics and Automation (ICRA). 2017: 3357-3364. DOI: 10.1109/ICRA.2017.7989381.
- [6] DOSOVITSKIY A, BEYER L, KOLESNIKOV A, et al. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale[C/OL]//International Conference on Learning Representations (ICLR). 2021. <https://openreview.net/forum?id=YicbFdNTTy>.
- [7] PEEBLES W, XIE S. Scalable Diffusion Models with Transformers[C/OL]//Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV). 2023: 4172-4182. DOI: 10.1109/ICCV51070.2023.00387.
- [8] SHAH D, SRIDHAR A, BHORKAR A, et al. GNM: A General Navigation Model to Drive Any Robot[C/OL]//2023 IEEE International Conference on Robotics and Automation (ICRA). 2023: 7226-7233. DOI: 10.1109/ICRA48891.2023.10161227.
- [9] SHAH D, SRIDHAR A, DASHORA N, et al. ViNT: A Foundation Model for Visual Navigation [C]//Proceedings of Machine Learning Research: Proceedings of the 7th Conference on Robot Learning (CoRL): vol. 229. PMLR, 2023.
- [10] SRIDHAR A, SHAH D, GLOSSOP C, et al. NoMaD: Goal Masked Diffusion Policies for Navigation and Exploration[C/OL]//2024 IEEE International Conference on Robotics and Automation (ICRA). 2024: 63-70. <https://arxiv.org/abs/2310.07896>.
- [11] CHI C, FENG S, DU Y, et al. Diffusion Policy: Visuomotor Policy Learning via Action Diffusion [C/OL]//Proceedings of Robotics: Science and Systems (RSS). 2023. DOI: 10.15607/RSS.2023.XIX.026.
- [12] HWANGBO J, LEE J, DOSOVITSKIY A, et al. Learning Agile and Dynamic Motor Skills for Legged Robots[J/OL]. Science Robotics, 2019, 4(26): eaau5872. DOI: 10.1126/scirobotics.aau5872.
- [13] LEE J, HWANGBO J, WELLHAUSEN L, et al. Learning Quadrupedal Locomotion over Challenging Terrain[J/OL]. Science Robotics, 2020, 5(47): eabc5986. DOI: 10.1126/scirobotics.abc5986.
- [14] SHAH D, EYSENBACH B, KAHN G, et al. ViNG: Learning Open-World Navigation with Visual Goals[C/OL]//2021 IEEE International Conference on Robotics and Automation (ICRA). 2021: 13215-13222. DOI: 10.1109/ICRA48506.2021.9561936.
- [15] WIJMANS E, KADIAN A, MORCOS A, et al. DD-PPO: Learning Near-Perfect PointGoal Navigators from 2.5 Billion Frames[C/OL]//International Conference on Learning Representations

- (ICLR). 2020. <https://openreview.net/forum?id=H1gX8C4YPr>.
- [16] SAVINOV N, DOSOVITSKIY A, KOLTUN V. Semi-Parametric Topological Memory for Navigation[C/OL]//International Conference on Learning Representations (ICLR). 2018. <https://openreview.net/forum?id=SygwwGbRW>.
- [17] CHAPLOT D S, SALAKHUTDINOV R, GUPTA A, et al. Neural Topological SLAM for Visual Navigation[C/OL]//Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). 2020: 12875-12884. DOI: 10.1109/CVPR42600.2020.01289.
- [18] HO J, JAIN A, ABBEEL P. Denoising Diffusion Probabilistic Models[C/OL]//Advances in Neural Information Processing Systems (NeurIPS): vol. 33. 2020. <https://arxiv.org/abs/2006.11239>.
- [19] SONG J, MENG C, ERMON S. Denoising Diffusion Implicit Models[C/OL]//International Conference on Learning Representations (ICLR). 2021. <https://openreview.net/forum?id=St1giarCHLP>.
- [20] JANNER M, DU Y, TENENBAUM J B, et al. Planning with Diffusion for Flexible Behavior Synthesis[C]//Proceedings of Machine Learning Research: Proceedings of the 39th International Conference on Machine Learning (ICML): vol. 162. PMLR, 2022: 9902-9915.
- [21] MIKI T, LEE J, HWANGBO J, et al. Learning Robust Perceptive Locomotion for Quadrupedal Robots in the Wild[J/OL]. Science Robotics, 2022, 7(62): eabk2822. DOI: 10.1126/scirobotics.abk2822.
- [22] KAREER S, YOKOYAMA N, BATRA D, et al. ViNL: Visual Navigation and Locomotion Over Obstacles[C/OL]//2023 IEEE International Conference on Robotics and Automation (ICRA). 2023: 2018-2024. DOI: 10.1109/ICRA48891.2023.10160612.
- [23] HO J, SALIMANS T. Classifier-Free Diffusion Guidance[J/OL]. arXiv preprint arXiv:2207.12598, 2022. <https://arxiv.org/abs/2207.12598>.
- [24] MA N, TONG S, JIA H, et al. Inference-Time Scaling for Diffusion Models beyond Scaling Denoising Steps[J/OL]. arXiv preprint arXiv:2501.09732, 2025. <https://arxiv.org/abs/2501.09732>.
- [25] ZHANG X, LIN H, YE H, et al. Inference-time Scaling of Diffusion Models through Classical Search[J/OL]. arXiv preprint arXiv:2505.23614, 2025. <https://arxiv.org/abs/2505.23614>.
- [26] VASWANI A, SHAZEER N, PARMAR N, et al. Attention is All You Need[C]//Advances in Neural Information Processing Systems (NeurIPS): vol. 30. 2017: 5998-6008.
- [27] TAN M, LE Q V. EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks[C]//Proceedings of Machine Learning Research: Proceedings of the 36th International Conference on Machine Learning (ICML): vol. 97. PMLR, 2019: 6105-6114.
- [28] OQUAB M, DARCET T, MOUTAKANNI T, et al. DINOv2: Learning Robust Visual Features without Supervision[J/OL]. Transactions on Machine Learning Research (TMLR), 2024. <https://openreview.net/forum?id=a68SUt6zFt>.
- [29] LIU Z, MAO H, WU C Y, et al. A ConvNet for the 2020s[C/OL]//Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). 2022: 11976-11986. DOI: 10.1109/CVPR52688.2022.01167.
- [30] HE K, ZHANG X, REN S, et al. Deep Residual Learning for Image Recognition[C/OL]//Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). 2016: 770-778. DOI: 10.1109/CVPR.2016.90.
- [31] TODOROV E, EREZ T, TASSA Y. MuJoCo: A Physics Engine for Model-Based Control[C/OL]//2012 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS). 2012: 5026-5033. DOI: 10.1109/IROS.2012.6386109.
- [32] RUDIN N, HOELLER D, REIST P, et al. Learning to Walk in Minutes Using Massively Parallel

Deep Reinforcement Learning[C]//Proceedings of Machine Learning Research: Proceedings of the 5th Conference on Robot Learning (CoRL): vol. 164. PMLR, 2022: 91-100.

致 谢

行文至此，本科阶段的最后一份“作业”即将完成。本科四年，我笑过、哭过，迷茫过、挣扎过，也坚定过。本科的最后半年，一切都在变好，科研上有了继续深耕的环境，生活也看见了未来的希望。

感谢导师在研究方向选择、论文写作和实验设计中给予的悉心指导与持续帮助。

感谢 NoMaD 项目团队开源的 visualnav-transformer 代码仓库，为本文的研究提供了坚实的算法基础。

感谢云深处科技提供的 Lite3 四足机器人平台与技术支持，使本文能够将视觉导航研究从仿真推进到真机验证。

感谢实验室师兄在日常讨论、代码调试与真机实验中给予的无私帮助。初入科研时，我曾倍感忐忑，师兄们耐心指点使我突破瓶颈，顺利完成本课题。

感谢上海交通大学自动化与感知学院提供的学习与科研条件。

感谢智能感知工程专业，作为专业首届本科生，我曾无数次地怀疑这个专业是否能够给本科生足够的培养，但四年的学习与实践让我见证了它的快速成长与扎实积淀。在此，我不仅收获了系统的专业知识与工程能力，更结识了良师益友。

感谢自感 2253 班级的兄弟姐妹们，感谢 2253 卷帘门。每逢期末的相互扶持与挑灯夜战，必将成为我青春岁月中最温暖的印记。

感谢挚友同行，四载交大岁月，因你们而熠熠生辉。

感谢家人一直以来的理解与支持。感谢爸爸妈妈，感谢你们二十多年的养育之恩，以及始终如一的理解与鼓励，是我前行路上最坚实的底气。

感谢我的女朋友，这半年来，在我最困顿迷茫之时，是你的陪伴与鼓励让我重拾方向；无数个疲惫的夜晚，因你的理解与支持而不再难熬。你的温柔与坚定，是我坚持至今的重要力量。

最后感谢一直走在路上的我自己，我没有被困难打倒，我没有被压力击溃，我一直在坚持，我一直在成长。我是好样的！愿我带着这份坚韧与热爱，继续奔赴下一程山海！

VISUAL GOAL NAVIGATION AND EXPLORATION FOR LEGGED ROBOTS BASED ON ACTION DIFFUSION POLICY

Background and Motivation

Visual goal navigation is a core task in embodied intelligence that connects perception with decision-making. Given the current visual observation and a goal image, the policy must output executable local motion without relying on high-precision maps or global localization, thereby achieving autonomous movement from the current viewpoint to the goal viewpoint. As robots are increasingly deployed in unstructured open-world environments such as campus patrol, indoor guidance, last-mile delivery and home service, navigation can no longer be reduced to a pure geometric path-planning problem on a known map. It must instead handle weak priors, partial observability, dynamic obstacles and ambiguous goal specifications, which has motivated a paradigm shift from classical SLAM-plus-planner pipelines toward learning-based, image-conditioned policies that consume raw visual observations directly.

Recently, general navigation models such as GNM, ViNT, and NoMaD have introduced diffusion policies into visual navigation, modelling the future local trajectory as a conditional distribution and generating multi-modal candidates through iterative denoising sampling. This formulation has significantly stronger expressive power than regression-based end-to-end policies that collapse multi-modal decisions (such as turning left or right at a T-junction) into a single mean output. By modelling the full distribution of feasible local trajectories, diffusion-based policies are able to preserve multi-modality, sample diverse candidates, and integrate naturally with test-time selection and guidance mechanisms. However, closed-loop validation of these models has so far concentrated on wheeled platforms (LoCoBot, TurtleBot, Jackal), while deployment pipelines targeting legged robots remain underexplored. Legged platforms differ substantially from wheeled ones in gait-induced jitter, control-loop frequency, body-posture coupling, foot-impact disturbance, and real-time safety requirements. These differences pose new challenges for the collaboration between high-level diffusion policies and mid-/low-level control interfaces, and they cannot be addressed by simply re-routing the wheeled-robot velocity command to a quadruped's locomotion controller.

Objectives and Approach

This thesis takes NoMaD as the high-level navigation core and the DeepRobotics Lite3 quadruped as the target deployment platform, and addresses three deployment-oriented questions: (i) how the visual encoder, distance-prediction head, and diffusion policy head are coupled in the NoMaD implementation and where inference-level optimizations are possible without retraining the model; (ii) whether, without retraining, sampling acceleration via DDIM, conditional guidance via Classifier-Free Guidance (CFG), test-time search via Test-Time Scaling (TTS), and visual encoder replacement can meaningfully improve policy usability for resource-constrained edge deployment; and (iii) whether these optimizations can be integrated into a hierarchical closed-loop system for Lite3 and validated through MuJoCo simulation as well as real-world deployment on the Jetson Orin onboard computer. The technical roadmap proceeds through algorithm-level optimization, system-level integration, and deployment-level validation, corresponding respectively to Chapters 3, 4, and 5 of the thesis. A deliberate emphasis is placed on bridging the offline-evaluation versus closed-loop deployment gap, ensuring that every algorithmic conclusion is re-examined under realistic timing, safety, and platform constraints.

Main Contributions

Algorithm layer. A unified offline statistical pipeline is built to compare DDIM inference acceleration, Classifier-Free Guidance (CFG) enhancement, Test-Time Scaling (TTS) search, and visual encoder upgrades under a single evaluation protocol. Concretely, DDIM-2 attains about $4.84\times$ speedup over the DDPM-10 baseline on the GoStanford evaluation set without notable trajectory-quality degradation, with the mean-squared deviation against the baseline trajectory distribution remaining below 0.022. The CFG analysis, derived from score-function modification, establishes a guidance-scale selection rule and shows that CFG roughly doubles the per-step compute while delivering only marginal Forward-Progress gain in the diffusion-policy regime; hence CFG is best treated as a tunable knob rather than a default-on component. The TTS analysis decomposes total gain into a nearly zero-latency *scoring-based selection* component (which re-ranks the eight candidates already produced by a single diffusion batch) and a linearly priced but rapidly diminishing *candidate-pool expansion* component (which spawns extra batches to enlarge the pool). TTS-8 wins the

three-factor joint ranking at minimal cost, providing a novel interpretation of inference-budget allocation as “re-rank cheaply first, expand only when necessary.” Encoder upgrades use a two-stage framework to compare EfficientNet-B0, ResNet-50, DINOv2-Small, and ConvNeXt-Tiny on identical offline cases; the encoder $\times$ DDIM / CFG / TTS joint ablation establishes that DDIM-2 and CFG default-off remain robust across all four encoders, while TTS gain scales with the initial candidate diversity of the chosen encoder.

System layer. A hierarchical closed-loop system—comprising a unified inference module, an explicit finite-state machine, a navigation host, a mid-level waypoint-to-velocity mapping, and a platform interface—supports both MuJoCo and real-robot backends through one set of high-level code. The state machine (idle / stand / navigate / explore / recovery / estop / completed / failed) is decoupled from the physical platform via a uniform interface, so that switching between MuJoCo and Lite3 only changes the backend implementation rather than the high-level logic. The navigation host supports both batched JSON plans and interactive single commands, allowing capture, topomap building, navigation, and exploration to be composed within a single experiment session. Mid-level mapping converts the diffusion policy’s local waypoints into bounded forward, lateral, and yaw velocity commands, respecting Lite3-specific safety envelopes ($v_x \leq 0.4$ m/s in simulation, $v_x \leq 0.12$ m/s during real-robot deployment). In MuJoCo closed-loop experiments, Lite3 successfully completes point navigation, multi-goal navigation, and free exploration on easy / medium / hard maps, reaching every goal across repeated runs. The same unified inference module that handles the four offline optimizations is re-used at deployment time, demonstrating that the algorithm-layer findings translate directly into operational system behaviour.

Deployment layer. For the Jetson Orin onboard host and the Lite3 upper-computer link, the thesis develops a staged commissioning procedure, multi-level safety mechanisms, and an image-recording convention. Stage 1 (Orin standalone commissioning) validates camera capture, model loading, inference benchmarking, and the end-to-end pipeline in isolation from the robot, so that any inference-side defect is discovered before the platform is energized. Stage 2 (Orin and Lite3 joint commissioning) progresses from network and heartbeat verification, through standup and low-speed twist tests, to interactive and goal-directed navigation under conservative speed limits. The bridging layer wraps the Lite3 SDK with a 25 Hz velocity-refresh thread and a 5 Hz heartbeat thread, repeating the most recent clipped

velocity between higher-level NoMaD inference cycles to guarantee continuous control flow even when the policy runs at single-digit hertz. Multi-level safety guards include NaN/Inf clipping, magnitude limits, fallen-robot detection, and a connection-stale threshold of two seconds; any inference anomaly or platform fault is routed to a soft emergency stop. Real-robot experiments on multiple indoor scenes with pre-collected topomaps record closed-loop frequencies of 2.89 Hz (DDPM baseline), 6.80 Hz (DDIM-2 + CFG=0 + TTS-8), 3.43 Hz (DDIM-2 + CFG=2 + TTS-8), and 5.90 Hz (DDIM-3 + CFG=0, no TTS), with a 100% success rate across all four configurations under the present conservative speed setting. The frequency ranking is fully consistent with the offline analysis in Chapter 3: DDIM step count dominates latency, CFG approximately halves the achievable rate, and TTS-8’s overhead is negligible because re-ranking eight already-generated candidates does not require an extra diffusion batch.

Experiments and Conclusions

Offline results indicate that DDIM-2 plus optional CFG and configurable TTS-8 is the most promising configuration to bring into closed-loop validation, and the encoder upgrade pipeline supplies a reusable framework for future visual-backbone exploration rather than a single best-encoder verdict. MuJoCo experiments confirm that the hierarchical system can robustly complete quadruped navigation and exploration tasks, that multi-goal task switching preserves visual context, and that the recovery sub-state successfully unblocks the robot from corridor-corner stalls without requiring task abortion. Stage-1 validation on Jetson Orin demonstrates the baseline feasibility of the real-robot pipeline (about 7 Hz benchmarked NoMaD inference, $\approx 151\text{--}157$ ms steady-state pipeline cycle once warm-up cost is amortized), and Stage-2 multi-scene results provide the first real-robot evidence that diffusion-policy-based image-goal navigation can run end-to-end on a Lite3 quadruped at single-digit hertz while still completing every commissioned trial. Crucially, the fact that even the slow 2.89 Hz baseline achieves 100% success demonstrates that the closed-loop system has substantial robustness margin: the conservative 0.12 m/s speed cap means each cycle’s displacement is only about 4 cm—far smaller than the topomap node spacing—so high-level latency tolerance is absorbed by the topomap sliding window and the 25 Hz bridge replay rather than manifesting as catastrophic drift.

Limitations and Future Work

Three limitations are explicitly acknowledged. First, the encoder upgrade conclusions remain offline statistics plus simulator validation, and a final real-robot-conditioned encoder ranking will require benchmarking inference latency, image-distribution robustness, and failure modes directly on the Orin/Lite3 stack with real cameras. Second, the present real-robot statistics, while sufficient to show that all four inference configurations succeed under the current scenario scale, do not yet include large-sample success-rate confidence intervals or formal trajectory-deviation distributions; expanding the experiment matrix to outdoor scenes, dynamic obstacles, and longer corridors should enhance differentiation between configurations and unlock the latency margin that the fastest scheme has reserved. Third, the topomap is still collected offline by an operator; an online maintenance loop that refreshes nodes when the environment changes would improve long-term autonomy. Future work will therefore focus on (i) finalizing the on-robot encoder choice with end-to-end real-world metrics, (ii) running large-scale repeated trials with formal statistics, (iii) generalizing the unified inference module and state machine to additional platforms such as the Tron1 humanoid robot to test cross-embodiment portability, and (iv) extending image-goal conditioning to language-grounded targets, allowing operators to issue natural-language commands such as “go to the red door at the end of the corridor” that get translated into NoMaD-compatible visual conditions.

The thesis extends NoMaD from a wheeled-platform research framework into an analyzable, simulatable, and deployable research pipeline for quadruped platforms, distilling a reusable inference-optimization protocol, a hierarchical closed-loop system organization, and a real-robot safety-guard checklist. Together, these artefacts provide a practical basis for subsequent research on diffusion-policy-based visual navigation for legged robots, and they lower the engineering threshold for follow-up work that wishes to validate algorithmic ideas under genuine on-robot conditions rather than only on offline benchmarks.